



Universidad de Valladolid

**Escuela de Ingeniería Informática de
Valladolid**

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

Mención en Ingeniería de Software

**Diseño y desarrollo de un videojuego de
estrategia por turnos con Godot**

Autor:

D. Sergio Alcón González

Tutores:

**D. Luis Ignacio Jiménez Gil
D. Mario Corrales Astorgano**

Resumen

Este Trabajo Fin de Grado presenta el diseño e implementación de *NeuroForge*, un videojuego de estrategia por turnos con mecánicas de información oculta desarrollado sobre el motor Godot. El videojuego está inspirado en el clásico juego de mesa *Stratego*, adaptando sus mecánicas fundamentales a un entorno digital con ambientación de ciencia ficción futurista.

El proyecto abarca el ciclo de vida completo del desarrollo de software, desde el análisis de requisitos y el diseño de la arquitectura hasta la implementación, las pruebas y la documentación. La arquitectura adoptada separa en tres capas la lógica del videojuego, la interfaz de usuario y la infraestructura, lo que ha facilitado el mantenimiento y la escalabilidad del sistema a lo largo de las cinco iteraciones de desarrollo. Se han aplicado patrones de diseño orientados a objetos como *Singleton*, *Strategy*, *Fachada* y *Template Method* para garantizar una estructura robusta y desacoplada.

Como componente avanzado, se ha desarrollado un agente de inteligencia artificial entrenado mediante aprendizaje por refuerzo, utilizando la biblioteca Gymnasium como interfaz del entorno y el algoritmo MaskablePPO como método de entrenamiento. El agente aprende a jugar mediante una estrategia de *self-play*, enfrentándose progresivamente a versiones anteriores de sí mismo, lo que le permite desarrollar comportamientos estratégicos sin necesidad de una función de evaluación diseñada manualmente.

El resultado es un videojuego completo y jugable, con una interfaz visual pulida, efectos sonoros, múltiples menús y un oponente controlado por inteligencia artificial, distribuible como ejecutable autocontenido para Windows.

Palabras clave: videojuego, estrategia por turnos, información oculta, Godot, aprendizaje por refuerzo, Gymnasium, MaskablePPO, *self-play*, arquitectura en capas, patrones de diseño.

Abstract

This Bachelor's Thesis presents the design and implementation of *NeuroForge*, a turn-based strategy video game with hidden information mechanics developed using the Godot engine. The game is inspired by the classic board game *Stratego*, adapting its core mechanics to a digital environment with a futuristic science fiction setting.

The project covers the complete software development lifecycle, from requirements analysis and architectural design to implementation, testing, and documentation. The adopted architecture separates game logic, user interface, and infrastructure into three layers, which has facilitated system maintainability and scalability throughout the five development iterations. Object-oriented design patterns such as *Singleton*, *Strategy*, *Facade*, and *Template Method* have been applied to ensure a robust and decoupled structure.

As an advanced component, an artificial intelligence agent has been developed and trained using reinforcement learning, employing the Gymnasium library as the environment interface and the MaskablePPO algorithm as the training method. The agent learns to play through a self-play strategy, progressively competing against earlier versions of itself, which allows it to develop strategic behaviors without the need for a manually designed evaluation function.

The result is a complete and playable video game, featuring a polished visual interface, sound effects, multiple menus, and an AI-controlled opponent, distributable as a self-contained executable for Windows.

Keywords: video game, turn-based strategy, hidden information, Godot, reinforcement learning, Gymnasium, MaskablePPO, self-play, layered architecture, design patterns.

Índice general

1	Introducción	5
1.1	Contexto y motivaciones	5
1.2	Objetivos	6
1.3	Estructura del documento	7
2	Antecedentes	9
2.1	Evolución de los juegos de estrategia	9
2.2	El juego original: <i>Stratego</i>	13
2.3	Variaciones, adaptaciones y videojuegos relacionados	14
2.4	Motores de desarrollo de videojuegos	16
2.5	Inteligencia artificial en videojuegos de estrategia	17
3	Diseño del videojuego	19
3.1	Resumen del videojuego	19
3.2	Mecánicas	20
3.2.1	Tablero	20
3.2.2	Colocación inicial	20
3.2.3	Piezas	21
3.2.4	Turnos	22
3.2.5	Resolución de combates	22
3.2.6	Condiciones de victoria	23
3.2.7	Interacción y control	23
3.3	Ambientación	24

3.3.1	Escenario	24
3.3.2	Unidades y estructuras	25
3.4	Diseño de niveles	25
3.5	Audiencia	26
4	Planificación	27
4.1	Metodología empleada	27
4.1.1	Etapas de desarrollo y ordenación en la planificación	28
4.2	Planificación inicial	29
4.2.1	Tareas Globales Iniciales: Análisis y Diseño	29
4.2.2	Iteración 1: Núcleo jugable (Videojuego base)	30
4.2.3	Iteración 2: Interfaz de usuario	31
4.2.4	Iteración 3: Mejoras audiovisuales	31
4.2.5	Iteración 4: Implementación del bot inteligente	32
4.2.6	Iteración 5: Pulido final	32
4.3	Riesgos	36
4.4	Restricciones	37
4.5	Herramientas	37
4.5.1	Selección del Motor de Videojuegos	38
4.5.2	Otras herramientas utilizadas	38
4.6	Variación de planificación	40
4.7	Costes del proyecto	42
4.7.1	Costes de personal	42
4.7.2	Costes de software	42
4.7.3	Costes de infraestructura	43
4.7.4	Distribución en Steam	43
4.7.5	Resumen y punto de amortización	44
5	Análisis	45
5.1	Actores y roles	45

5.2	Modelado de Requisitos	46
5.2.1	Requisitos funcionales	46
5.2.2	Requisitos no funcionales	47
5.2.3	Requisitos de información	47
5.3	Casos de uso	48
5.3.1	Lista de casos de uso	49
5.3.2	CU-01: Iniciar partida	49
5.3.3	CU-02: Jugar ronda	50
5.3.4	CU-03: Resolver combate	51
5.3.5	CU-04: Pausar partida	52
5.3.6	CU-05: Abandonar partida	53
5.3.7	CU-06: Consultar reglas	53
5.3.8	CU-07: Consultar opciones	54
5.4	Modelo de dominio	55
5.5	Modelo de proceso de negocio	56
5.5.1	Actividad: Iniciar partida	56
5.5.2	Actividad: Jugar ronda	57
5.5.3	Actividad: Resolver combate	58
6	Diseño	59
6.1	Arquitectura del sistema	59
6.1.1	El modelo de ejecución de Godot	59
6.1.2	Arquitectura de NeuroForge	62
6.2	Patrones de diseño aplicados	63
6.2.1	Singleton	63
6.2.2	Strategy	64
6.2.3	Fachada	65
6.2.4	Template Method	65
6.3	Diseño de interfaz de usuario	66

6.3.1	Partida en curso	69
6.3.2	Interfaz de combate	70
6.3.3	Fin de partida	71
6.3.4	Menú de pausa	72
6.3.5	Menú de opciones	73
6.4	Diagrama de despliegue	74
7	Implementación	75
7.1	Organización del código	75
7.2	Licencias requeridas	76
7.2.1	Motor de Videojuegos: Godot Engine	76
7.2.2	Recursos de Audio	77
7.2.3	Recursos Gráficos	77
7.2.4	Modificación de recursos	77
7.3	Detalles de implementación relevantes	78
7.3.1	Arquitectura del bot	78
8	Pruebas	81
8.1	Enfoque de pruebas	81
8.2	Batería de pruebas	82
8.2.1	Bloque 1 — Despliegue inicial	82
8.2.2	Bloque 2 — Movimiento	83
8.2.3	Bloque 3 — Combate	84
8.2.4	Bloque 4 — Condiciones de victoria	85
8.2.5	Bloque 5 — Interfaz y feedback visual/sonoro	86
8.2.6	Bloque 6 — Menús y navegación	87
8.2.7	Bloque 7 — Comportamiento del bot	88
8.2.8	Bloque 8 — Requisitos no funcionales	89
8.3	Pruebas de usabilidad	90
8.3.1	Cuestionario de evaluación	91

8.4	Pruebas automatizadas con NUnit	94
9	Seguimiento del proyecto	97
9.1	Iteración 1: Núcleo jugable y lógica básica	97
9.2	Iteración 2: Interfaz de usuario y pulido visual	100
9.3	Iteración 3: Mejoras audiovisuales	106
9.4	Iteración 4: Implementación del bot inteligente	106
9.5	Iteración 5: Pulido final	107
10	Conclusiones	109
10.1	Valoración de objetivos	109
10.2	Uso de inteligencia artificial	110
10.3	Líneas futuras	110
A	Manual de instalación	113
A.1	Requisitos del sistema	113
A.2	Procedimiento de ejecución	114
A.3	Solución de problemas comunes	114
B	Manual de usuario	115
B.1	Menú principal	115
B.2	Menú de reglas	116
B.3	Fase de despliegue	116
B.4	Partida en curso	118
B.5	Interfaz de combate	119
B.6	Menú de pausa	120
B.7	Menú de opciones	120
B.8	Fin de partida	120
	Bibliografía	121

Índice de figuras

2.1	Tablero del Juego Real de Ur	10
2.2	Tablero del juego Senet	10
2.3	Tablero del juego Dou Shou Qi	12
2.4	Tablero del juego Luzhanqi	14
3.1	Esquema del tablero	20
3.2	Esquema del tablero	23
4.1	Metáfora gráfica del desarrollo iterativo e incremental	28
4.2	Parte 1 del diagrama de Gantt general del desarrollo del TFG	34
4.3	Parte 2 del diagrama de Gantt general del desarrollo del TFG	35
5.1	Diagrama de casos de uso	48
5.2	Diagrama del modelo de dominio	55
5.3	Diagrama de actividad: Iniciar partida	56
5.4	Diagrama de actividad: Jugar ronda	57
5.5	Diagrama de actividad: Resolver combate	58
6.1	Estructura de la escena <code>Tile</code> en el editor de Godot	60
6.2	Modelo de ejecución de Godot ilustrado con algunos nodos del proyecto.	61
6.3	Vista simplificada de la arquitectura de NeuroForge en tres capas.	62
6.4	Patrón Singleton aplicado en <code>GameScene</code> y <code>SceneManager</code>	64
6.5	Patrón Strategy con <code>MovementSystem</code> y <code>CombatSystem</code> como estrategias delegadas desde <code>Board</code>	64
6.6	Patrón Fachada con <code>Board</code> como punto de acceso unificado al subsistema de tablero.	65

6.7	Diagrama de navegación de las interfaces de NeuroForge.	66
6.8	Menú principal de NeuroForge.	66
6.9	Menú de reglas con la pestaña de piezas activa.	67
6.10	Fase de despliegue con algunas piezas ya colocadas en la zona del jugador.	68
6.11	Pantalla de partida con una pieza seleccionada y sus acciones posibles resaltadas. .	69
6.12	Interfaz de combate tras la resolución: pieza perdedora oscurecida y mensaje de resultado visible.	70
6.13	Pantalla de fin de partida con mensaje de victoria.	71
6.14	Menú que emerge al pausar durante una partida.	72
6.15	Menú de opciones donde configurar audio y resolución.	73
6.16	Diagrama de despliegue de NeuroForge.	74
8.1	Resultado de la ejecución de las pruebas unitarias con NUnit.	96
9.1	Detalle de la interfaz de despliegue inicial basada en botones de texto.	98
9.2	Mensaje de final de partida en la Iteración 1	99
9.3	Capturas del juego durante la Iteración 1.	99
9.4	Primer boceto de la interfaz de partida con indicadores laterales y texto de turno. .	100
9.5	Evolución del diseño de partida con indicadores visuales de turno.	101
9.6	Layout definitivo del boceto de la interfaz de partida.	102
9.7	Boceto de inicio de combate: presentación de piezas.	103
9.8	Boceto de resolución: comparación de rangos.	103
9.9	Boceto de fin de combate: pieza eliminada.	104
9.10	Comparativa entre el diseño inicial de combate y el diseño final simplificado. . . .	104
9.11	Boceto del menú de pausa de la partida.	105
9.12	Boceto de la interfaz de reglas con sistema de navegación por viñetas.	105
B.1	Menú principal de NeuroForge.	115
B.2	Fase de despliegue con algunas piezas ya colocadas en la zona del jugador.	117
B.3	Pantalla de partida con una pieza seleccionada y sus acciones posibles resaltadas. .	118

B.4	Interfaz de combate tras la resolución: pieza perdedora oscurecida y mensaje de resultado visible.	119
B.5	Pantalla de fin de partida con mensaje de victoria.	120

Índice de tablas

2.1	Comparativa entre motores de desarrollo de videojuegos	17
4.1	Correspondencia entre las fases de la metodología y la ordenación de las tareas en la planificación	29
4.2	Distribución ajustada de horas por tareas globales e iteraciones del proyecto	33
4.3	Riesgos identificados y planes de contingencia	36
4.4	Herramientas empleadas en el desarrollo del proyecto	40
4.5	Variaciones en fechas de finalización y desviación respecto a la planificación inicial	41
4.6	Desglose de costes estimados en escenario individual y empresarial	44
5.1	Lista de casos de uso del Sistema	49
5.2	CU-01: Iniciar partida	49
5.3	CU-02: Jugar ronda	50
5.4	CU-03: Resolver combate	51
5.5	CU-04: Pausar partida	52
5.6	CU-05: Abandonar partida	53
5.7	CU-06: Consultar reglas	53
5.8	CU-07: Consultar opciones	54
7.1	Hiperparámetros del modelo MaskablePPO tras la búsqueda con Optuna	79
8.1	Pruebas del bloque 1: Despliegue inicial	82
8.2	Pruebas del bloque 2: Movimiento	83
8.3	Pruebas del bloque 3: Combate	84
8.4	Pruebas del bloque 4: Condiciones de victoria	85

8.5	Pruebas del bloque 5: Interfaz y feedback	86
8.6	Pruebas del bloque 6: Menús y navegación	87
8.7	Pruebas del bloque 7: Comportamiento del bot	88
8.8	Pruebas del bloque 8: Requisitos no funcionales	89
8.9	Pruebas de usabilidad	90
8.10	Matriz de frecuencias y distribución de respuestas del cuestionario de evaluación. .	92

Agradecimientos

En primer lugar, me gustaría expresar mi más sincero agradecimiento a mis tutores, Luis Ignacio y Mario Corrales, por haber aceptado desde el principio mi propuesta de desarrollar un videojuego como Trabajo Fin de Grado. Desarrollar un videojuego no es fácil y es algo que Ignacio me dejó claro desde el primer día. Aprecio mucho su confianza al no tomarme por cualquiera que se piensa que hacer un videojuego por muy tonto que sea es fácil. Sobre todo aprecio la constante orientación, feedback y recomendaciones que me han ido dando ambos tutores a lo largo de desarrollo de este Trabajo Fin de Grado.

También me gustaría agradecer a todos los compañeros, y ahora amigos, con los que he cursado a lo largo de todo el periodo académico, ya que sin ellos esta carrera habría sido un camino mas desagradable y poco ameno. Sin duda son lo mejor que me llevo de esta carrera junto a los conocimientos adquiridos.

Y por último y mas importante, mi mas profundo agradecimiento a mis padres por haberme brindado la oportunidad, los recursos, tiempo y apoyo necesarios para llevar a cabo esta carrera a lo largo de los últimos años. Pese a haber tenido de los peores comienzos en toda mi trayectoria académica, siguieron apoyándome y confiando en mi. Son a los que mas agradecidos estoy y estaré durante el resto de mi vida.

Glosario

API	Application Programming Interface (Interfaz de Programación de Aplicaciones)
CPU	Central Processing Unit (Unidad Central de Procesamiento)
CU	Caso de Uso
DQN	Deep Q-Network
EXE	Executable (Archivo Ejecutable de Windows)
FOGASA	Fondo de Garantía Salarial
GPU	Graphics Processing Unit (Unidad de Procesamiento Gráfico)
HUD	Heads-Up Display
IA	Inteligencia Artificial
It.	Iteración
MaskablePPO	Maskable Proximal Policy Optimization
MCTS	Monte Carlo Tree Search (Búsqueda de Árboles de Montecarlo)
MEI	Mecanismo de Equidad Intergeneracional
OpenGL	Open Graphics Library
PMBOK	Project Management Body of Knowledge
PPO	Proximal Policy Optimization
PVPC	Precio Voluntario para el Pequeño Consumidor
RAM	Random Access Memory (Memoria de Acceso Aleatorio)
RF	Requisito Funcional
RI	Requisito de Información
RL	Reinforcement Learning (Aprendizaje por Refuerzo)
RNF	Requisito No Funcional

TFG	Trabajo Fin de Grado
UCB	Upper Confidence Bound
UI	User Interface (Interfaz de Usuario)
UVa	Universidad de Valladolid
ZIP	Formatos de Almacenamiento y Compresión de Datos

Introducción

Este documento contiene la memoria del trabajo fin de grado (TFG), adscrito a la mención en Ingeniería de Software, que trata el diseño y la implementación de *NeuroForge*, un videojuego táctico por turnos caracterizado por mecánicas de información oculta, desarrollado sobre el motor Godot y el lenguaje C#. Asimismo, el proyecto incluye el diseño y entrenamiento de un agente de inteligencia artificial avanzado para ofrecer un componente de juego en solitario competitivo. Como punto de partida de la memoria, este primer capítulo introduce el proyecto a través del contexto de la industria y las motivaciones que justifican el desarrollo del software, y también se delimitan los objetivos técnicos fijados y se detalla la estructura organizativa que conforma el resto del documento.

1.1. Contexto y motivaciones

Los videojuegos constituyen hoy en día una de las industrias culturales y de entretenimiento más relevantes a nivel global. Los ingresos globales del sector alcanzaron los 188.800 millones de dólares en 2025, con una base de jugadores que supera los 3.600 millones de personas, lo que equivale al 61,5 % de la población mundial con acceso a internet [1]. Estas cifras sitúan a los videojuegos por encima de los ingresos combinados del *streaming* y la industria cinematográfica, consolidando su posición como el medio de entretenimiento dominante del siglo XXI.

Más allá de su peso económico, los videojuegos han demostrado tener un impacto positivo documentado en sus jugadores. Diversas investigaciones señalan que su uso moderado mejora capacidades cognitivas como la atención, la memoria de trabajo, la toma de decisiones y las habilidades espaciales [2]. Los videojuegos de estrategia, en particular, fomentan la autorregulación y la planificación, obligando al jugador a evaluar riesgos y anticipar movimientos bajo presión. Además, el componente interactivo y social de muchos títulos contribuye al desarrollo de valores como la cooperación y el trabajo en equipo [3]. El videojuego, por tanto, no es únicamente un producto de ocio, sino un medio expresivo con capacidad de influir de forma significativa en quienes lo experimentan.

Esta capacidad de generar experiencias entretenidas o memorables es precisamente lo que ha motivado el desarrollo del presente trabajo. A lo largo de los años, numerosos videojuegos han dejado una huella duradera, títulos que, independientemente de su presupuesto o escala, han conseguido transmitir emociones, plantear desafíos estimulantes o simplemente brindar momentos de disfrute genuino.

Poder contribuir, aunque sea a pequeña escala, a generar ese mismo tipo de experiencia en otras personas es la motivación principal de este proyecto. El desarrollo de un videojuego propio supone además una oportunidad única de adquirir de forma práctica las competencias técnicas y creativas que conforman esta industria, como el diseño de sistemas interactivos, implementación de lógica del videojuego, creación de interfaces y producción de contenido audiovisual.

1.2. Objetivos

El objetivo principal de este TFG es diseñar y desarrollar un videojuego de estrategia por turnos tipo *Stratego* completamente funcional utilizando el motor Godot y el lenguaje de programación C#, capaz de ofrecer una experiencia de juego completa y satisfactoria de principio a fin.

Para alcanzar este objetivo principal, se han definido los siguientes objetivos específicos:

- Diseñar e implementar la lógica fundamental del videojuego, en concreto, la estructura de un tablero, la definición de las piezas y las reglas de movimiento y combate entre ellas.
- Desarrollar un sistema de control de turnos y gestión del estado del videojuego que permita ejecutar partidas completas respetando las reglas definidas.
- Adquirir experiencia práctica en el uso del motor de videojuegos Godot y en el desarrollo de sistemas interactivos con C#.
- Implementar un bot que implemente técnicas de inteligencia artificial capaz de analizar el estado del tablero y seleccionar los movimientos válidos más adecuados para cada situación, ofreciendo al jugador la posibilidad de disputar partidas contra un oponente desafiante.

1.3. Estructura del documento

Este documento se encuentra estructurado en nueve capítulos principales que guían al lector a través de todas las fases de concepción, análisis, diseño, planificación, implementación y validación del proyecto. A continuación se presenta una descripción de cada uno de ellos:

- **Capítulo 1: Introducción.** Define el contexto y las motivaciones del proyecto, establece los objetivos generales y específicos a alcanzar y describe la estructura organizativa del documento.
- **Capítulo 2: Antecedentes.** Realiza un recorrido por la evolución histórica de los juegos de estrategia, desde los primeros juegos de mesa de la antigüedad hasta *Stratego* y sus adaptaciones digitales, contextualizando además los motores de videojuegos y los algoritmos de inteligencia artificial empleados en el proyecto.
- **Capítulo 3: Descripción del juego.** Detalla la propuesta del videojuego, definiendo sus mecánicas fundamentales, las reglas del tablero y las piezas, la ambientación y el perfil de la audiencia objetivo.
- **Capítulo 4: Planificación.** Describe la metodología de desarrollo, la planificación, el análisis de riesgos y restricciones, la justificación de las herramientas seleccionadas, las variaciones respecto al plan inicial y el análisis de costes del proyecto.
- **Capítulo 5: Análisis.** Contiene la especificación formal de los requisitos funcionales, no funcionales y de información, complementada con el modelo de dominio, los casos de uso y los diagramas de actividad correspondientes.
- **Capítulo 6: Diseño.** Explica la arquitectura del sistema y el modelo de ejecución de Godot, el diseño detallado de clases y componentes, el diseño de la interfaz de usuario y los patrones de diseño orientados a objetos aplicados.
- **Capítulo 7: Implementación.** Describe la organización del código fuente, la gestión de licencias de los recursos empleados y los detalles técnicos del módulo de inferencia del bot inteligente.
- **Capítulo 8: Pruebas.** Presenta la estrategia de verificación adoptada y una batería de pruebas funcionales manuales estructurada en bloques temáticos que cubren los requisitos del sistema, así como las pruebas de usabilidad con usuarios.
- **Capítulo 9: Seguimiento del proyecto.** Detalla el transcurso real y el progreso del desarrollo a lo largo de las cinco iteraciones completadas, ilustrando la evolución visual y los cambios de diseño del software.
- **Capítulo 10: Conclusiones.** Recoge una valoración del cumplimiento de los objetivos fijados, el análisis del uso de herramientas de inteligencia artificial durante el desarrollo y las líneas de trabajo futuro propuestas para la evolución de la aplicación.

Finalmente, tras las conclusiones, se incluye la bibliografía consultada con todas las fuentes referenciadas y los anexos correspondientes al manual de instalación y al manual de usuario del videojuego.

Antecedentes

En este capítulo se analizan los antecedentes relacionados con el TFG. Para ello, se realiza un recorrido por la evolución de los juegos de estrategia desde sus orígenes en los juegos de mesa clásicos hasta sus adaptaciones modernas, tanto físicas como digitales. Este análisis permite identificar las mecánicas fundamentales que han influido en el diseño del proyecto, así como contextualizar las decisiones adoptadas durante su desarrollo. Asimismo, se examina el estado del arte de los motores de desarrollo de videojuegos, justificando la elección del software base, y se revisan los enfoques y algoritmos de inteligencia artificial aplicados históricamente en este género para fundamentar el diseño del agente inteligente del sistema.

2.1. Evolución de los juegos de estrategia

Desde la antigüedad, los juegos de mesa han servido como una representación lúdica del conflicto, la planificación y la toma de decisiones. En las primeras civilizaciones, estos juegos cumplían una función tanto recreativa como cultural, y sentaron las bases de conceptos estratégicos que perduran hasta la actualidad.

Entre los ejemplos más antiguos se encuentra el *Juego Real de Ur*, originario de Iraq (aprox. del 2600-2400 a.C.) [4], presentaba un tablero dividido en secciones conectadas, tal y como se muestra en la Figura 2.1. Este diseño introducía decisiones tácticas relacionadas con el avance y la protección de las piezas, reforzando la importancia de la planificación a medio plazo. Ambos juegos, pese a su dependencia parcial del azar, contribuyeron a establecer una base conceptual sobre la que evolucionarían juegos de estrategia más complejos.

Aunque no se ha conservado un reglamento completo que permita reconstruir con total precisión las normas del juego, sí existe una descripción parcial recogida en una tablilla cuneiforme de origen babilonio, escrita entre los años 177 y 176 a.C. por el escriba Itti-Marduk-Balāṭu [5]. A partir de este testimonio y de los tableros conservados, se considera que el Juego Real de Ur consistía en una competición entre dos jugadores en forma de carrera a lo largo del tablero, con mecánicas comparables a juegos actuales como el parchís o el backgammon.



Figura 2.1: Tablero del Juego Real de Ur

De forma similar, el juego egipcio *Senet*, documentado arqueológicamente durante el Imperio Nuevo, con ejemplares datados en el reinado de Amenhotep III (aprox. 1390–1353 a.C.) [6], se desarrollaba sobre un tablero de treinta casillas, como se observa en la Figura 2.2. El desplazamiento de las piezas se determinaba mediante el lanzamiento de palos de conteo o huesos, introduciendo un componente de azar en la cantidad de casillas que podían avanzarse en cada turno. No obstante, el jugador conservaba un grado significativo de control estratégico, ya que podía decidir qué pieza mover, bloquear el avance del oponente o forzar retrocesos en determinadas situaciones. Esta combinación de aleatoriedad en la generación de movimientos y toma de decisiones tácticas en su aplicación convierte a *Senet* en uno de los primeros ejemplos conocidos de juego que equilibra azar y planificación. Aunque su mecánica era relativamente simple, introducía ya la idea de progresión de piezas y competición directa entre jugadores, aspectos clave en el desarrollo temprano del pensamiento estratégico [7].



Figura 2.2: Tablero del juego Senet

Ambos juegos, pese a su dependencia parcial del azar, contribuyeron a establecer una base conceptual sobre la que evolucionarían juegos de estrategia más complejos.

Con el avance del tiempo, los juegos de estrategia comenzaron a abandonar progresivamente la dependencia del azar para centrarse en la planificación deliberada y la toma de decisiones informadas. En este contexto surge el *Ajedrez*, cuyo origen se sitúa en la India alrededor del siglo VI d.C., a partir del juego conocido como *Chaturanga* [8]. A diferencia de los juegos antiguos basados en carreras o desplazamientos condicionados por el azar, el ajedrez introduce una jerarquía claramente definida de piezas con capacidades diferenciadas y un sistema de captura directa. Estas características refuerzan conceptos como el sacrificio estratégico, el control del espacio y la planificación táctica a largo plazo. Asimismo, el ajedrez se caracteriza por ofrecer información completa a ambos jugadores, lo que lo convierte en un paradigma de la estrategia basada exclusivamente en la anticipación y el cálculo.

Dentro de esta evolución resulta especialmente relevante el juego chino *Dou Shou Qi*, también conocido como *Selva* o *Ajedrez animal*. A diferencia de juegos clásicos cuya antigüedad está bien documentada, el origen histórico de *Dou Shou Qi* no se encuentra claramente establecido. Las fuentes disponibles lo describen como un juego tradicional chino desarrollado con posterioridad al *Xiangqi*, del que probablemente deriva a nivel conceptual [9].

El juego se estructura en torno a una jerarquía asimétrica de unidades representadas por animales, cada uno con un rango definido, y emplea un sistema de captura por desplazamiento similar al de los juegos de ajedrez, pero con un tablero peculiarmente distinto como se puede ver en la Figura 2.3. No obstante, introduce excepciones explícitas en las reglas de captura, como la capacidad de una pieza de rango inferior para derrotar a otra superior bajo determinadas condiciones, lo que rompe la jerarquía estricta y añade una dimensión estratégica basada en la gestión del riesgo y la anticipación del comportamiento del oponente. Este tipo de jerarquía no lineal anticipa principios fundamentales que aparecerán más adelante en los juegos de estrategia militar modernos.

El siguiente paso se produce a comienzos del siglo XX con la aparición de *L'Attaque*, un juego de mesa diseñado por Hermance Edan, quien registró su patente el 26 de noviembre de 1908 [10]. En su descripción original, el juego se define como un «*jeu de bataille avec des pièces mobiles sur damier*», es decir, un juego de batalla con piezas móviles sobre un tablero cuadriculado. La patente fue publicada oficialmente en 1909 por la Oficina de Patentes francesa, y el juego fue comercializado de forma continuada desde ese año hasta comienzos de la década de 1930.

L'Attaque introduce por primera vez de manera explícita la ocultación de información como mecánica central del diseño. Cada jugador dispone de un conjunto de piezas con valores numéricos jerárquicos, cuyo rango permanece oculto al oponente hasta el momento de la confrontación directa. El resultado del combate se resuelve comparando los valores de las piezas implicadas, eliminándose generalmente la de menor rango. La partida concluye cuando uno de los jugadores localiza y captura la pieza objetivo del adversario, identificada como la bandera.

Estas mecánicas suponen una ruptura clara con los juegos de información completa predominantes hasta ese momento, al trasladar una parte sustancial de la toma de decisiones al ámbito de la inferencia, el engaño y la gestión del riesgo. Asimismo, *L'Attaque* consolida una jerarquía militar explícita y permite la configuración libre de la disposición inicial de las unidades, lo que incrementa la profundidad estratégica desde el primer turno y refuerza la importancia de la planificación previa. En conjunto, estas innovaciones marcan un punto de inflexión en el diseño de juegos de estrategia, al introducir por primera vez de forma sistemática la incertidumbre como recurso estratégico.

La progresión histórica desde los primeros juegos de mesa hasta títulos como *L'Attaque* pone de manifiesto una evolución gradual desde mecánicas dominadas por el azar y el movimiento lineal hacia sistemas estratégicos cada vez más complejos, basados en la jerarquía de unidades, la confrontación directa y, finalmente, la ocultación de información. Esta acumulación progresiva de conceptos estratégicos, planificación, gestión del riesgo, engaño e inferencia, configura el marco teórico sobre el que se desarrollaría posteriormente *Stratego*.

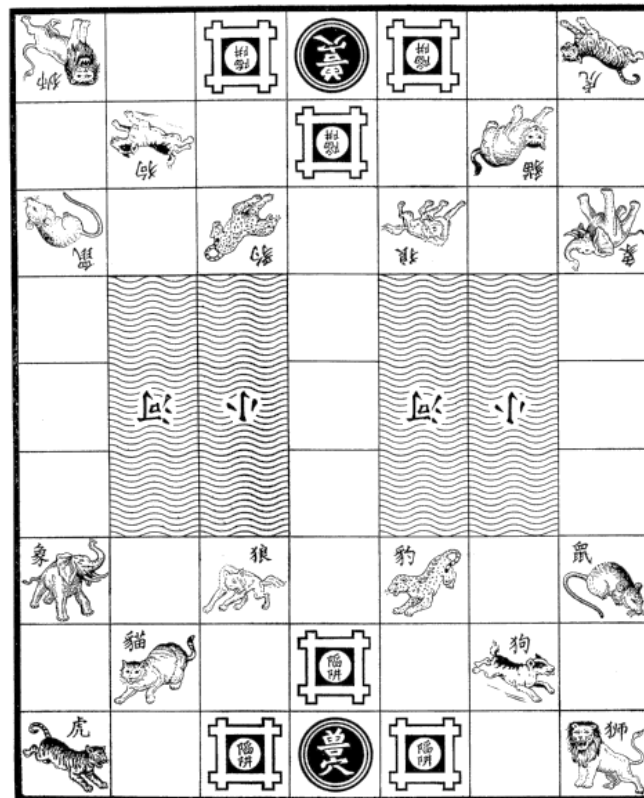


Figura 2.3: Tablero del juego Dou Shou Qi

2.2. El juego original: *Stratego*

El juego *Stratego* constituye uno de los referentes más influyentes dentro del ámbito de los juegos de estrategia con información imperfecta. Su diseño no surge de forma aislada, sino que se apoya en una serie de innovaciones mecánicas desarrolladas a comienzos del siglo XX, especialmente a partir del juego europeo *L'Attaque*, considerado su antecedente directo.

La versión moderna de *Stratego* fue desarrollada tras la Segunda Guerra Mundial por la compañía neerlandesa Jumbo, que adoptó una estética de inspiración militar napoleónica. El juego fue registrado comercialmente en 1960 y su difusión internacional se consolidó en 1961, cuando Milton Bradley obtuvo la licencia para su distribución en Estados Unidos [11].

Desde el punto de vista mecánico, *Stratego* se articula en torno a una jerarquía militar explícita de unidades, cuyos rangos permanecen ocultos al oponente hasta el momento del enfrentamiento directo. El sistema de combate se resuelve mediante la comparación de los valores de las piezas implicadas, eliminándose generalmente la de menor rango. El objetivo principal de la partida consiste en localizar y capturar la pieza objetivo del adversario, identificada como la bandera.

Una de las decisiones de diseño más relevantes es la configuración inicial libre del despliegue de las unidades, que traslada una parte sustancial del peso estratégico al momento previo al inicio de la partida. Esta característica, combinada con la ocultación de información, fomenta el engaño, la deducción y la gestión del riesgo como elementos centrales de la experiencia de juego.

Desde el punto de vista material, *Stratego* ha experimentado diversas adaptaciones físicas a lo largo del tiempo. Las primeras ediciones utilizaban piezas de cartón impreso, que fueron sustituidas posteriormente por piezas de madera pintada. A partir de finales de la década de 1960, estas fueron reemplazadas por piezas de plástico, más económicas y estables. Este cambio no solo respondió a criterios de producción, sino que también redujo el riesgo de vuelco accidental de las piezas, un problema crítico en un juego donde la revelación involuntaria de la información compromete el desarrollo estratégico de la partida.

Desde una perspectiva de diseño, *Stratego* se caracteriza por la ausencia total de elementos aleatorios durante la partida. Todas las decisiones se basan en la planificación, la inferencia y la lectura del comportamiento del oponente, lo que lo convierte en un paradigma de la estrategia bajo información imperfecta.

2.3. Variaciones, adaptaciones y videojuegos relacionados

Además de sus múltiples ediciones físicas, *Stratego* ha dado lugar a numerosas variantes temáticas ambientadas en universos de ficción como *Star Wars*, *El Señor de los Anillos*, o *Marvel*. Estas adaptaciones mantienen la estructura básica del juego original, introduciendo cambios estéticos y ajustes menores en las reglas según la ambientación.

En el ámbito digital, *Stratego* cuenta con versiones en línea que permiten partidas a distancia entre jugadores de todo el mundo como el *Stratego Online*¹ y apps móviles disponibles en iOS y Android. Estas plataformas incorporan modos clásicos y variantes con tableros alternativos o un número reducido de piezas, contribuyendo a mantener vigente el interés por el juego. El juego conserva además presencia activa en el ámbito competitivo, especialmente en países como Países Bajos, Alemania y Bélgica, Incluyendo torneos oficiales y rankings europeos.

Junto a *Stratego*, existen otros juegos de mesa y digitales basados en principios similares. Un ejemplo es *Luzhanqi* [12], originario de China, cuyo tablero se muestra en la Figura 2.4. Este juego comparte la mecánica de piezas ocultas y configuración libre inicial, evidenciando una convergencia de ideas estratégicas entre distintas tradiciones culturales.

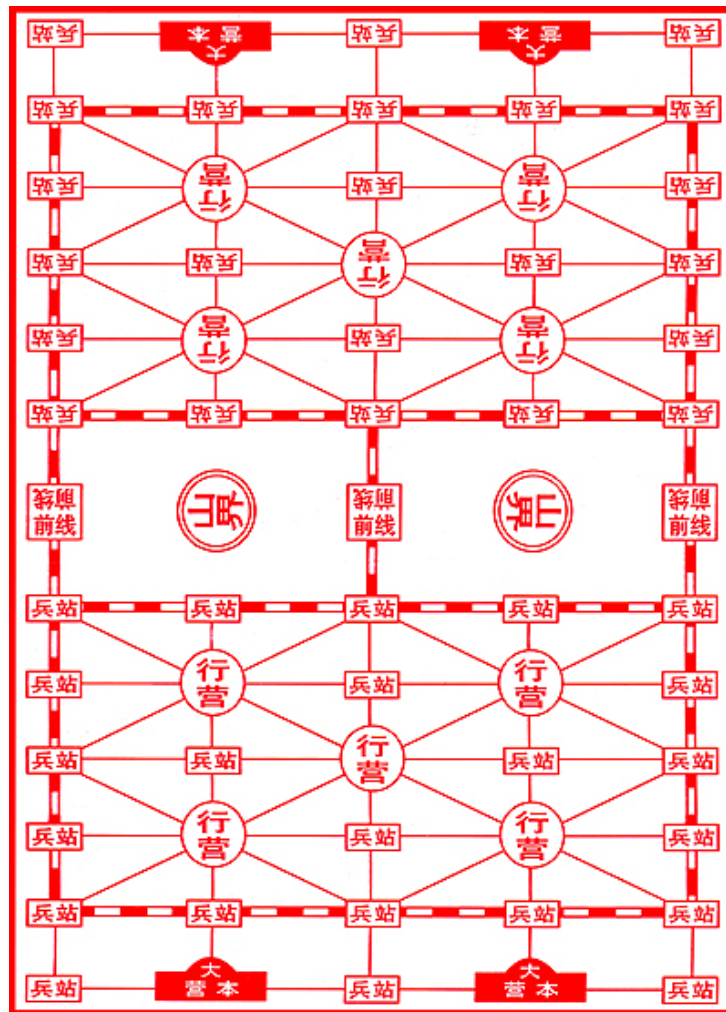


Figura 2.4: Tablero del juego Luzhanqi

¹<https://www.stratego.com>

En el ámbito de los videojuegos, destacan títulos de estrategia por turnos que enfatizan la planificación, la jerarquía de unidades y la toma de decisiones en entornos discretos. Entre ellos se encuentran la saga *Advance Wars* (Intelligent Systems, Advance Wars series, Nintendo) y *Into the Breach* (Subset Games, Into the Breach). En ambos casos, aunque la información sobre el enemigo es visible, el peso estratégico recae en la gestión de unidades, el posicionamiento y la anticipación de las acciones del rival.

Dentro del ámbito de los videojuegos de estrategia por turnos que comparten principios mecánicos con *Stratego*, destacan títulos como la saga *Advance Wars*. Esta serie, desarrollada por Intelligent Systems para plataformas de Nintendo entre 2001 y 2019, se basa en combates tácticos sobre un tablero cuadriculado donde los jugadores despliegan y gestionan unidades con jerarquías y características diferenciadas. Cada unidad posee fortalezas y debilidades específicas frente a otros tipos de unidades, lo que obliga a planificar los movimientos cuidadosamente y anticipar las acciones del adversario. Aunque la información sobre la ubicación de las unidades enemigas es visible, la profundidad estratégica recae en la gestión eficiente de recursos, el posicionamiento táctico y la secuencia de ataques, elementos que recuerdan la planificación y la jerarquía presentes en *Stratego*.

De manera similar, el videojuego *Into the Breach* (Subset Games, 2018) traslada el concepto de estrategia por turnos a un entorno de tablero discretizado donde cada unidad tiene capacidades específicas y limitadas. La mecánica principal consiste en anticipar el comportamiento del enemigo y planificar acciones para proteger objetivos clave, equilibrando riesgo y recompensa en cada turno. Esta combinación de toma de decisiones estrictamente calculada, jerarquía de unidades y confrontación directa recuerda la esencia de *Stratego*, donde el control del espacio y la gestión de las piezas son determinantes para la victoria. La simplicidad aparente del tablero contrasta con la profundidad estratégica requerida, consolidando a *Into the Breach* como un ejemplo contemporáneo de cómo los principios clásicos de los videojuegos de estrategia pueden adaptarse eficazmente al medio digital.

Estas aproximaciones digitales ponen de manifiesto la diversidad de soluciones adoptadas en el diseño de videojuegos de estrategia y sirven como referencia para el desarrollo del TFG. El objetivo del proyecto es combinar la jerarquía de unidades y la confrontación directa propias de *Stratego* con las ventajas del medio digital, manteniendo la tensión estratégica derivada de la información oculta y aprovechando las posibilidades de automatización, accesibilidad y escalabilidad que ofrece el formato de videojuego.

2.4. Motores de desarrollo de videojuegos

El motor de videojuegos es el componente central de software sobre el que se construye un proyecto, ya que proporciona las herramientas necesarias para gestionar la lógica del juego, el renderizado gráfico, la reproducción de audio, la interfaz de usuario y la exportación a distintas plataformas. Gracias a esta abstracción, el desarrollador puede centrarse principalmente en la lógica del videojuego, el diseño de mecánicas y la experiencia del usuario.

Los primeros motores reutilizables surgieron en la década de 1990 a raíz del éxito de títulos como *Doom* (id Software, 1993), cuyo motor fue licenciado a terceros y sentó las bases del concepto moderno de motor de videojuegos [13]. A partir de entonces, la industria evolucionó hacia soluciones cada vez más genéricas y accesibles.

Unreal Engine, lanzado por Epic Games en 1998, se consolidó como el motor de referencia en producciones de gran presupuesto y alta exigencia gráfica. Sus sistemas avanzados de renderizado, iluminación y simulación física permiten desarrollar entornos tridimensionales de alta fidelidad visual, lo que lo convierte en la herramienta estándar en la industria AAA. Esta potencia técnica tiene como contrapartida una curva de aprendizaje pronunciada, unos requisitos de hardware elevados y tiempos de compilación considerables, factores que pueden suponer una desventaja significativa en proyectos de pequeña escala o con recursos limitados [27].

Unity, publicado en 2005, se consolidó como el segundo gran referente del sector gracias a su versatilidad para desarrollar videojuegos en dos y tres dimensiones sobre una amplia variedad de plataformas, con una presencia especialmente destacada en el desarrollo independiente y móvil. Cuenta con una comunidad muy extensa, una gran cantidad de documentación y un ecosistema amplio de herramientas y recursos. Sin embargo, su arquitectura interna está históricamente orientada al desarrollo tridimensional, lo que puede introducir una sobrecarga innecesaria en proyectos 2D más sencillos. Además, la controversia generada en 2023 [28, 29] por una propuesta de tarificación por instalación del videojuego, ampliamente rechazada por la comunidad y finalmente revertida, puso en evidencia los riesgos asociados a depender de un motor de código cerrado con modelo de licencia comercial [30].

En este contexto surge *Godot Engine*, un motor de código abierto publicado bajo licencia MIT cuya primera versión estable se lanzó en 2014 [14]. Su arquitectura está basada en un sistema de nodos y escenas que facilita la organización modular del proyecto, y ofrece dos lenguajes de scripting nativos: *GDScript*, con una sintaxis inspirada en Python orientada a la accesibilidad, y *C#*, integrado mediante el entorno .NET, que permite aprovechar el ecosistema .NET y resulta especialmente adecuado para proyectos que requieren mayor control sobre el rendimiento o integración con otras herramientas. Su motor 2D es un sistema nativo e independiente del motor 3D, lo que lo convierte en una herramienta especialmente eficiente para el desarrollo de videojuegos bidimensionales. Su editor ocupa menos de 200 MB, arranca en segundos y permite ciclos de iteración muy rápidos [31]. A esto se suma la ausencia total de costes de licencia y la imposibilidad estructural de que las condiciones de uso cambien de forma unilateral al tratarse de software de código abierto.

Godot ha experimentado un crecimiento notable en los últimos años dentro del desarrollo independiente, en parte impulsado por la polémica de Unity de 2023. El caso más representativo es el de *Slay the Spire II* (MegaCrit, 2026): el estudio migró el proyecto íntegramente a Godot tras dicha polémica, asumiendo reescribir más de dos años de trabajo [32]. El resultado fue uno de los lanzamientos independientes más exitosos de los últimos años, alcanzando un pico de más de 500.000 jugadores simultáneos en Steam en sus primeros tres días en Early Access y superando los tres millones de unidades vendidas en su primera semana [33]. Este caso ilustra que Godot es capaz de sustentar productos comerciales de alto nivel cuando el tipo de videojuego se alinea con sus fortalezas.

La Tabla 2.1 resume las principales diferencias entre los tres motores.

Criterio	Unreal Engine	Unity	Godot
Rendimiento en 3D	Muy alto	Alto	Medio
Rendimiento en 2D	Medio	Alto	Muy alto
Curva de aprendizaje	Alta	Media	Baja
Requisitos de hardware	Altos	Medios	Bajos
Modelo de licencia	Comercial	Comercial	Código abierto
Adecuación a proyectos individuales	Media	Alta	Muy alta
Ecosistema y comunidad	Muy amplio	Muy amplio	En crecimiento

Tabla 2.1: Comparativa entre motores de desarrollo de videojuegos

2.5. Inteligencia artificial en videojuegos de estrategia

El desarrollo de agentes inteligentes capaces de competir en videojuegos de estrategia ha sido uno de los campos de investigación más activos en inteligencia artificial desde sus inicios. La elección del enfoque adecuado depende en gran medida de las características del videojuego: si la información es completa o parcial, si el espacio de estados es manejable o exponencialmente grande, y si se dispone de una función de evaluación fiable.

Los primeros enfoques consolidados se basaron en algoritmos de búsqueda en árbol. El algoritmo *Minimax*, formalizado en la década de 1950 a partir de los trabajos de John von Neumann sobre teoría de juegos [15], evalúa el árbol de posibles estados del videojuego asumiendo que ambos jugadores actúan de forma óptima: el agente maximiza su propio beneficio mientras el rival lo minimiza. Su variante *Alpha-Beta Pruning*, propuesta por John McCarthy y desarrollada por otros investigadores durante los años 1960 [16], mejora considerablemente la eficiencia al descartar ramas del árbol que no pueden influir en la decisión final. Estos algoritmos demostraron gran efectividad en videojuegos de información completa, siendo la base de programas históricos como *Deep Blue* [17]. Sin embargo, en videojuegos con información oculta como *Stratego*, el espacio de estados se vuelve exponencialmente mayor al desconocer la disposición enemiga, lo que limita su aplicabilidad práctica.

Para abordar entornos con incertidumbre, *Monte Carlo Tree Search* (MCTS) representó un avance significativo. En lugar de explorar exhaustivamente el árbol de juego, MCTS realiza simulaciones aleatorias desde el estado actual para estimar la calidad de cada movimiento, combinando exploración y explotación mediante la política UCB (*Upper Confidence Bound*) [18]. Este enfoque alcanzó notoriedad mundial como componente central de AlphaGo [19], el primer sistema en superar a jugadores profesionales en el juego del Go. Aunque MCTS es más tolerante a la información imperfecta que Minimax, su eficacia sigue dependiendo del número de simulaciones disponibles y de la calidad de la función de evaluación utilizada.

Un paradigma que mejor se adapta a entornos con información parcial y espacios de estados amplios es el aprendizaje por refuerzo (*Reinforcement Learning*, RL). En lugar de explorar explícitamente el árbol de juego, un agente entrenado mediante RL aprende a tomar decisiones a partir de la experiencia acumulada en miles de episodios de juego, ajustando su política de actuación en función de las recompensas obtenidas. Este enfoque fue popularizado por *DeepMind* con la introducción de *Deep Q-Network* (DQN) [20], que combinaba redes neuronales profundas con RL para aprender directamente desde la observación del estado. Posteriormente, algoritmos como *Proximal Policy Optimization* (PPO) [21] mejoraron la estabilidad del entrenamiento y se convirtieron en uno de los estándares de facto en aplicaciones de RL modernas.

Para facilitar la experimentación con estos algoritmos, *OpenAI* publicó en 2016 la biblioteca *Gym* [22], que proporciona una interfaz estándar para definir entornos de aprendizaje por refuerzo. Su sucesor, *Gymnasium* [23], mantiene dicha interfaz y se ha convertido en la referencia actual del ecosistema. Sobre esta base, la biblioteca *Stable Baselines3* [24] ofrece implementaciones robustas de algoritmos como PPO y DQN, y su extensión *SB3-Contrib* incorpora *MaskablePPO*. Esta última es una variante especializada de PPO que introduce la capacidad de aplicar máscaras sobre el espacio de acciones en cada paso de tiempo. El enmascaramiento permite bloquear dinámicamente aquellas decisiones que violan las reglas del juego antes de que la red neuronal las evalúe, evitando que el agente pierda tiempo de computación explorando movimientos prohibidos o requiera complejas funciones de penalización para aprender a evitarlos.

Para el desarrollo del agente inteligente del TFG se ha optado por esta combinación: *Gymnasium* como interfaz del entorno y *MaskablePPO* como algoritmo de entrenamiento. La elección de *MaskablePPO* resulta fundamental debido a la naturaleza de las mecánicas tácticas de *NeuroForge*, donde el conjunto de movimientos válidos de las piezas varía drásticamente en cada turno según el posicionamiento y la jerarquía en el tablero. De este modo, el entorno discretizado permite que el agente observe el estado actual, seleccione únicamente acciones legalmente viables a través de la máscara y reciba recompensas en función del resultado, optimizando el proceso de convergencia sin necesidad de diseñar manualmente una función de evaluación analítica del tablero.

Diseño del videojuego

En este capítulo se describe el diseño general de *NeuroForge*, abordando sus mecánicas principales, reglas de juego y los elementos que definen su experiencia estratégica. Se detallan el funcionamiento del sistema de juego, la interacción con el jugador, la ambientación y el público al que va dirigido. Asimismo, se analiza la propuesta desde la perspectiva del diseño de niveles y la escalabilidad de sus escenarios, se definen los perfiles de la audiencia objetivo bajo el principio de accesibilidad y profundidad, y se justifica la clasificación del contenido de acuerdo con el estándar europeo PEGI.

3.1. Resumen del videojuego

NeuroForge es un videojuego de estrategia por turnos para un jugador inspirado en el clásico *Stratego*. Dos bandos se enfrentan en un tablero de 10×10 casillas: el jugador humano y un oponente controlado por la máquina. El objetivo es destruir el *núcleo de energía* rival o dejarlo sin movimientos posibles.

Cada bando despliega en secreto un conjunto de piezas de distintos rangos en las cuatro filas más cercanas a su borde del tablero. Las identidades permanecen ocultas para el rival hasta que dos piezas entran en combate, momento en el que ambas se revelan y se resuelve el enfrentamiento según su rango.

En cada turno, el jugador puede realizar una única acción: mover una pieza a una casilla adyacente vacía, o moverla a una casilla ocupada por una pieza enemiga para iniciar un combate. Existe una excepción, los *exploradores*, que pueden desplazarse varias casillas en línea recta en un mismo turno. El combate se resuelve de la siguiente forma:

- La pieza de mayor rango vence y ocupa la casilla de la derrotada.
- Si ambas piezas tienen el mismo rango, ambas son eliminadas del tablero.
- Ciertas piezas tienen reglas especiales que se detallan más adelante.

3.2. Mecánicas

Gran parte del atractivo de *NeuroForge* reside en el desconocimiento mutuo de la identidad de las piezas rivales. Esto convierte la deducción y el engaño en elementos estratégicos centrales, el jugador debe inferir la disposición del enemigo a partir de su comportamiento, al tiempo que protege la información sobre sus propias piezas.

3.2.1. Tablero

El tablero es una cuadrícula de 10×10 casillas (Figura 3.1). Las cuatro filas más próximas a cada borde constituyen el **área de despliegue** de cada jugador, donde se colocan las piezas al inicio de la partida. Las dos filas centrales permanecen despejadas al comienzo y son terreno neutral.

En el centro del tablero se encuentran **casillas intransitables**, dispuestas en dos zonas de 2×2 . Ninguna pieza puede moverse a ellas, ocuparlas ni atravesarlas. Estos obstáculos dividen el tablero en dos mitades y condicionan las rutas de ataque y defensa, actuando como cuellos de botella que añaden profundidad táctica al posicionamiento.

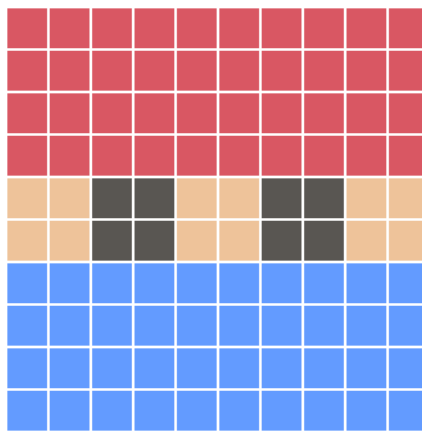


Figura 3.1: Esquema del tablero

3.2.2. Colocación inicial

Al inicio de cada partida, ambos bandos colocan sus piezas dentro de su respectiva área de despliegue con la distribución que consideren oportuna. No es posible colocar piezas fuera de dicha área antes de comenzar la partida. Una vez confirmada la colocación, la partida da comienzo y ningún jugador conoce la identidad de las piezas rivales.

La fase de despliegue tiene un peso estratégico considerable, una disposición bien planificada puede proteger las piezas clave, dificultar la deducción del rival y preparar aperturas ofensivas o defensivas para los primeros turnos.

3.2.3. Piezas

Las piezas se rigen por un sistema de rangos que determina el resultado de los combates, donde la pieza de mayor rango vence a la de menor rango. Cada tipo de pieza tiene una cantidad limitada por bando. Algunas piezas cuentan con propiedades especiales que modifican las reglas generales de movimiento o combate. A continuación se describen todos los tipos disponibles:

- **Núcleo de energía (cantidad: 1):** La pieza más importante del bando. No puede moverse. Su destrucción supone la derrota inmediata de su propietario, y puede ser atacado por cualquier pieza enemiga capaz de moverse.
- **Torretas (cantidad: 6):** Estructuras defensivas inmóviles que no pueden atacar, pero destruyen automáticamente cualquier pieza que las ataque, independientemente de su rango. La única excepción son los **saboteadores**, que son los únicos capaces de destruirlas.
- **Máquina de guerra (cantidad: 1, rango 10):** La unidad de combate más poderosa. Vence a cualquier pieza por rango, salvo al **Phantom** si este ataca primero.
- **Nova (cantidad: 1, rango 9):** Sin propiedades especiales.
- **Mecha (cantidad: 2, rango 8):** Sin propiedades especiales.
- **Centinela (cantidad: 3, rango 7):** Sin propiedades especiales.
- **Canino (cantidad: 4, rango 6):** Sin propiedades especiales.
- **Cyborg (cantidad: 4, rango 5):** Sin propiedades especiales.
- **Soldado (cantidad: 4, rango 4):** Sin propiedades especiales.
- **Saboteador (cantidad: 5, rango 3):** La única pieza capaz de destruir las **torretas**. En cualquier otro combate se rige por las reglas generales de rango.
- **Explorador (cantidad: 6, rango 2):** Puede desplazarse cualquier número de casillas en línea recta, horizontal o verticalmente, siempre que no haya obstáculos ni piezas en su camino. Si se desplaza más de una casilla en un turno, **su identidad queda revelada**.
- **Phantom (cantidad: 1, rango 1):** La pieza de menor rango. Su única ventaja es que derrota a la **Máquina de guerra** si es quien inicia el ataque. En cualquier otro combate pierde por rango.

3.2.4. Turnos

La partida se desarrolla en turnos alternos, comenzando siempre el jugador humano. En cada turno se realiza una única acción con una sola pieza, moverla a una casilla vacía adyacente, o moverla a una casilla ocupada por una pieza enemiga para iniciar un combate. No es posible mover y atacar en el mismo turno, salvo en el caso del **explorador**, que puede cubrir varias casillas en un único movimiento y atacar al final de su recorrido si la casilla de destino está ocupada por un enemigo.

Las reglas que rigen el movimiento son las siguientes. Solo se puede mover una pieza por turno, desplazándola una única casilla en cualquiera de las cuatro direcciones ortogonales (arriba, abajo, izquierda, derecha), nunca en diagonal. No puede haber dos piezas en la misma casilla, y ninguna pieza puede saltar ni atravesar casillas ocupadas por otras piezas o por zonas intransitables. Las piezas sin capacidad de movimiento, como el **núcleo de energía** y las **torretas**, permanecen fijas durante toda la partida.

Para evitar situaciones de pasividad o movimientos repetitivos sin intención estratégica, existe una restricción adicional, una pieza no puede moverse entre 2 casillas durante **mas de tres turnos**, al cuarto turno se le bloqueara la posibilidad de regresar a la casilla de la que viene hasta el siguiente turno o se mueva a otra distinta. Esta limitación afecta únicamente a la pieza que salió de esa casilla, otras piezas pueden ocuparla sin restricción.

En cuanto al ataque, este se produce cuando una pieza se desplaza a una casilla ocupada por una pieza enemiga, iniciando así un combate. Si la pieza atacante vence, ocupa la casilla de la pieza destruida. Si pierde, es ella quien es eliminada y la pieza defendida permanece en su posición. Si al comienzo del turno de un bando ninguna de sus piezas puede realizar movimiento o ataque válido alguno, la partida termina y ese bando pierde.

3.2.5. Resolución de combates

El combate se produce cuando una pieza se desplaza a una casilla ocupada por una pieza rival. En ese momento, la identidad de ambas piezas queda revelada y el resultado se determina según las siguientes reglas:

- Si la pieza atacante tiene **mayor rango**, la pieza rival es destruida y el atacante vence, ocupando su casilla. Si tiene **menor rango**, es destruida y la pieza defensora permanece en su posición.
- Si ambas piezas tienen el **mismo rango**, el combate termina en empate y ambas son eliminadas del tablero.
- Una *torreta* destruye a cualquier pieza que la ataque, excepto al **saboteador**, que la destruye y ocupa su casilla.
- Si el *Phantom* ataca a la *Máquina de guerra*, el *Phantom* vence independientemente del rango.

En la Figura 3.2 hay representado un diagrama de flujo resumiendo de manera visual la decisión del resultado de un combate, desde que se inicia hasta que se elimina una o ambas piezas.

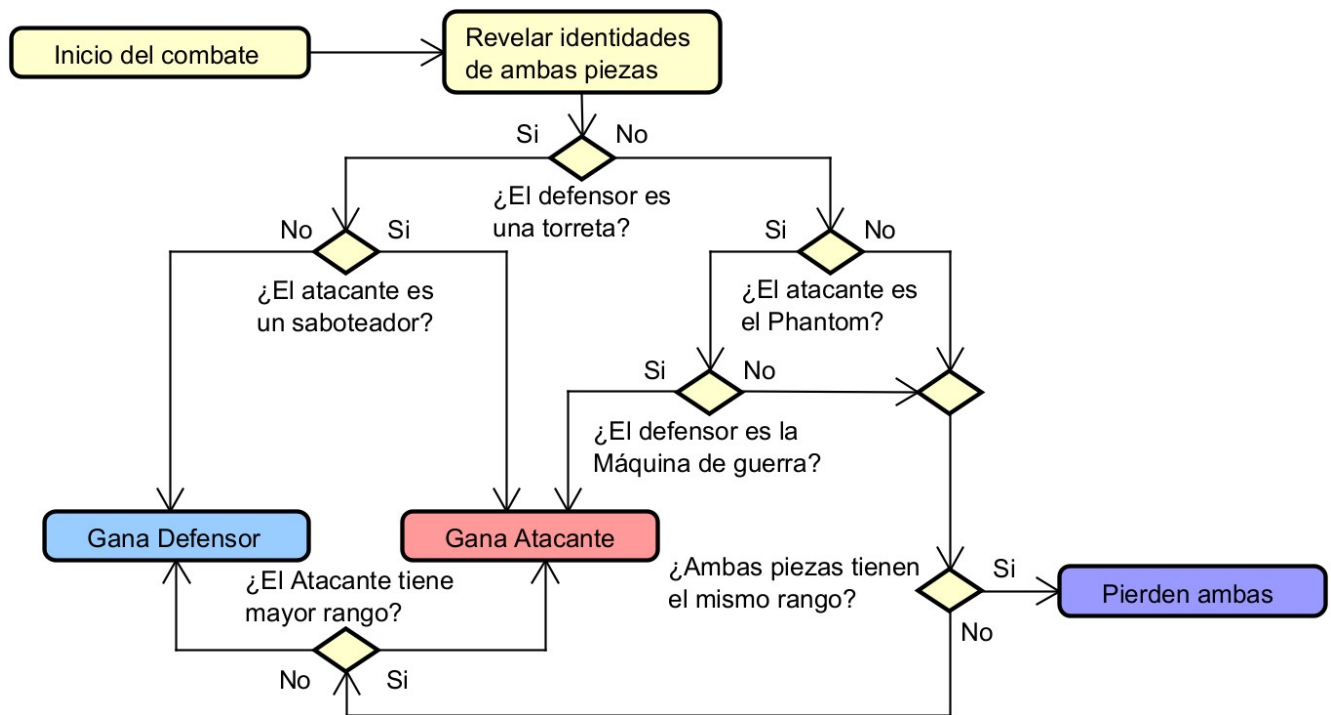


Figura 3.2: Esquema del tablero

3.2.6. Condiciones de victoria

La partida puede concluir de dos formas. La primera es la destrucción del *núcleo de energía* rival, lo que supone la derrota inmediata de su propietario.

La segunda es que uno de los bandos quede sin movimientos posibles, es decir, si todas sus piezas con capacidad de moverse han sido destruidas o bloqueadas, ese bando pierde. En ambos casos, el bando que provoca la condición de victoria gana la partida.

3.2.7. Interacción y control

El jugador realiza todas sus acciones mediante el ratón. El sistema de interacción está diseñado para que las posibilidades disponibles en cada momento sean siempre visualmente evidentes, minimizando la necesidad de memorizar reglas durante la partida.

Al inicio del turno del jugador, las piezas que pueden actuar aparecen resaltadas en el tablero; las que no tienen movimientos disponibles no se destacan. Para mover una pieza, cuando el jugador pulsa sobre ella, lo que hace que se muestren las casillas de destino válidas. Haciendo click en una de esas casillas, el movimiento queda confirmado. Si se hace click en cualquier otro lado que no sea una de esas casillas (incluyendo la propia pieza), la acción se cancela y el jugador puede volver a intentarlo.

Al desplazarse a una casilla vacía, el turno finaliza y pasa al oponente. Al desplazarse a una casilla ocupada por una pieza enemiga, se inicia el combate, ambas piezas se resaltan brevemente en pantalla, las piezas ocultas se revelan si aún no habían sido identificadas, y el resultado se resuelve según las reglas establecidas. La pieza perdedora desaparece del tablero y la ganadora queda visible en la casilla donde se produjo el enfrentamiento. Cada acción se acompaña de animaciones y efectos sonoros que refuerzan la claridad de lo ocurrido.

Una vez concluida la acción del jugador, el bot ejecuta su turno de forma automática. El jugador observa el movimiento del oponente y recupera el control al inicio de su siguiente turno.

3.3. Ambientación

NeuroForge sitúa al jugador en un universo de ciencia ficción futurista en el que las guerras se libran mediante ejércitos de inteligencias artificiales, drones y armamento avanzado. El título hace referencia al proceso de forjar una red neuronal, cada partida representa un ciclo de entrenamiento intensivo en el que una IA militar es sometida a prueba, refinada y preparada para el combate real. Los enfrentamientos no tienen lugar en un campo de batalla físico, sino dentro de un programa de simulación de combate de alta fidelidad, desarrollado para evaluar y optimizar las capacidades estratégicas de estas inteligencias antes de su despliegue. Cada simulación es un escenario controlado en el que las unidades son instancias virtuales, los combates son pruebas de rendimiento y cada resultado, victoria o derrota, alimenta el proceso de entrenamiento de la IA, ajustando sus parámetros para el siguiente ciclo.

3.3.1. Escenario

Los escenarios de entrenamiento se generan a partir de registros digitalizados de ciudades abandonadas y destruidas por conflictos previos. Las zonas intransitables del tablero representan sectores urbanos críticos, completamente colapsados por la acumulación masiva de escombros y ruinas estructurales que impiden físicamente el paso. Asimismo, estas áreas presentan una densa contaminación ambiental por residuos de armamento avanzado, resultando en una atmósfera altamente letal tanto para componentes orgánicos humanos como para los sistemas electrónicos de las máquinas. Debido a la peligrosidad de estos focos, dichos sectores han sido contenidos de forma artificial, quedando su acceso completamente restringido. De este modo, el tránsito o posicionamiento de cualquier unidad dentro de estas zonas queda totalmente imposibilitado durante la simulación. En el entorno real recreado, estas áreas solo volverían a ser accesibles a largo plazo, una vez que los sistemas de contención cumplan su ciclo de aislamiento y la contaminación se haya disipado.

3.3.2. Unidades y estructuras

- **Núcleo de energía:** Centro vital de cada bando. Su destrucción interrumpe el suministro energético de todas las unidades y supone la derrota inmediata.
- **Torretas:** Sistemas defensivos estáticos con capacidad de eliminar cualquier amenaza, salvo los **saboteadores**, que las neutralizan mediante camuflaje avanzado.
- **Máquina de guerra:** Mecha masivo pilotado por la IA suprema del bando. La unidad más poderosa del campo de batalla, vulnerable únicamente frente al ataque de malware del **Phantom**.
- **Nova:** Guardia cyborg de alto rango con partes orgánicas prácticamente inexistentes. Coordina el despliegue de las fuerzas en combate.
- **Mecha:** Robots de combate pesados diseñados para resistir y dominar en combate directo.
- **Centinela:** Unidades de asalto mecanizadas optimizadas para el combate, con una resistencia estructural muy superior a cualquier unidad humana.
- **Canino:** Cuadrúpedos mecánicos de alta movilidad diseñados para la interceptación rápida.
- **Cyborg:** Soldados rápidos y eficaces con implantes cibernéticos ligeros.
- **Soldado:** Refuerzos humanos destinados a mantener las líneas de combate.
- **Saboteador:** Especialistas en infiltración capaces de neutralizar torretas mediante tecnología de camuflaje avanzado. Indefensos en combate directo contra otras unidades.
- **Explorador:** Drones de reconocimiento ágiles capaces de recorrer largas distancias en línea recta. Si se desplazan más de una casilla en un turno, su identidad queda revelada, ya que son los únicos con ese tipo de movimiento.
- **Phantom:** Dron de infiltración diseñado con un único propósito: destruir a la *Máquina de guerra* enemiga mediante malware especializado. Es la unidad más débil en un combate convencional, pero puede destruir instantáneamente a la *Máquina de guerra* si ataca primero.

3.4. Diseño de niveles

NeuroForge se desarrolla sobre un único tablero estándar de 10×10 casillas. Pese a su aparente simplicidad, las zonas intransitables del centro generan cuellos de botella que condicionan las rutas de movimiento y obligan a planificar cuidadosamente las aperturas y las líneas de avance.

El diseño actual se centra en este tablero como escenario principal. No obstante, la arquitectura modular del videojuego abre la puerta a posibles expansiones futuras, entre las que podrían contemplarse tableros de dimensiones distintas, eventos ambientales que alteren temporalmente la movilidad de ciertas piezas, o casillas especiales con efectos estratégicos concretos. Estas posibilidades no forman parte del alcance del proyecto, pero ilustran la escalabilidad del diseño sin necesidad de modificar las mecánicas fundamentales.

3.5. Audiencia

El perfil del jugador objetivo se divide en dos grandes segmentos. Por un lado, los jugadores experimentados, usuarios habituados a títulos clásicos de tablero como el ajedrez o Stratego, o a videojuegos de estrategia táctica actuales. Este perfil busca profundidad estratégica, desafíos intelectuales complejos y mecánicas basadas en la gestión de información oculta y la anticipación de los movimientos del rival. Por otro lado, los jugadores casuales, personas que, sin experiencia previa en el género, se sienten atraídas por la temática de simulación cibernética o por la resolución de problemas lógicos, y que prefieren aprender las reglas de forma progresiva durante la partida.

Para satisfacer a ambos perfiles, el diseño sigue el principio de *fácil de aprender, difícil de dominar*, una interfaz limpia e intuitiva que reduce la carga cognitiva inicial, combinada con una lógica de juego que gana en complejidad conforme avanza la partida. De este modo, la accesibilidad no sacrifica en ningún momento la profundidad táctica.

Con el fin de garantizar que el producto sea apto para su distribución en el mercado europeo, los contenidos de NeuroForge han sido evaluados bajo los criterios del sistema *Pan European Game Information (PEGI)*, determinándose una clasificación de PEGI 3, con PEGI 7 como límite superior dado el carácter abstracto del conflicto. Esta catalogación se apoya en tres argumentos principales.

En primer lugar, la ausencia total de violencia explícita o gráfica. Aunque el videojuego implica el enfrentamiento entre unidades y la eliminación de piezas, todo ello se representa de forma abstracta mediante metáforas informáticas, sin ninguna representación de daño físico, sangre o elementos que pudieran resultar perturbadores para menores.

En segundo lugar, la inexistencia de lenguaje inapropiado. Al tratarse de un sistema local sin canales de comunicación abierta, no incorpora chats de voz ni texto sin moderar, ni contiene diálogos o textos pregrabados con expresiones vulgares o inapropiadas.

En tercer lugar, la ausencia de monetización o mecánicas de azar. La aplicación no implementa pasarelas de pago, compras integradas ni sistemas de recompensa aleatoria, cumpliendo así con las directrices más recientes del estándar PEGI en materia de protección financiera de los usuarios.

En conjunto, la accesibilidad del diseño y el cumplimiento de los estándares de contenido seguro posicionan a NeuroForge como una experiencia estimulante para los aficionados a la estrategia, a la vez que garantizan un entorno adecuado para cualquier franja de edad.

Planificación

En este capítulo se presenta la planificación del desarrollo del proyecto *NeuroForge*, definiendo la metodología iterativa empleada, la organización temporal del trabajo, los recursos y herramientas utilizados, las restricciones existentes y los riesgos identificados. Asimismo, se detallan las variaciones y desviaciones sufridas respecto al cronograma inicial y se incluye un estudio económico exhaustivo para evaluar los costes y la viabilidad comercial del videojuego tanto en un escenario individual como empresarial.

La planificación se ha concebido como una guía flexible, susceptible de ser ajustada a lo largo del desarrollo conforme se adquiría una visión más precisa del alcance real del proyecto y de las dificultades técnicas asociadas a su implementación.

4.1. Metodología empleada

El desarrollo del TFG se ha llevado a cabo siguiendo una adaptación del *Proceso Unificado* [25] (*Unified Process*, UP), un marco de desarrollo de software caracterizado por estar dirigido por casos de uso, centrado en la arquitectura y, sobre todo, por ser *iterativo e incremental* [26].

La distinción entre los términos *iterativo* e *incremental* es relevante. El componente *iterativo* hace referencia a la repetición sistemática de un conjunto de disciplinas en cada ciclo de trabajo. El componente *incremental* implica que el resultado de cada iteración es un incremento funcional que se integra con el trabajo anterior y contribuye directamente al producto final. En la Figura 4.1 se presenta una metáfora visual que ilustra el carácter iterativo e incremental del proceso, en la que cada ciclo produce un incremento funcional que se integra con el trabajo anterior.

Para este proyecto, el Proceso Unificado organiza el ciclo de vida en cuatro grandes fases:

- **Inicio:** Se establece el alcance del sistema, se identifican los casos de uso principales y se evalúa la viabilidad.

- **Elaboración:** Se refina la arquitectura del sistema, se detallan los requisitos más relevantes y se mitigan los riesgos técnicos críticos.
- **Construcción:** Constituye el núcleo del desarrollo, donde el sistema se construye de forma iterativa e incremental.
- **Transición:** El sistema se despliega, se realizan pruebas de integración definitivas y se produce la versión de entrega.

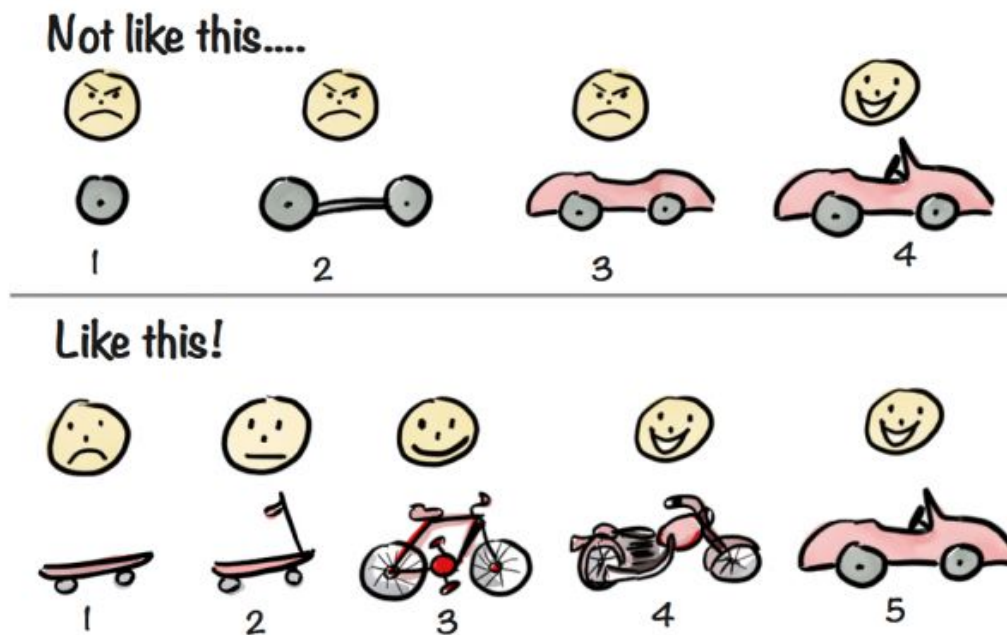


Figura 4.1: Metáfora gráfica del desarrollo iterativo e incremental

4.1.1. Etapas de desarrollo y ordenación en la planificación

Para cumplir con el rigor metodológico en un ámbito de desarrollo individual, el proyecto se ha estructurado en etapas secuenciales y disciplinas transversales que dictan el orden estricto de la planificación temporal. Estas etapas se ejecutan de la siguiente manera:

1. **Etapas de Análisis Global y Requisitos:** Configura el punto de partida del proyecto (Fase de Inicio). Su objetivo es la definición formal de los requisitos funcionales y no funcionales del sistema, modelando los actores y el alcance técnico indispensable antes de escribir código.
2. **Etapas de Diseño de Arquitectura Base:** Establece las bases estructurales del software (Fase de Elaboración). En ella se define la jerarquía de escenas en Godot, el modelo de datos del tablero y la abstracción de las piezas.
3. **Etapas de Construcción Iterativa e Incremental:** Una vez fijados el análisis y el diseño globales, el trabajo se ordena en bloques funcionales que ejecutan internamente las disciplinas de desarrollo de manera cíclica:
 - *Implementación:* Codificación concreta en el motor Godot utilizando C#.

- *Pruebas unitarias y de caja negra:* Verificación inmediata de que cada nuevo incremento funcional responde a lo esperado sin romper los sistemas previos.

4. **Etapa de Transición y Pulido:** Bloque final orientado a la consolidación del sistema, optimización de rendimiento y pruebas de integración globales.

Esta estructura metodológica se traslada directamente a la línea temporal del proyecto, ordenando las tareas cronológicamente desde la concepción teórica (Análisis y Diseño) hasta la materialización técnica y su posterior pulido en cinco iteraciones sucesivas. La correspondencia exacta se detalla en la Tabla 4.1.

Etapa	Iteración	Objetivo principal
Inicio	Análisis	Visión, requisitos y alcance formal del sistema
Elaboración	Diseño	Arquitectura base, esquema de datos y escenas
Construcción	It. 1: Videojuego base	Núcleo jugable mínimo y lógicas de tablero
	It. 2: Interfaz de usuario	Interfaz, menús y flujo de navegación visual
	It. 3: Mejoras audiovisuales	Sonido, música y ambientación del entorno
	It. 4: Bot inteligente	Implementación y entrenamiento de la IA
Transición	It. 5: Pulido final	Optimización, corrección de errores y entrega

Tabla 4.1: Correspondencia entre las fases de la metodología y la ordenación de las tareas en la planificación

4.2. Planificación inicial

La planificación inicial del proyecto se realizó con el objetivo de establecer una estimación razonable del trabajo a desarrollar, sirviendo como referencia para organizar las tareas y distribuir el esfuerzo a lo largo del periodo de realización del TFG.

A petición metodológica, el análisis y el diseño conceptual se han extraído y unificado como tareas de entidad propia al inicio del cronograma para garantizar una base teórica sólida. El resto del desarrollo se distribuye en cinco iteraciones incrementales bien acotadas, manteniendo intacto el presupuesto temporal total de 300 horas.

4.2.1. Tareas Globales Iniciales: Análisis y Diseño

Estas dos tareas constituyen el cimiento del proyecto antes de proceder a cualquier tipo de implementación en el motor de videojuegos.

- **Análisis de requisitos y casos de uso (6 h):** Identificación detallada de los requisitos funcionales del videojuego, definición de los actores (jugador, bot) y especificación formal de los casos de uso principales (despliegue de piezas, movimiento, combate y condiciones de victoria).
- **Diseño de la arquitectura y el modelo de datos (8 h):** Definición de la estructura de escenas madre en Godot, el esquema de clases principal en C#, la representación interna del tablero matricial, las piezas y el modelado conceptual del sistema de toma de decisiones del bot.

4.2.2. Iteración 1: Núcleo jugable (Videojuego base)

El objetivo de esta iteración es desarrollar un *Producto Mínimo Viable* funcional —sin elementos visuales pulidos ni inteligencia artificial compleja— que sirva como base estructural para el resto del proyecto y permita validar las decisiones técnicas adoptadas. Se estima una dedicación de **17 horas**.

- **Tablero y casillas (2 h):** Implementación de la cuadrícula de 10×10 casillas, incluyendo las zonas intransitables del centro del tablero.
- **Definición de piezas (3 h):** Desarrollo de los distintos tipos de piezas con sus propiedades individuales: rangos, piezas inmóviles, y reglas especiales de combate y movimiento.
- **Sistema de movimiento (3 h):** Implementación de la selección de piezas, la visualización de las casillas de destino válidas y el desplazamiento sobre una casilla válida.
- **Sistema de combate (3 h):** Resolución inmediata de los enfrentamientos entre piezas según las reglas de rango definidas, sin animaciones ni retroalimentación visual elaborada.
- **Sistema de ocultación de piezas (2 h):** Implementación de la mecánica de identidades ocultas, de forma que las piezas rivales no muestran su tipo hasta que entran en combate.
- **Despliegue inicial aleatorio del bot (1 h):** Colocación automática de las piezas del oponente en su zona de despliegue de forma aleatoria al comienzo de la partida.
- **Bot de movimiento aleatorio (1 h):** Implementación de un oponente básico que ejecuta movimientos válidos de forma aleatoria durante su turno, actuando como versión mínima funcional del oponente.
- **Sistema de despliegue del jugador (1 h):** Pantalla básica que permite al jugador colocar sus piezas en su zona antes de comenzar la partida.
- **Condición de fin de partida (1 h):** Detección de las condiciones de victoria y derrota, con parada completa de la partida al producirse alguna de ellas.

4.2.3. Iteración 2: Interfaz de usuario

El objetivo de esta iteración es dotar al videojuego de una interfaz completa, visualmente cuidada e intuitiva. Para dar cabida al bloque de Diseño global inicial, esta iteración reajusta su peso a **50 horas** sin perder alcance, optimizando las tareas de bocetado e integración.

- **Bocetado de interfaces** (6 h): Elaboración de bocetos y propuestas visuales para todas las pantallas del videojuego antes de su implementación.
- **Búsqueda e integración de assets** (6 h): Selección de recursos gráficos para construir las interfaces de forma visualmente coherente.
- **Animaciones de movimiento de piezas** (1 h): Incorporación de animaciones básicas al desplazamiento de las piezas sobre el tablero.
- **Rediseño visual del tablero** (4 h): Mejora de la representación gráfica del tablero y de los indicadores de selección.
- **Sprites de las piezas** (10 h): Creación e integración de la representación gráfica de cada tipo de pieza.
- **Interfaz de despliegue mejorada** (4 h): Rediseño de la pantalla de colocación inicial de piezas.
- **Interfaz de resolución de combate** (6 h): Pantalla que muestra el resultado de cada enfrentamiento de forma explícita.
- **Pantalla de fin de partida** (1 h): Escena que informa al jugador del resultado final de la partida.
- **Menú principal** (1 h): Escena inicial con opciones para comenzar, consultar reglas y salir.
- **Menú de pausa** (2 h): Escena accesible durante la partida.
- **Menú de reglas** (6 h): Escena explicativa de las mecánicas y reglas del juego.
- **Ajustes y correcciones** (3 h): Revisión general de los elementos implementados.

4.2.4. Iteración 3: Mejoras audiovisuales

El objetivo de esta iteración es completar la experiencia audiovisual del videojuego mediante la incorporación de sonido y los últimos elementos visuales de ambientación. Se estima una dedicación de **15 horas**.

- **Fondos y elementos decorativos** (2 h): Incorporación de fondos y elementos visuales de carácter estético.

- **Efectos de sonido** (6 h): Integración de efectos sonoros asociados a las distintas acciones del videojuego.
- **Música** (3 h): Incorporación de música de fondo para las distintas escenas del videojuego.
- **Menú de opciones de audio** (4 h): Implementación de una pantalla de configuración para ajustar volúmenes.

4.2.5. Iteración 4: Implementación del bot inteligente

El objetivo de esta iteración es desarrollar el sistema de inteligencia artificial estratégico. Puesto que el diseño conceptual de la IA se ha absorbido en la tarea de Diseño global inicial, esta iteración pasa a centrarse puramente en su desarrollo técnico, entrenamiento y validación, reduciendo su cómputo a **40 horas**.

- **Desarrollo de la lógica algorítmica del modelo** (18 h): Programación del motor de toma de decisiones e integración de las matrices de pesos para gestionar la información oculta del rival.
- **Entrenamiento del modelo** (10 h): Proceso de optimización de la IA mediante partidas simuladas contra versiones previas de sí misma.
- **Integración del bot inteligente** (5 h): Sustitución del bot aleatorio y acoplamiento directo con el sistema de turnos de la arquitectura.
- **Evaluación y balanceo del comportamiento** (7 h): Pruebas intensivas del bot frente a jugadores humanos y escenarios límite para garantizar decisiones coherentes y sin bloqueos.

4.2.6. Iteración 5: Pulido final

El objetivo de esta última iteración es revisar el conjunto del sistema, corregir los errores detectados y garantizar la estabilidad y calidad del producto. Se estima una dedicación de **14 horas**.

- **Optimización del rendimiento** (4 h): Análisis y mejora del rendimiento del sistema, hilos de ejecución y consumos de recursos.
- **Revisión del feedback al jugador** (4 h): Comprobación de que toda la información visual y sonora se transmite de forma limpia.
- **Corrección de errores** (4 h): Detección y resolución de bugs.
- **Pruebas de integración** (2 h): Verificación final del build completo del videojuego.

Paralelamente al desarrollo técnico, se estiman exactamente **140 horas** para la elaboración progresiva de la memoria del TFG, incluyendo la descripción del diseño del videojuego, la justificación de las decisiones técnicas adoptadas y el análisis de los resultados obtenidos.

En conjunto, la estimación total de dedicación para el proyecto asciende a unas **300 horas**, distribuidas de forma exacta en, 14 horas de tareas globales (Análisis y Diseño), 136 horas de desarrollo técnico y pulido en las cinco iteraciones, y las 140 restantes a la documentación de la memoria.

La Tabla 4.2 recoge el resumen definitivo de los tiempos de desarrollo adaptados.

Etapa / It.	Descripción de la Tarea	Horas est.	Fecha est. fin.
Tarea Global	Análisis de requisitos y casos de uso	6 h	2026-02-05
Tarea Global	Diseño de arquitectura y datos	8 h	2026-02-12
1	Implementación del videojuego base	17 h	2026-02-26
2	Interfaz de usuario y pulido visual	50 h	2026-04-05
3	Mejoras audiovisuales	15 h	2026-04-19
4	Implementación del bot inteligente (IA)	40 h	2026-05-31
5	Pulido final e integración	14 h	2026-06-07
-	Documentación de la Memoria (Paralela)	140 h	2026-06-14
Total		300 h	

Tabla 4.2: Distribución ajustada de horas por tareas globales e iteraciones del proyecto

La Figura 4.2 y 4.3 muestra de manera de manera actualizada el cronograma con la inserción de las tareas macro de análisis y diseño al inicio del desarrollo.

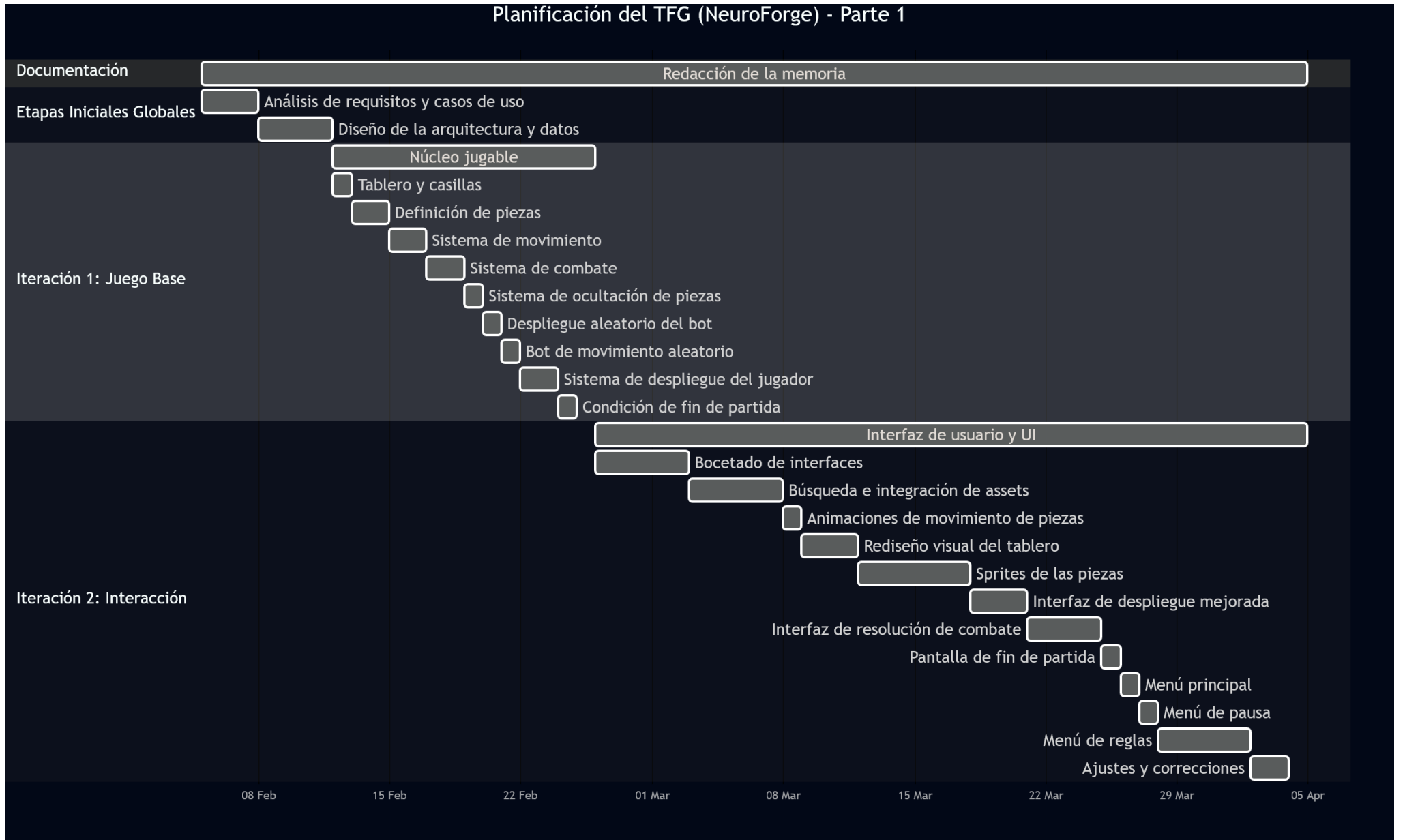


Figura 4.2: Parte 1 del diagrama de Gantt general del desarrollo del TFG

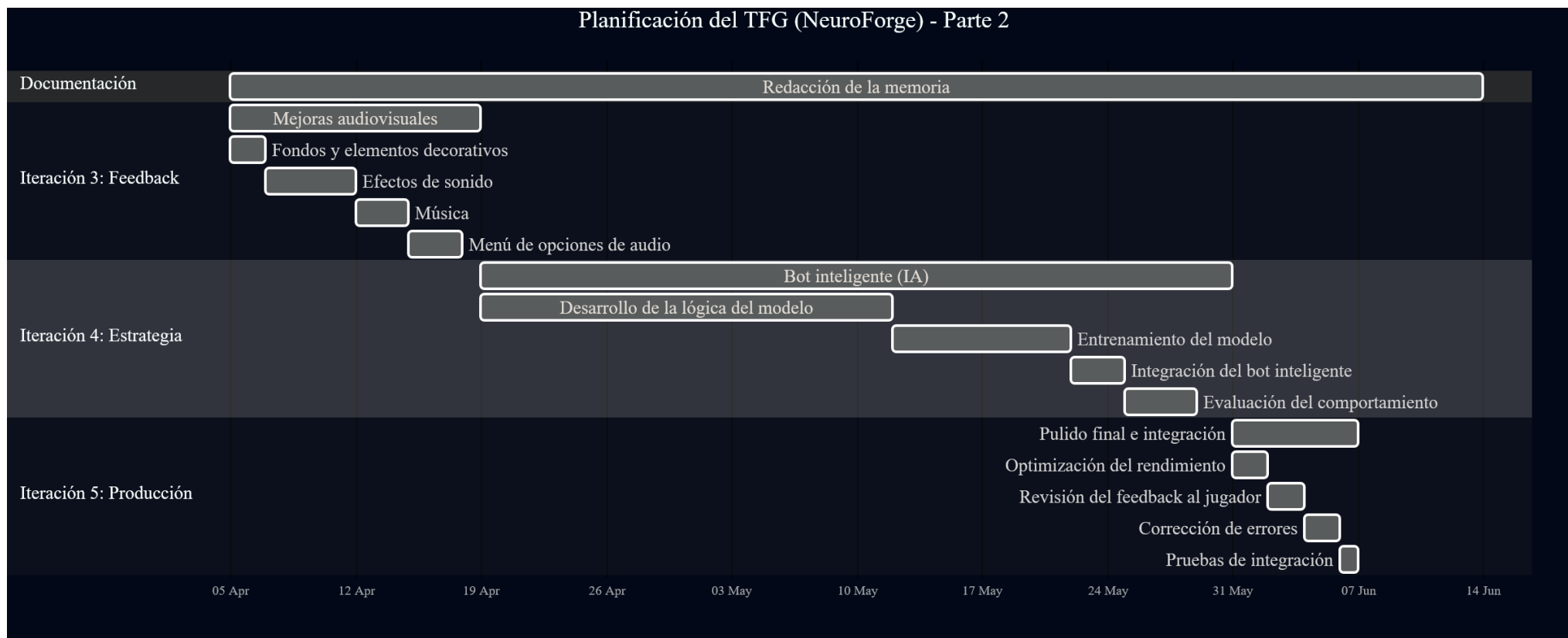


Figura 4.3: Parte 2 del diagrama de Gantt general del desarrollo del TFG

4.3. Riesgos

Durante la fase de planificación se identificaron distintos riesgos que podrían afectar al correcto desarrollo del proyecto, recogidos en la Tabla 4.3. Para cada uno se ha analizado su naturaleza e impacto potencial, definiendo medidas de contingencia orientadas a minimizar sus efectos en caso de materializarse.

La identificación temprana de estos riesgos permite anticipar posibles problemas técnicos y organizativos, facilitando la toma de decisiones durante el desarrollo y contribuyendo a mantener la estabilidad del proyecto.

Riesgo	Tipo	Impacto	Plan de contingencia
R1: Fallos del motor o incompatibilidades	Técnico	Alto	Revisar dependencias y adaptar los scripts afectados.
R2: Retrasos en el cronograma	Temporal	Medio	Reajustar tareas no prioritarias y reducir funcionalidades accesorias.
R3: Errores en el bot o en las reglas del videojuego	Técnico	Medio	Simplificar el comportamiento de la IA y priorizar la estabilidad de las mecánicas fundamentales.
R4: Pérdida de datos o código fuente	Técnico	Alto	Uso de control de versiones y copias de seguridad en la nube.
R5: Falta de tiempo para pruebas y ajuste del videojuego	Temporal	Alto	Priorizar pruebas de jugabilidad y corregir únicamente los errores críticos.
R6: Carencia de recursos gráficos o sonoros adecuados	Logístico	Bajo	Crear recursos propios o adaptar recursos de terceros de libre uso.
R7: Incapacidad temporal para el desarrollo	Temporal	Alto	Reorganizar tareas, priorizar funcionalidades críticas y ajustar el cronograma.
R8: Dificultad en el diseño o implementación del bot inteligente	Técnico	Alto	Simplificar el modelo de decisión si fuera necesario; mantener operativo el bot aleatorio como versión mínima funcional.
R9: Bloqueos por depuración de errores difíciles	Técnico	Medio	Mantener control de versiones; realizar pruebas frecuentes; documentar los problemas encontrados y consultar la documentación oficial de Godot y C#.

Tabla 4.3: Riesgos identificados y planes de contingencia

4.4. Restricciones

El desarrollo de *NeuroForge* ha estado condicionado por una serie de restricciones que han influido directamente en la planificación, la selección de herramientas y el alcance final del proyecto.

- **Técnicas:** Se establece el uso del motor *Godot* junto con el lenguaje de programación *C#*. Aunque Godot emplea de forma nativa GDScript, se ha optado por *C#* debido a su soporte oficial y su mayor adecuación al perfil del proyecto. Asimismo, se limita el uso a librerías y recursos gratuitos o de código abierto.
- **Temporales:** El desarrollo se limita al periodo de realización del TFG, lo que impone una duración finita y condiciona directamente el número de funcionalidades que pueden implementarse.
- **Económicas:** No se dispone de presupuesto para la adquisición de licencias de software de pago, la contratación de colaboradores ni la compra de recursos gráficos o sonoros.
- **Personales:** El proyecto es desarrollado íntegramente por un único estudiante, lo que limita la paralelización de tareas y exige una gestión eficiente del tiempo disponible.
- **Académicas:** El trabajo debe ajustarse a la normativa, estructura y requisitos establecidos por la Escuela de Ingeniería Informática de Valladolid para los Trabajos de Fin de Grado.

4.5. Herramientas

Para llevar a cabo este Trabajo de Fin de Grado, ha sido necesario seleccionar un conjunto integrado de herramientas de software que abarquen todas las necesidades del ciclo de vida del proyecto, desde la propia implementación del videojuego hasta la gestión de tareas, el diseño de recursos y la redacción de esta memoria.

A continuación, se detalla en primer lugar el análisis y la elección de la pieza central del entorno de desarrollo, el motor de videojuegos, para posteriormente describir el resto de herramientas complementarias que conforman el flujo de trabajo.

4.5.1. Selección del Motor de Videojuegos

Un motor de videojuegos es un *framework* de desarrollo que proporciona un conjunto integrado de herramientas, librerías y sistemas sobre los que construir un videojuego, abstrayendo tareas como el renderizado gráfico, la gestión de escenas, el control de entrada del usuario, la simulación de físicas o la exportación a distintas plataformas.

Tal como se describe en el capítulo de Antecedentes, el panorama actual del desarrollo independiente está dominado por *Unreal Engine*, *Unity* y *Godot*, tres motores con enfoques técnicos y modelos de licencia distintos. Para este proyecto se ha seleccionado **Godot** como motor de desarrollo por las siguientes razones técnicas:

- Su motor 2D es un sistema nativo e independiente del motor 3D, lo que lo hace especialmente eficiente para un videojuego de estrategia por turnos bidimensional como *NeuroForge*.
- Su editor ocupa menos de 200 MB y permite ciclos de iteración muy rápidos, ventaja significativa en un proyecto de desarrollo individual con restricciones temporales.
- Se distribuye bajo licencia MIT, eliminando cualquier coste de licencia y garantizando que las condiciones de uso no pueden cambiar de forma unilateral [31].
- La escala del proyecto no requiere el ecosistema de extensiones de los motores comerciales, por lo que la comunidad más reducida de Godot no supone ninguna restricción práctica.

El desarrollo se ha realizado concretamente sobre *Godot* en su versión 4.6.1. Su editor visual se ha utilizado para la composición y gestión de escenas, la definición de nodos y señales, la configuración de la interfaz de usuario mediante el sistema de nodos *Control*, y la gestión de recursos gráficos y sonoros. Su sistema de exportación ha permitido generar ejecutables de escritorio sin dependencias externas, facilitando la distribución del producto final.

4.5.2. Otras herramientas utilizadas

Una vez determinada la elección de Godot como núcleo tecnológico, se seleccionó un conjunto de herramientas de soporte atendiendo a criterios de adecuación técnica, disponibilidad gratuita o bajo coste, compatibilidad con el motor y facilidad de integración en el flujo de trabajo del proyecto.

El desarrollo del videojuego se ha realizado concretamente sobre el motor *Godot* en su versión 4.6.1. Godot ha servido como entorno integral de desarrollo, su editor visual se ha utilizado para la composición y gestión de escenas, la definición de nodos y señales, la configuración de la interfaz de usuario mediante el sistema de nodos *Control*, y la gestión de recursos gráficos y sonoros. Además, su sistema de exportación ha permitido generar ejecutables de escritorio sin dependencias externas, facilitando la distribución del producto final.

Como lenguaje de programación se ha empleado *C#* sobre el entorno de ejecución .NET 9.0, que cuenta con soporte oficial dentro de la versión de Godot utilizada. La elección de *C#* frente a *GDScript*, el lenguaje nativo del motor, se debe a varias razones. Su sistema de tipado estático y fuerte reduce los errores en tiempo de compilación, su orientación a objetos facilita la organización de un proyecto con múltiples sistemas interrelacionados (tablero, piezas, combate, IA), y su amplia adopción en la industria del desarrollo de software proporciona una base de conocimiento y herramientas de depuración más maduras.

El control de versiones del proyecto se ha gestionado mediante *Git*, utilizando *GitHub* como plataforma de alojamiento del repositorio remoto. Git ha permitido mantener un historial completo de todos los cambios realizados en el código fuente, revertir modificaciones problemáticas y trabajar en ramas experimentales sin riesgo para la línea principal de desarrollo. GitHub, por su parte, ha proporcionado una copia de seguridad permanente en la nube y ha facilitado la revisión del código mediante su interfaz web. Se ha utilizado un flujo de trabajo basado en *commits* frecuentes, organizados por funcionalidad o tarea completada.

Para la creación y edición de los recursos gráficos del videojuego se han empleado *Aseprite*. *Aseprite* es un editor especializado en *pixel art*, utilizado para diseñar los sprites de las piezas, los elementos de la interfaz y modificar los assets de terceros. Su soporte nativo para capas, paletas de color y hojas de sprites (*sprite sheets*) lo convierte en una herramienta especialmente adecuada para el estilo visual del proyecto. Paint se ha utilizado de forma complementaria para ediciones rápidas y ajustes menores sobre recursos ya existentes.

La gestión del apartado sonoro se ha apoyado en *Audacity*, un editor de audio de código abierto que ha permitido recortar, normalizar y ajustar el volumen de los efectos de sonido antes de integrarlos en el motor. Los recursos de audio se han obtenido de las plataformas *Freesound* y *Pixabay*, ambas repositorios de efectos sonoros y pistas musicales de libre uso bajo licencias compatibles con proyectos de esta naturaleza. La combinación de estas herramientas ha permitido dotar al videojuego de un apartado sonoro coherente sin necesidad de recurrir a recursos de pago.

La redacción y maquetación de la presente memoria se ha realizado íntegramente mediante *L^AT_EX*, un sistema de composición tipográfica ampliamente utilizado en el ámbito académico y científico. *L^AT_EX* ha facilitado la gestión de la estructura del documento, la numeración automática de secciones, figuras y tablas, la generación de referencias cruzadas y la compilación de la bibliografía mediante *BibL^AT_EX*. Su capacidad para separar contenido y formato ha resultado especialmente útil durante las revisiones sucesivas del documento.

Finalmente, la planificación y el seguimiento del proyecto se han gestionado a través de *Notion*, una herramienta de productividad y gestión de contenido. Notion ha servido como eje organizativo del proyecto, se ha utilizado para llevar a cabo las fases de desarrollo y sus hitos asociados, mantener un registro de las horas invertidas en cada tarea, anotar correcciones y decisiones de diseño pendientes, y elaborar borradores previos a su incorporación a la memoria. También se ha empleado *Mermaid* como soporte para la construcción del diagrama de Gantt y para el seguimiento del progreso global del trabajo.

A modo de síntesis, la Tabla 4.4 ofrece un resumen de conjunto de todas las herramientas mencionadas y su rol dentro de la ejecución de este trabajo.

Tipo	Herramienta	Uso principal
Motor de videojuegos	Godot 4.6.1	Desarrollo integral del videojuego: escenas, lógica, interfaz y exportación.
Lenguaje de programación	C# (.NET 9.0)	Implementación de la lógica del videojuego, sistemas de IA y scripts de control.
Lenguaje de IA y Entrenamiento	Python 3.x	Creación del entorno de Gymnasium, entrenamiento y optimización del bot.
Control de versiones	Git / GitHub	Historial de cambios, gestión de ramas y copia de seguridad en la nube.
Diseño gráfico	Aseprite	Creación de sprites en <i>pixel art</i> , animaciones y edición de recursos visuales.
Audio	Audacity / Freesound / Pixabay	Edición de efectos sonoros y obtención de música y SFX de libre uso.
Documentación	L ^A T _E X / BibL ^A T _E X	Redacción, maquetación y gestión bibliográfica de la memoria del TFG.
Gestión y planificación	Notion / Mermaid	Organización de fases, registro de horas, seguimiento de hitos y borradores.

Tabla 4.4: Herramientas empleadas en el desarrollo del proyecto

4.6. Variación de planificación

A lo largo del desarrollo del proyecto se han producido algunas variaciones respecto a la planificación inicial, aunque sin llegar a afectar de forma significativa al calendario general ni al alcance final del trabajo.

El cambio más relevante respecto a la planificación original fue la reorganización del orden de las iteraciones. En la planificación inicial se contemplaban cuatro iteraciones, en las que el desarrollo del bot inteligente se situaba en segundo lugar, antes del diseño de la interfaz de usuario. Sin embargo, durante el desarrollo se optó por invertir este orden, abordando primero la interfaz y posponiendo el bot a la cuarta iteración.

Esta decisión responde a una priorización deliberada del producto final. Dado que el bot inteligente constituye un objetivo secundario del proyecto, se consideró más adecuado garantizar primero la existencia de un videojuego completo y funcional de principio a fin, con una experiencia de juego clara y pulida. De este modo, aunque el desarrollo del bot no llegara a completarse en el tiempo disponible, el proyecto contaría en todo caso con un resultado presentable y jugable, apoyado en el bot aleatorio ya implementado en la primera iteración como oponente mínimo funcional. Como consecuencia de esta reorganización, la planificación pasó de cuatro a cinco iteraciones, separando las mejoras audiovisuales en una iteración propia. Este ajuste no supuso un incremento en el tiempo total estimado, sino una redistribución del trabajo en fases más cohesionadas.

En cuanto a las desviaciones cronológicas y de esfuerzo registradas, las dos primeras iteraciones experimentaron retrasos en sus fechas de entrega respecto a lo estimado. La primera iteración, cuyo núcleo jugable estaba previsto para el 26 de febrero, no se completó hasta el 19 de marzo. El motivo principal fue la necesidad de revisar y rediseñar aspectos del modelo de datos que no habían sido contemplados con suficiente detalle durante la fase de diseño inicial. Al comenzar la implementación se detectaron inconsistencias en la representación interna de las piezas y su relación con el tablero que obligaron a replantear parte de la arquitectura antes de poder continuar con el desarrollo. Esta revisión supuso una desviación de +12 horas de trabajo sobre el presupuesto de la tarea y desplazó la fecha de entrega de la iteración.

Como consecuencia directa de ese retraso en el calendario, la segunda iteración, dedicada a la interfaz de usuario y el pulido visual, también vio afectada su fecha de cierre, postergándose del 5 de abril al 18 del mismo mes. Sin embargo, en este caso no surgieron dificultades técnicas adicionales significativas, por lo que las horas estimadas de desarrollo se respetaron rigurosamente y la desviación temporal fue puramente un retraso arrastrado desde la iteración anterior.

A partir de este punto, la carga horaria planificada se estabilizó e incluso se logró optimizar el tiempo invertido en las fases restantes, contrarrestando el balance de horas del proyecto. La tercera iteración pudo completarse en la fecha estimada requiriendo 8 horas menos de lo previsto gracias a la solidez de la arquitectura base ya corregida. Del mismo modo, la quinta iteración se cerró reduciendo en 4 horas su estimación inicial. Esta recuperación del tiempo de desarrollo, sumada a una redistribución eficiente de las horas dedicadas en paralelo a la redacción de la memoria, permitió absorber por completo el sobre coste horario de la primera iteración, finalizando el proyecto dentro del cómputo global de horas presupuestado.

La Tabla 4.5 recoge el resumen de las desviaciones producidas respecto a la planificación inicial, detallando las fechas planificadas frente a las reales junto al balance neto de horas de cada fase.

It.	Descripción	Fecha est.	Fecha real	Desviación (h)
T. Global	Análisis y requisitos	2026-02-05	2026-02-05	0 h
T. Global	Diseño de arquitectura	2026-02-12	2026-02-12	0 h
1	Videojuego base	2026-02-26	2026-03-19	+12 h
2	Interfaz de usuario	2026-04-05	2026-04-18	0 h
3	Mejoras audiovisuales	2026-04-19	2026-04-19	-8 h
4	Bot inteligente (IA)	2026-05-31	2026-05-31	0 h
5	Pulido final	2026-06-07	2026-06-07	-4 h
Balance Total de Desviación				0 h

Tabla 4.5: Variaciones en fechas de finalización y desviación respecto a la planificación inicial

4.7. Costes del proyecto

A lo largo del desarrollo de este Trabajo de Fin de Grado el estudiante no ha incurrido en ningún gasto económico directo, todas las herramientas empleadas son de código abierto o gratuitas, y el espacio de trabajo ha sido el domicilio particular. No obstante, con el fin de evaluar la viabilidad del videojuego en un entorno profesional, se presenta a continuación una estimación de los costes en los que se habría incurrido de haberse desarrollado en ese contexto. Se distinguen dos escenarios, el *escenario individual*, que refleja el coste directo para un profesional autónomo, y el *escenario empresarial*, que incorpora las cargas adicionales que asumiría una empresa al contratarlo.

4.7.1. Costes de personal

El coste de personal es el factor dominante en cualquier proyecto de desarrollo de software. Para estimarlo se toma como referencia el salario mensual bruto de un desarrollador junior de videojuegos en España, situado en aproximadamente *2.333 € brutos/mes* (28.000 € anuales según datos de Glassdoor [34]). Dado que la dedicación total del proyecto ha sido de *300 horas* y que una jornada laboral estándar supone unas 160 horas mensuales, la duración equivalente es de *1,875 meses*, lo que da un coste salarial bruto de *4.125,00 €*. Este es el coste en el escenario individual, lo que percibiría el profesional por su trabajo.

En el escenario empresarial, la organización asume además las cotizaciones a la Seguridad Social a su cargo, que en España suponen aproximadamente un 32 % adicional sobre el salario bruto [35], elevando el coste real de personal para la empresa a *5.445,00 €*.

4.7.2. Costes de software

Los costes de software varían según el escenario. En el escenario individual, un profesional autónomo debería adquirir por su cuenta las herramientas necesarias: Aseprite por 16,79 € (pago único), Astah Professional en su licencia individual por 17,98 € y Notion Plus por 22,10 € durante los dos meses de desarrollo técnico efectivo, sumando *56,87 €* [36, 37, 38]. En el escenario empresarial, las licencias escalan a tarifas orientadas a organizaciones, Astah Professional pasa a su modalidad de licencia flotante anual a 180 €/año (15 €/mes) y Notion al plan Business a 20 USD/mes por usuario (aproximadamente 18,40 €/mes), lo que eleva el total de software empresarial a *87,59 €* durante ese mismo periodo. Godot, Git y Audacity no generan coste en ninguno de los dos escenarios al distribuirse bajo licencias de código abierto.

4.7.3. Costes de infraestructura

Los costes de infraestructura difieren de forma significativa entre ambos escenarios.

En el *escenario individual*, el profesional utiliza su domicilio particular como espacio de trabajo. El equipo, valorado en 2.000 € con una vida útil estimada de diez años, genera una amortización de 33,33 € correspondiente a los dos meses de desarrollo técnico. El coste de suministros se estima considerando que el equipo representa un 20 % del consumo eléctrico del hogar (69,34 €/mes según la OCU [39]) junto con una línea de internet de 20 €/mes [40], ascendiendo a 67,74 € para el periodo. El total de infraestructura en este escenario es de 101,07 €.

En el *escenario empresarial*, no resulta viable utilizar el domicilio particular, por lo que es necesario disponer de un espacio de trabajo profesional. La opción más económica es un puesto fijo en un espacio de coworking, cuyo precio orientativo en España oscila entre 150 y 450 €/mes dependiendo de la ciudad; como estimación conservadora se emplean 200 €/mes, lo que supone 400,00 € para los dos meses [41, 42]. El coste de electricidad e internet queda incluido en dicha tarifa. Respecto al equipo, la Agencia Tributaria establece para los equipos informáticos un coeficiente de amortización lineal máximo del 25 % anual [43], lo que sobre un equipo valorado en 2.000 € supone una amortización mensual de 41,67 € y un total de 83,33 € para el periodo. El total de infraestructura en el escenario empresarial asciende a 483,33 €.

4.7.4. Distribución en Steam

El canal de distribución natural para un videojuego independiente en PC es *Steam*. Los requisitos y costes de publicación difieren entre ambos escenarios.

En el *escenario individual*, un profesional autónomo puede registrarse directamente en Steamworks aportando sus datos personales y bancarios. La única tasa aplicable es la tarifa de publicación de 100 USD *por título* (aproximadamente 84,99 € al cambio actual) [44], reembolsable una vez el videojuego acumula 1.000 USD en ventas. Además, Steam retiene una comisión del 30 % sobre cada venta.

En el *escenario empresarial*, Steamworks exige que el firmante del Acuerdo de Distribución sea una entidad legal constituida [45], por lo que es necesario constituir previamente una sociedad, en la práctica habitual una *Sociedad Limitada* (S.L.). Los gastos mínimos de constitución incluyen la certificación de denominación social en el Registro Mercantil Central (aproximadamente 17 €), la escritura pública ante notario (entre 150 y 300 €) y la inscripción en el Registro Mercantil (aproximadamente 150 €) [46, 47]; tomando valores centrales, el coste estimado de constitución es de 392,00 €. A ello se suma la misma tarifa de publicación de Steam de 84,99 € [44]. El coste total asociado a la distribución en Steam es, por tanto, de 84,99 € en el escenario individual y de 476,99 € en el empresarial.

Se establece un *precio de venta de 1,99 €*, coherente con la escala del proyecto y habitual en el catálogo de videojuegos independientes de calidad modesta en Steam. Descontada la comisión del 30 % de la plataforma, el ingreso neto por copia vendida es de 1,39 €.

4.7.5. Resumen y punto de amortización

La Tabla 4.6 recoge el desglose completo de los costes estimados en ambos escenarios. Al coste directo se añade un *10 %* de margen de contingencia, porcentaje habitual en la estimación de proyectos de desarrollo de software según las directrices del *Project Management Body of Knowledge* (PMBOK) [48], destinado a absorber desviaciones e imprevistos no contemplados en la planificación inicial.

Concepto	Escenario individual	Escenario empresarial
Personal	4.125,00 €	5.445,00 €
Software	56,87 €	87,59 €
Infraestructura	101,07 €	483,33 €
Distribución Steam	84,99 €	476,99 €
Subtotal	4.367,93 €	6.492,91 €
Contingencia (10 %)	436,79 €	649,29 €
Total	4.804,72 €	7.142,20 €

Tabla 4.6: Desglose de costes estimados en escenario individual y empresarial

Con un ingreso neto de 1,39 € por copia, el número de unidades necesarias para recuperar la inversión es de *3.457 copias* en el escenario individual y de *5.139 copias* en el empresarial. Estas cifras son teóricamente alcanzables para un título independiente con buenas valoraciones en Steam, plataforma que cuenta con más de 130 millones de usuarios activos, aunque dado el carácter académico del proyecto y la ausencia de campaña de marketing, deben entenderse como una estimación de viabilidad comercial y no como una proyección de ventas real.

Análisis

En este capítulo se define el marco conceptual y funcional del videojuego, estableciendo las bases operativas necesarias para el posterior diseño técnico. En primer lugar, se delimitan los actores y se detallan los requisitos funcionales, no funcionales y de información que gobiernan el sistema. Posteriormente, se especifican los casos de uso a través de sus flujos normales y alternativos para describir detalladamente el comportamiento de la aplicación. Por último, se presenta el modelo de dominio con las entidades clave del juego y se analizan los procesos de negocio mediante diagramas de actividad, ofreciendo una visión dinámica de los turnos, combates y fases del desarrollo de la partida.

5.1. Actores y roles

En este apartado se identifican los actores que interactúan con el Sistema, entendiendo como actor toda entidad externa que inicia o participa en la ejecución de casos de uso.

El único actor del sistema es el **Jugador**, que corresponde a la persona que interactúa con el videojuego a través de la interfaz gráfica mediante el ratón. Es el actor principal y exclusivo, responsable de iniciar y configurar la partida, desplegar sus piezas en la fase inicial, realizar movimientos y ataques durante su turno, y pausar o abandonar la partida en cualquier momento. El resto de la lógica del videojuego, incluyendo las acciones del oponente, es gestionada internamente por el propio sistema sin intervención externa.

5.2. Modelado de Requisitos

Los requisitos describen las condiciones, capacidades y restricciones que debe cumplir un sistema software para satisfacer las necesidades de sus usuarios y los objetivos de la organización. Se distinguen principalmente tres tipos de requisitos: los requisitos funcionales (RF), que especifican los servicios y comportamientos que debe ofrecer el sistema; los requisitos no funcionales (RNF), que establecen restricciones y propiedades de calidad como rendimiento, fiabilidad o usabilidad; y los requisitos de información (RI), que definen los datos que el sistema debe almacenar y gestionar para proporcionar dichos servicios.

5.2.1. Requisitos funcionales

- **RF01:** El Sistema permitirá iniciar una nueva partida desde un estado inicial controlado, inicializando el tablero, las piezas y el turno inicial conforme a las reglas del videojuego.
- **RF02:** El Sistema permitirá al Jugador colocar sus piezas al inicio de la partida dentro de la zona de despliegue definida.
- **RF03:** El Sistema gestionará los turnos alternos entre el Jugador y el bot, impidiendo la realización de acciones fuera de turno.
- **RF04:** El Sistema validará las acciones realizadas por el Jugador, evitando movimientos o ataques no permitidos por las reglas del videojuego.
- **RF05:** El Sistema revelará la identidad de las piezas implicadas cuando se produzca un combate o movimiento característico, manteniendo visible la información revelada durante el resto de la partida.
- **RF06:** El Sistema hará que el bot tome las decisiones de movimiento y ataque de forma autónoma durante su turno, respetando las reglas y restricciones del videojuego.
- **RF07:** El Sistema mantendrá y actualizará el estado de la partida durante su desarrollo, gestionando turnos, acciones válidas y transiciones entre estados.
- **RF08:** El Sistema determinará automáticamente el final de la partida cuando se cumpla una condición de victoria o derrota, mostrando el resultado al Jugador.
- **RF09:** El Sistema mostrará información visual y sonora relacionada con las acciones realizadas, los combates y los cambios de estado de la partida.
- **RF10:** El Sistema permitirá al Jugador pausar o abandonar la partida en curso de forma controlada.
- **RF11:** El Sistema brindará al Jugador con la información necesaria para entender las reglas del videojuego en caso de no conocerlas.
- **RF12:** El Sistema permitirá al Jugador configurar los parámetros de audio del videojuego (volumen de música, efectos de sonido, etc.) desde el menú de opciones antes y durante una partida.

5.2.2. Requisitos no funcionales

- **RNF01:** El sistema deberá ser compatible con Windows 10, Windows 11, y distribuciones de Linux lanzadas a partir de 2018.
- **RNF02:** El videojuego deberá ejecutarse correctamente en un equipo con las siguientes especificaciones mínimas de hardware:
 - Procesador: CPU de doble núcleo a 2,0 GHz o superior.
 - Memoria RAM: 4 GB.
 - Tarjeta gráfica: GPU con soporte completo de Vulkan 1.0 (por ejemplo, Intel HD Graphics 510 o superior).
 - Almacenamiento: 200 MB de espacio libre en disco.
- **RNF03:** El sistema deberá responder a las acciones del jugador en un tiempo máximo de 2 segundos en condiciones normales de ejecución, garantizando una experiencia de juego fluida.
- **RNF04:** La interfaz deberá ser suficientemente clara e intuitiva para que un jugador sin experiencia previa en el videojuego pueda comprender las acciones disponibles en cada momento sin necesidad de consultar las reglas.
- **RNF05:** La interfaz del videojuego será completamente manejable mediante ratón.

5.2.3. Requisitos de información

- **RI01 — Partida:** El Sistema gestionará una sesión de juego con la siguiente información:
 - Referencia al tablero de juego.
 - Turno actual (Jugador o Bot).
 - Estado de la partida (Fase de despliegue, turno del Jugador, pieza seleccionada y fin de la partida).
 - Número de turno.
 - Conjunto de casillas que conforman el tablero.
- **RI02 — Casilla:** Cada casilla del tablero almacenará la siguiente información:
 - Posición en la cuadrícula (x, y) .
 - Tipo (Pasable, no pasable, despliegue del Jugador y del bot).
 - Pieza que la ocupa, si la hubiera.

- **RI03 — Pieza:** El Sistema almacenará la siguiente información para cada pieza presente en la partida:
 - Tipo de pieza (Los 12 tipos de piezas que vienen recogidos en el capítulo de Diseño del videojuego).
 - Rango numérico que determina el resultado de los combates.
 - Propietario (Jugador o Bot).
 - Si la pieza puede moverse o no.
 - Si es visible para el Jugador.
 - Si el bot conoce la identidad de la pieza.
 - Casilla del tablero en la que se situa.
 - Historial de los últimos tres movimientos realizados como origen-destino.

5.3. Casos de uso

En este apartado se describen los casos de uso del Sistema, que representan las interacciones posibles entre el actor y el Sistema para alcanzar un objetivo concreto. Cada caso de uso recoge el flujo normal de acciones, las posibles excepciones y los requisitos funcionales asociados. La Figura 5.1 muestra el diagrama general de casos de uso, donde se reflejan las relaciones entre el actor, los casos de uso principales y las dependencias entre ellos.

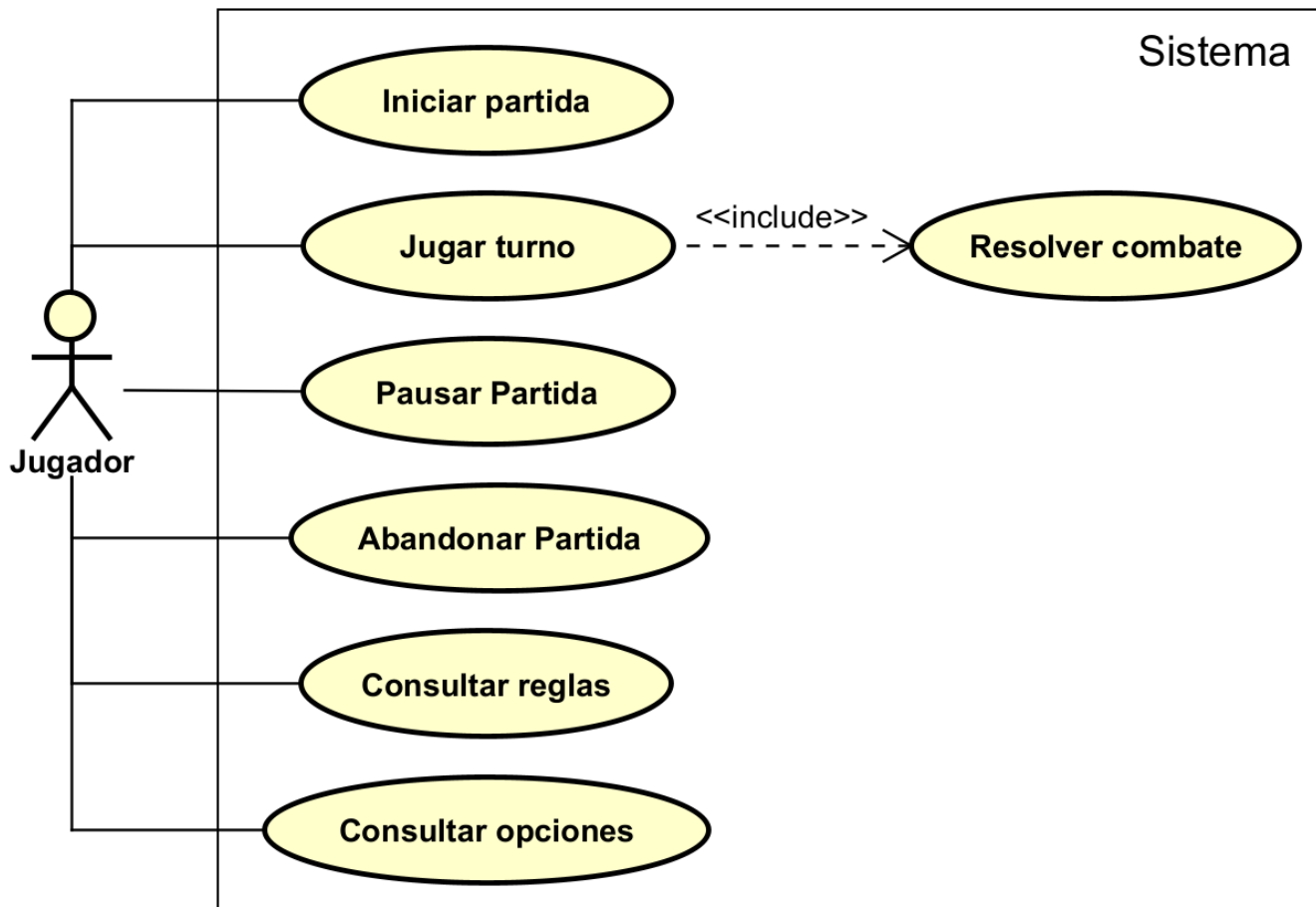


Figura 5.1: Diagrama de casos de uso

5.3.1. Lista de casos de uso

ID	Nombre	Descripción
CU-01	Iniciar partida	El Sistema permite al Jugador iniciar una nueva partida.
CU-02	Jugar ronda	El Sistema permite al Jugador llevar a cabo un turno completo durante el transcurso de una partida.
CU-03	Resolver combate	El Sistema gestiona el combate entre 2 piezas de manera automática.
CU-04	Pausar partida	El Sistema permitirá al Jugador pausar en mitad de una partida cuando sea su turno.
CU-05	Abandonar partida	El Sistema permitirá al Jugador salir durante el transcurso de una partida cuando sea su turno.
CU-06	Consultar reglas	El Sistema permite al Jugador consultar las reglas dentro del videojuego.
CU-07	Consultar opciones	El Sistema permite al Jugador consultar y configurar las opciones.

Tabla 5.1: Lista de casos de uso del Sistema

5.3.2. CU-01: Iniciar partida

ID	CU-01
Nombre	Iniciar partida
Actor	Jugador
Descripción	El Sistema permite al Jugador iniciar una nueva partida.
Req. relacionados	RF01, RF02.
Pre-condición	El Sistema se encuentra mostrando la escena del menú principal, o la del final de partida.
Secuencia normal	1. El Jugador selecciona la opción de jugar una partida. 2. El Sistema inicia el tablero y espera a que el Jugador distribuya sus piezas en su zona. 3. El Jugador coloca todas sus piezas en el tablero. 4. El Jugador solicita comenzar la partida. 5. El Sistema distribuye todas las piezas del bot e inicia la partida.
Flujos alternativos	3.a. El Jugador solicita pausar el juego → Se inicia CU-04 (Pausar partida). 4.a. No están todas las piezas colocadas → Regresa al paso 3.
Post-condición	La partida comienza con el primer turno el cual espera acción del Jugador.

Tabla 5.2: CU-01: Iniciar partida

5.3.3. CU-02: Jugar ronda

ID	CU-02
Nombre	Jugar ronda
Actor	Jugador
Descripción	El Sistema permite al Jugador llevar a cabo un turno completo durante el transcurso de una partida.
Req. relacionados	RF03, RF04, RF05, RF06, RF07, RF08, RF09, RF10.
Pre-condición	El Jugador se encuentra jugando a mitad de una partida y es su turno.
Secuencia normal	1. El Jugador solicita mover una de sus piezas. 2. El Sistema realiza la acción solicitada por el Jugador. 3. El Sistema cede el turno al bot. 4. El Sistema ejecuta el turno del bot, realizando una acción valida con una de sus piezas. 5. El Sistema cede el turno al Jugador.
Flujos alternativos	1.a. El Jugador no puede mover ninguna pieza → el Sistema termina la partida y el caso de uso finaliza. 1.b. El Jugador solicita hacer un movimiento inválido → el Sistema ignora la acción y regresa al paso 1. 1.c. El Jugador solicita pausar la partida → Se inicia CU-04 (Pausar partida), y una vez terminado el caso de uso continua. 2.a. La pieza se ha movido a una casilla vacía con un movimiento característico → El Sistema revela la identidad de esa pieza y el caso de uso continua. 2.b. La casilla destino esta ocupada por una pieza del bot → Se inicia CU-03 (Resolver combate), y una vez terminado el caso de uso continua. 3.a. Tras la acción del Jugador se da una condición para finalizar la partida → El Sistema termina la partida y el caso de uso finaliza. 4.a. La pieza se ha movido a una casilla vacía con un movimiento característico → El Sistema revela la identidad de esa pieza y el caso de uso continua. 4.b. La casilla destino esta ocupada por una pieza del Jugador → Se inicia CU-03 (Resolver combate), y una vez terminado el caso de uso continua. 5.a. Tras la acción del bot se da una condición para finalizar la partida → El Sistema termina la partida y el caso de uso finaliza.
Post-condición	N/A

Tabla 5.3: CU-02: Jugar ronda

5.3.4. CU-03: Resolver combate

ID	CU-03
Nombre	Resolver combate
Actor	Jugador
Descripción	El Sistema gestiona el combate entre 2 piezas de manera automática.
Req. relacionados	RF05, RF09.
Pre-condición	Durante una partida en curso, se ha movido una pieza a una casilla ocupada por otra pieza del bando rival.
Secuencia normal	1. El Sistema revela las piezas que entran en combate. 2. El Sistema lleva a cabo el combate y obtiene el resultado según las reglas. 3. El Jugador conoce el resultado y solicita continuar con la partida. 4. El Sistema elimina del tablero la pieza que resulta perdedora del combate. En caso de que pierdan ambas piezas se eliminan ambas.
Flujos alternativos	1.a. No hay piezas ocultas → El Sistema ignora este paso y continua al paso 2.
Post-condición	N/A

Tabla 5.4: CU-03: Resolver combate

5.3.5. CU-04: Pausar partida

ID	CU-04
Nombre	Pausar partida
Actor	Jugador
Descripción	El Sistema permitirá al Jugador pausar en mitad de una partida cuando sea su turno.
Req. relacionados	RF10.
Pre-condición	El Jugador se encuentra jugando su turno en el transcurso de una partida.
Secuencia normal	1. El Jugador solicita pausar la partida. 2. El Sistema pausa la partida. 3. El Jugador decide continuar con la partida. 4. El Sistema permite al Jugador continuar con la partida.
Flujos alternativos	1.a. El Jugador decide consultar las reglas → Se inicia CU-06 (Consultar reglas), y una vez terminado el caso de uso continua. 1.b. El Jugador decide consultar las opciones → Se inicia CU-07 (Consultar opciones), y una vez terminado el caso de uso continua. 1.c. El Jugador decide abandonar la partida → Se inicia CU-05 (Abandonar partida), y el caso de uso finaliza.
Post-condición	N/A

Tabla 5.5: CU-04: Pausar partida

5.3.6. CU-05: Abandonar partida

ID	CU-05
Nombre	Abandonar partida
Actor	Jugador
Descripción	El Sistema permitirá al Jugador salir durante el transcurso de una partida cuando sea su turno.
Req. relacionados	RF10.
Pre-condición	El Jugador está en una partida actualmente pausada.
Secuencia normal	1. El Jugador solicita abandonar la partida. 2. El Sistema finaliza la partida.
Flujos alternativos	■ N/A
Post-condición	N/A

Tabla 5.6: CU-05: Abandonar partida

5.3.7. CU-06: Consultar reglas

ID	CU-06
Nombre	Consultar reglas
Actor	Jugador
Descripción	El Sistema permite al Jugador consultar las reglas dentro del videojuego.
Req. relacionados	RF11.
Pre-condición	N/A
Secuencia normal	1. El Jugador solicita consultar las reglas del videojuego. 2. El Sistema da a conocer las reglas al Jugador.
Flujos alternativos	■ N/A
Post-condición	N/A

Tabla 5.7: CU-06: Consultar reglas

5.3.8. CU-07: Consultar opciones

ID	CU-07
Nombre	Consultar opciones
Actor	Jugador
Descripción	El Sistema permite al Jugador consultar y configurar las opciones.
Req. relacionados	RF12.
Pre-condición	N/A
Secuencia normal	1. El Jugador solicita configurar las opciones del videojuego. 2. El Sistema permite al Jugador modificar las distintas opciones de manera controlada.
Flujos alternativos	■ N/A
Post-condición	N/A

Tabla 5.8: CU-07: Consultar opciones

5.4. Modelo de dominio

El modelo de dominio es una representación visual de las entidades conceptuales (objetos del mundo real o del Sistema) y las relaciones que existen entre ellas dentro del contexto del videojuego. Su objetivo es identificar los conceptos clave, sus atributos y cómo interactúan, sirviendo como puente entre los requisitos funcionales y el diseño técnico final.

A continuación, en la Figura 5.2, se presenta el modelo de dominio que estructura la lógica del videojuego.

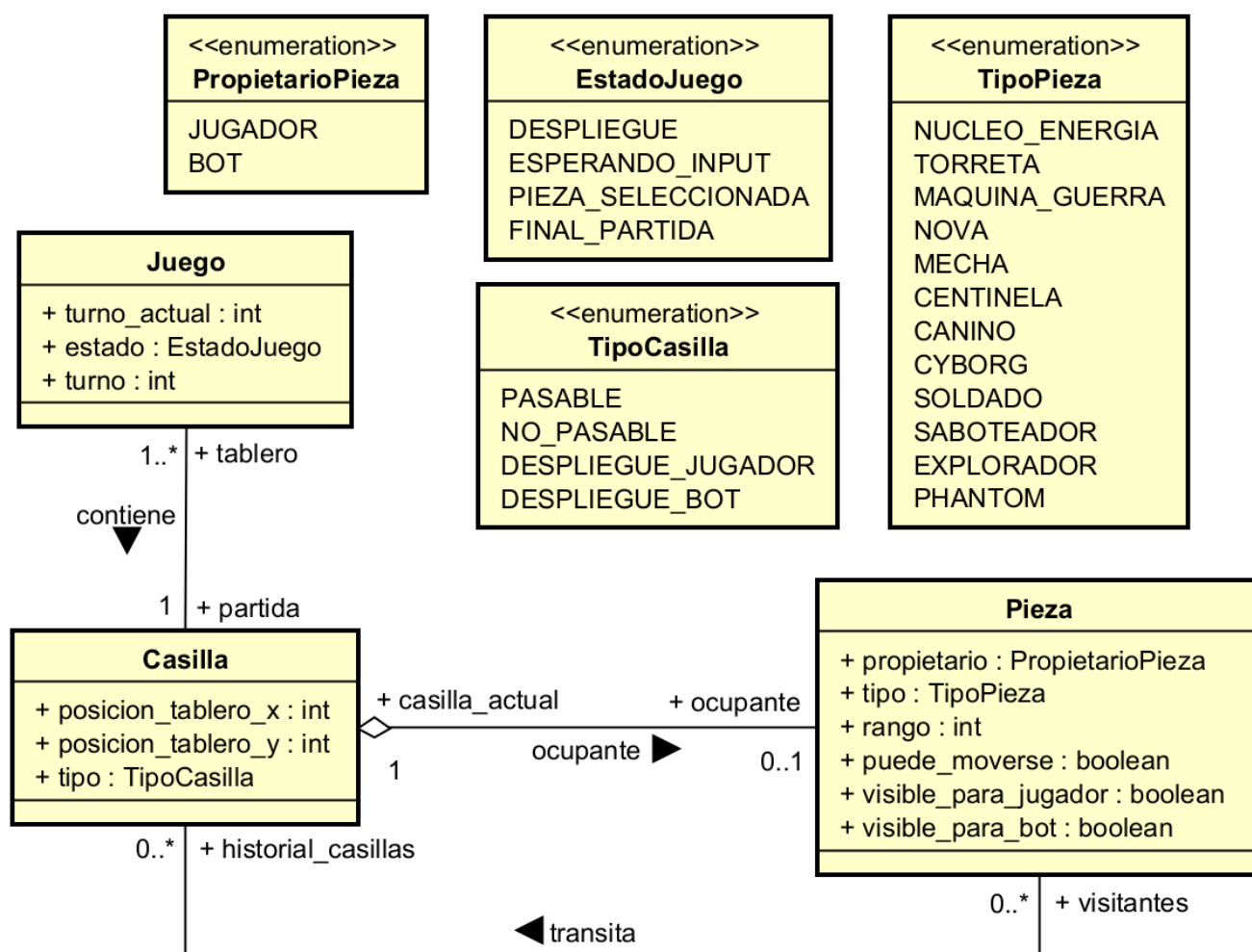


Figura 5.2: Diagrama del modelo de dominio

5.5. Modelo de proceso de negocio

El modelo de proceso de negocio describe el flujo detallado de actividades y decisiones que tienen lugar durante la ejecución del Sistema. Mientras que los casos de uso definen *qué* hace el Sistema, los diagramas de actividad reflejan *cómo* se ejecutan dichas interacciones, mostrando la secuencia lógica, las bifurcaciones y el procesamiento de datos.

A continuación, se detallan los flujos de actividad correspondientes a los casos de uso mas importantes del Sistema, proporcionando una visión dinámica de la lógica del videojuego.

5.5.1. Actividad: Iniciar partida

El diagrama de actividad de la Figura 5.3 representa el CU-01 (Iniciar Partida), que transcurre de forma que el Jugador decide iniciar una nueva partida, el Sistema instancia las piezas con su identidad oculta para el rival, tanto del Jugador según las va colocando, como las del bot cuando el Jugador solicita iniciar la partida.

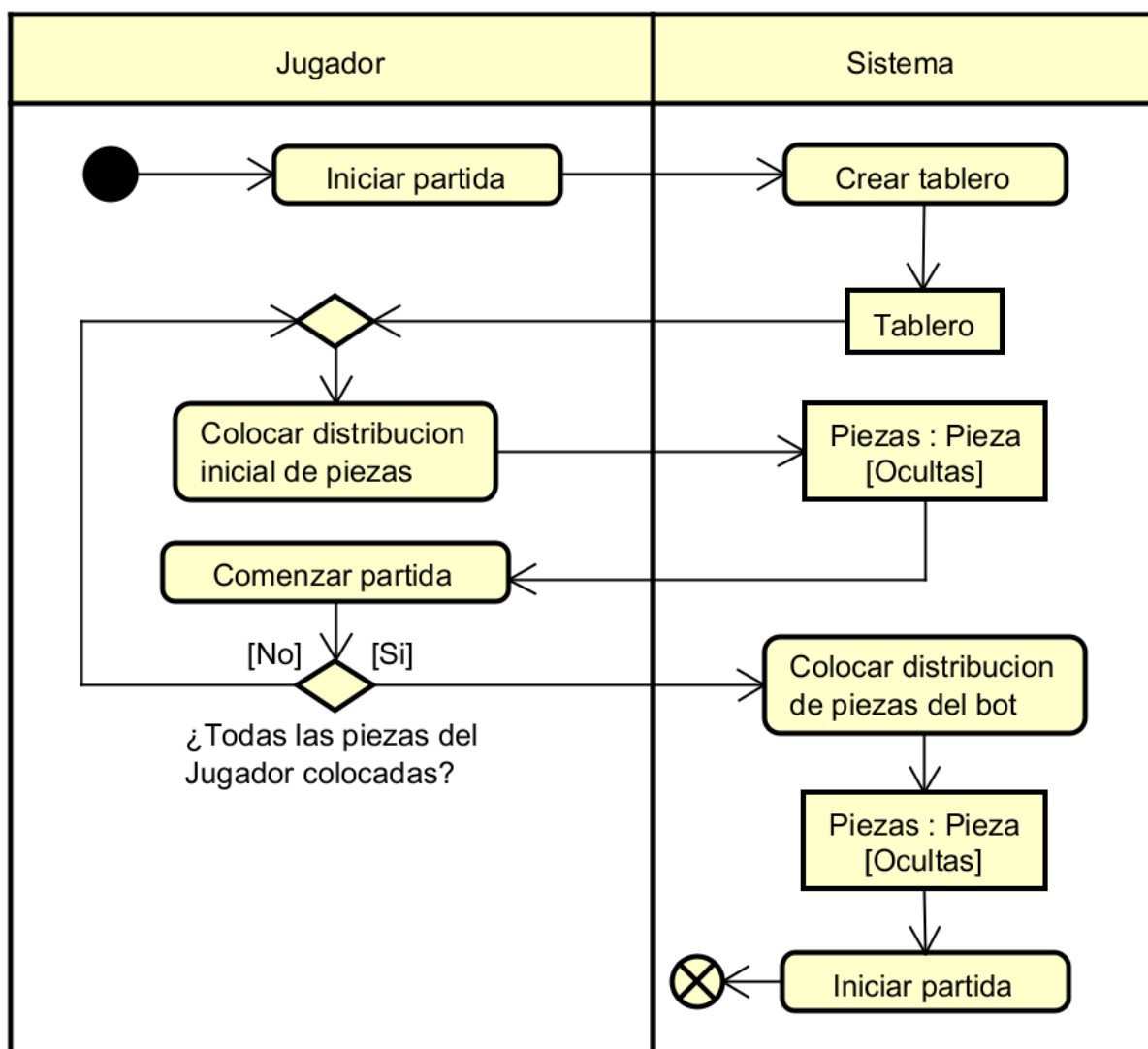


Figura 5.3: Diagrama de actividad: Iniciar partida

5.5.2. Actividad: Jugar ronda

El diagrama de actividad que se muestra en la Figura 5.4, representa el CU-02 (Jugar ronda), en el cual se puede ver el transcurso del turno del jugador y del bot. Desde un movimiento de pieza normal, uno peculiar que revela la identidad de una pieza, o un movimiento que acaba en la casilla de una pieza rival y se disputa un combate entre ambas piezas.

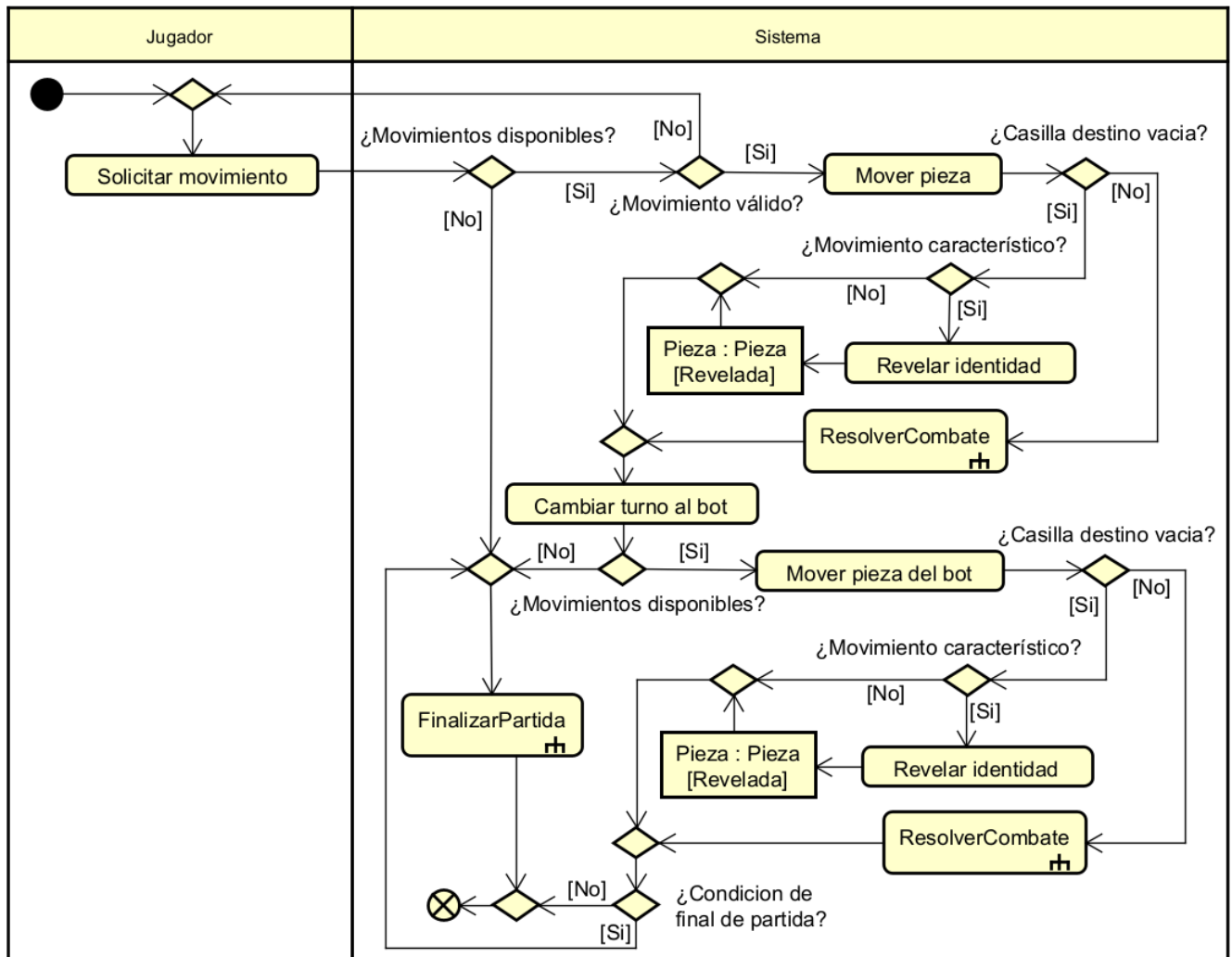


Figura 5.4: Diagrama de actividad: Jugar ronda

5.5.3. Actividad: Resolver combate

El diagrama de la Figura 5.5 muestra el transcurso de un combate entre 2 piezas. Siempre que una pieza entra a combate queda revelada, y aquella que pierde según las reglas del videojuego, es eliminada. En caso de empate ambas se eliminan.

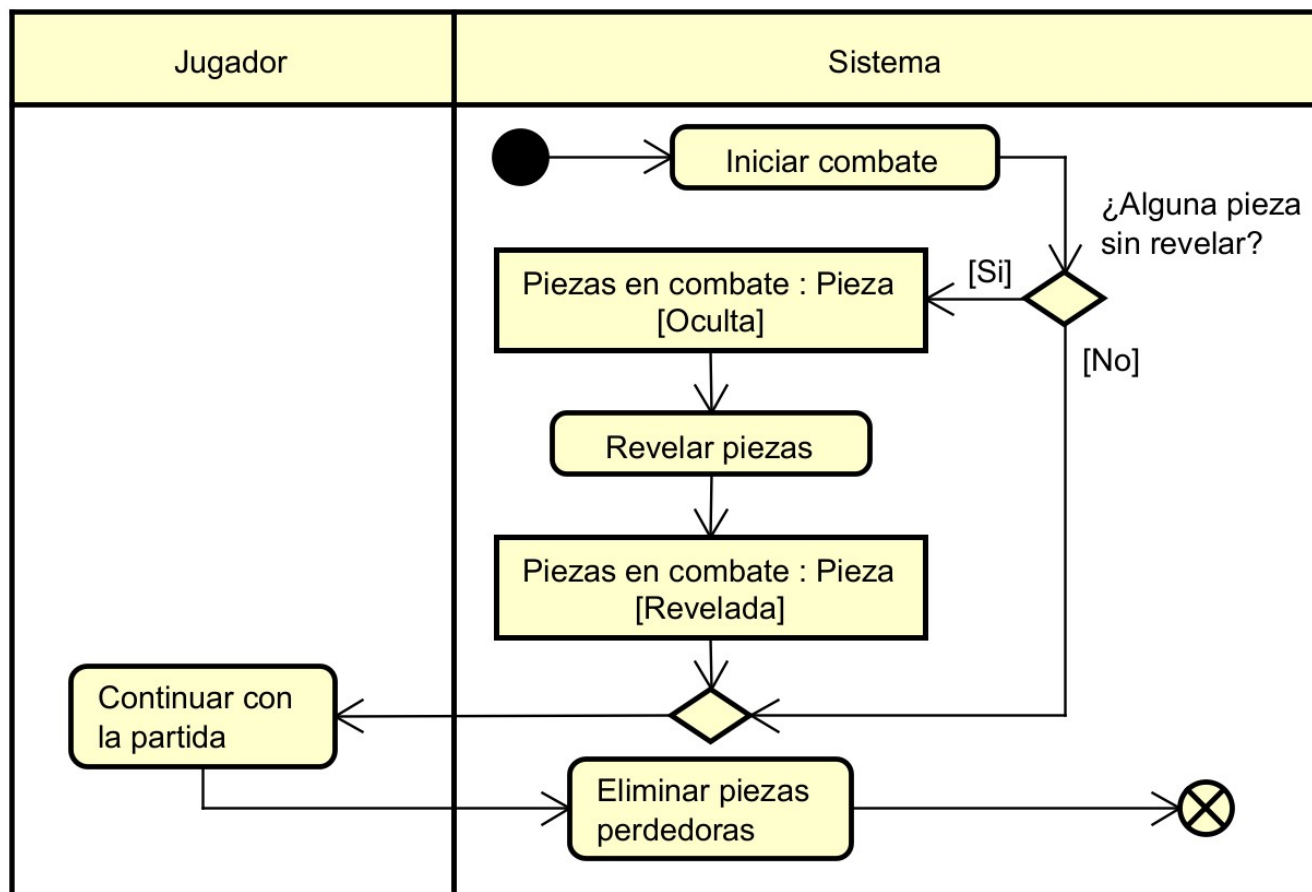


Figura 5.5: Diagrama de actividad: Resolver combate

Diseño

En este capítulo se describe el diseño técnico de *NeuroForge* a partir del código implementado. Se aborda la arquitectura general del sistema basada en el modelo de Godot, la separación en tres capas para aislar la lógica del juego, los patrones de diseño aplicados, el flujo detallado de la interfaz de usuario y, por último, el diagrama de despliegue físico del ejecutable y el modelo de la IA.

6.1. Arquitectura del sistema

En este apartado se describe cómo se organiza internamente el sistema de *NeuroForge*, desde el modelo de ejecución propio del motor hasta la estructura en capas adoptada para separar la lógica del videojuego de su representación visual. Comprender primero el funcionamiento de Godot es necesario para contextualizar las decisiones de diseño tomadas durante el desarrollo, ya que la arquitectura del sistema está directamente condicionada por las capacidades y restricciones que el motor impone.

6.1.1. El modelo de ejecución de Godot

Para comprender las decisiones de diseño adoptadas en este proyecto es necesario entender primero cómo organiza Godot el contenido de un videojuego. Godot estructura toda la lógica y los elementos visuales en un *árbol de escenas* (*SceneTree*), una jerarquía de objetos llamados *nodos*, donde cada nodo representa una unidad funcional del videojuego, ya sea un personaje, una casilla del tablero, un menú o el gestor de partida.

Cada nodo hereda, en última instancia, de la clase base `Node`. Esta es la unidad mínima funcional del motor; no posee posición ni representación visual, pero define el ciclo de vida compartido por todos los objetos del árbol. El método `Ready()` se ejecuta cuando el nodo entra al árbol y es el punto habitual de inicialización, mientras que `Process()` se invoca en cada fotograma para la lógica continua.

A partir de esta base, el proyecto utiliza especializaciones más complejas que expanden las capacidades del nodo raíz. Este mecanismo de herencia permite que, por ejemplo, **Node2D** añada propiedades de posición, rotación y escala para elementos en el mundo 2D (como las piezas y el tablero), mientras que **Control** se especializa en interfaces de usuario con gestión de anclajes y márgenes. Gracias a esta estructura, incluso los componentes de lógica pura pueden integrarse en el ciclo de vida del motor heredando simplemente de **Node**, evitando así la sobrecarga de datos espaciales que no requieren.

Dicha jerarquía de nodos se organiza de forma práctica mediante *escenas*, que funciona como un plano reutilizable compuesto por una rama de nodos. Una escena permite agrupar estos nodos especializados para que trabajen como una única entidad lógica dentro del árbol principal del videojuego.

Un ejemplo representativo en NeuroForge es la escena **Tile**, la cual dentro del editor se ve como en la Figura 6.1, encargada de las casillas del tablero. Como se observa en la imagen, el nodo raíz es un **Area2D**; este nodo no solo hereda las capacidades espaciales de **Node2D**, sino que aporta funcionalidades propias, como por ejemplo, permitir detectar eventos de ratón y colisiones. Bajo él, la escena se compone de hijos específicos: un **CollisionShape2D** para delimitar el área de interacción de dicho área (Específicamente los **Areas2D** necesitan un **CollisionShape2D** hijo para funcionar) y nodos **Sprite2D** para la representación visual. Esta composición ilustra cómo la suma de nodos permite construir objetos más complejos y funcionales.

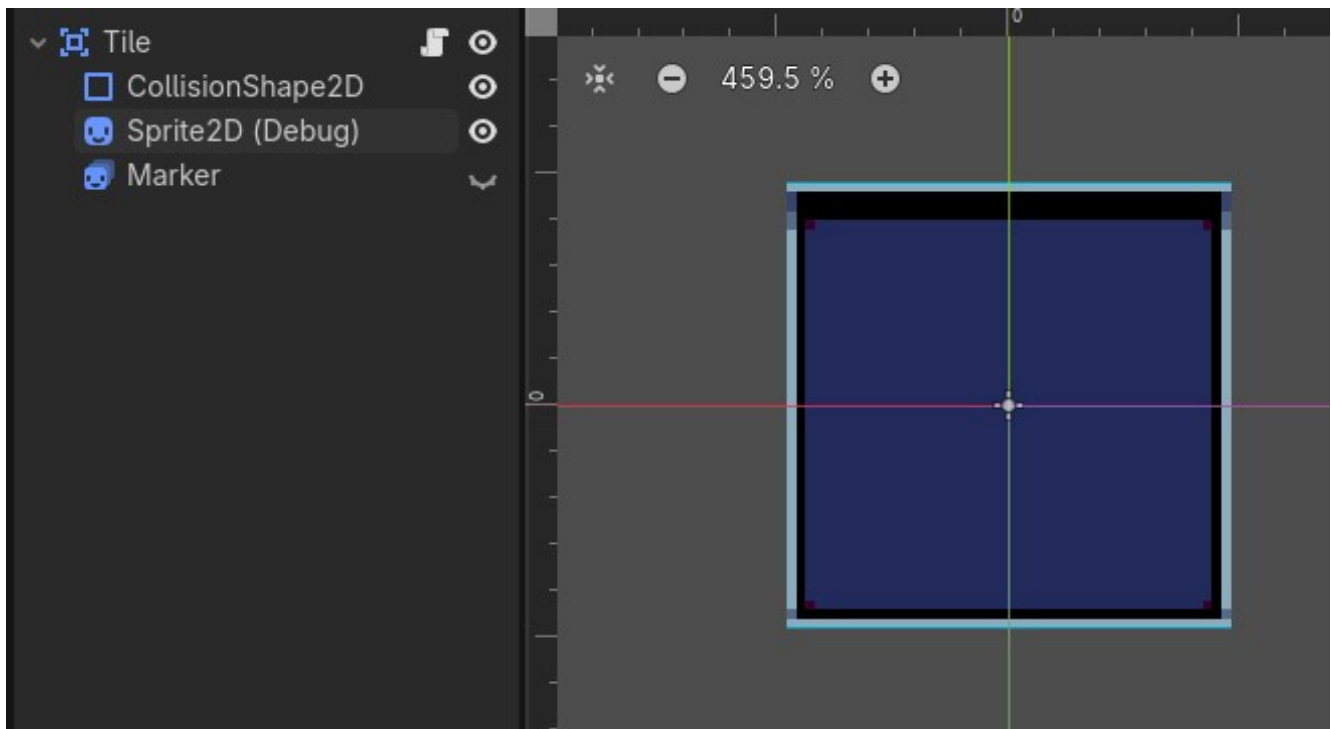


Figura 6.1: Estructura de la escena **Tile** en el editor de Godot

La comunicación entre nodos puede realizarse de tres formas. La primera es mediante referencias directas, donde un nodo obtiene acceso a otro con `GetNode()` y llama a sus métodos directamente. La segunda es mediante señales, el sistema de eventos nativo de Godot, un nodo emite una señal cuando ocurre algo relevante y otros nodos suscritos reaccionan sin necesidad de conocerse mutuamente. Para esta última he optado por utilizar los eventos de C# estándar (event Action), utilizados para una comunicación más rápida y con tipado fuerte entre clases de lógica pura y controladores, reduciendo así la dependencia con el editor de Godot. La tercera es mediante clases estáticas o instancias globales, accesibles desde cualquier punto del código sin depender de la posición en el árbol. Una característica importante de Godot con C# es que no toda clase tiene que ser un nodo, es válido definir clases C# ordinarias que operen sobre los datos del árbol sin pertenecer a él, lo que permite aislar la lógica de negocio del ciclo de vida del motor.

La Figura 6.2 ilustra este modelo con un subconjunto representativo de los nodos del proyecto. No pretende recoger la totalidad del árbol de escenas, sino mostrar algunos tipos de nodos más relevantes, su jerarquía y algunos de los tres mecanismos de comunicación descritos.

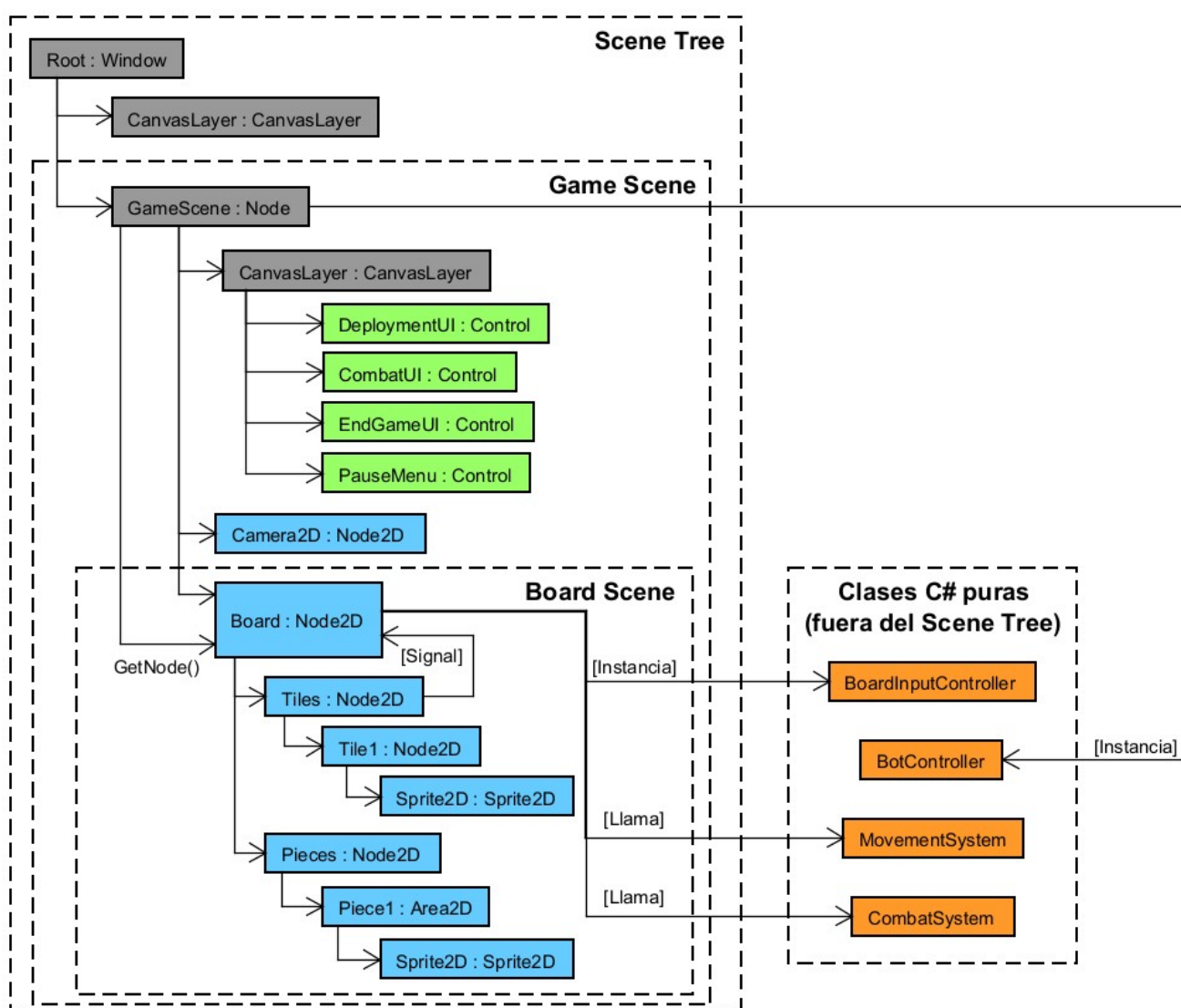


Figura 6.2: Modelo de ejecución de Godot ilustrado con algunos nodos del proyecto.

6.1.2. Arquitectura de NeuroForge

Con este modelo en mente, el sistema se organiza en una arquitectura en tres capas que separa la representación visual de la lógica pura del videojuego y de los datos. Esta separación es una decisión de diseño para evitar que las reglas del videojuego queden entrelazadas con el código de renderizado o de interfaz.

La Figura 6.3 muestra una vista simplificada de los principales componentes de cada capa. Se incluyen los elementos más representativos de cada nivel.

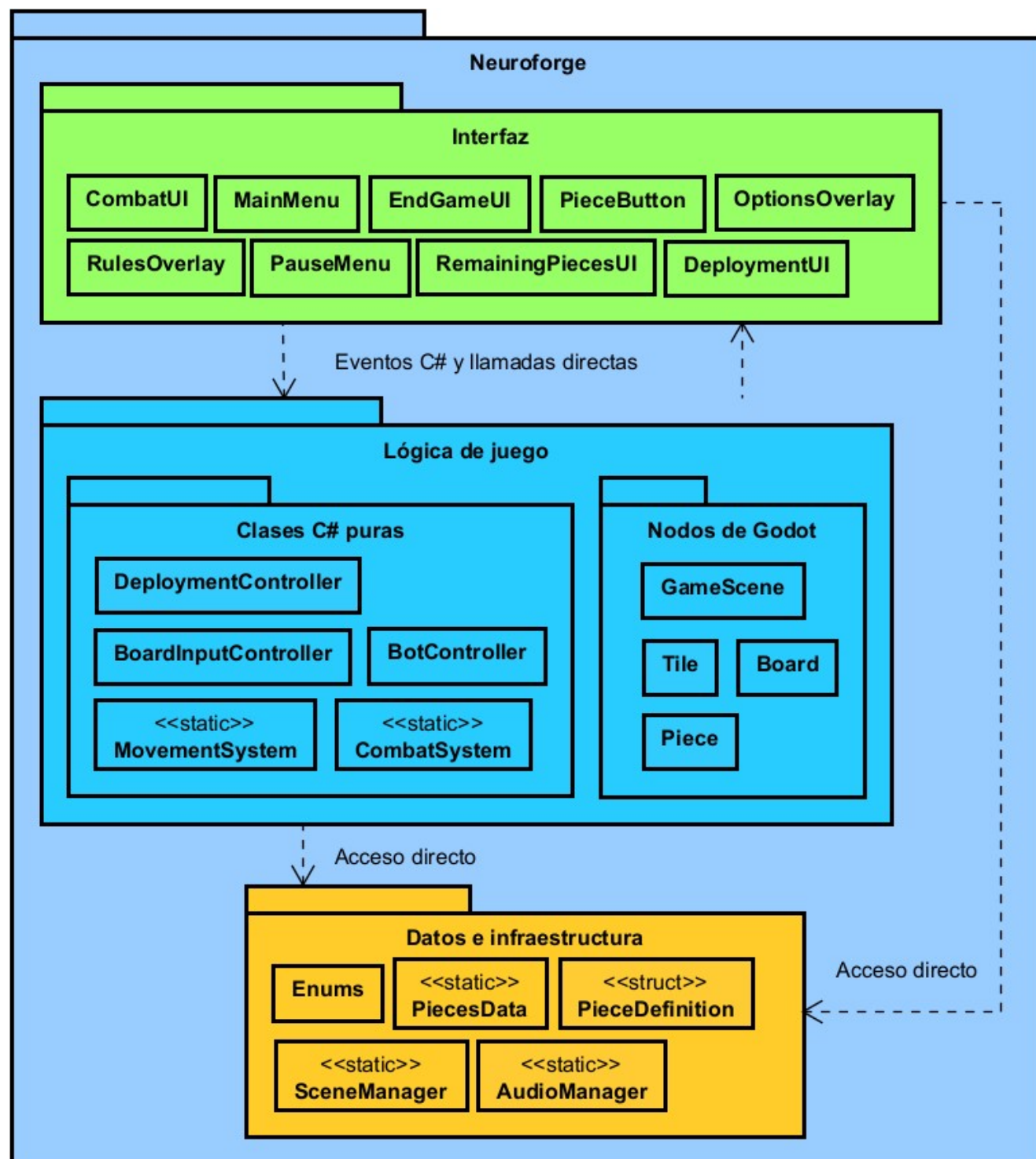


Figura 6.3: Vista simplificada de la arquitectura de NeuroForge en tres capas.

La capa de interfaz agrupa todos los nodos que heredan de **Control**. Su responsabilidad se limita a capturar las intenciones del jugador y reflejar el estado del videojuego visualmente. Ninguno de estos nodos toma decisiones de la lógica del videojuego, cuando el jugador pulsa un botón, la capa de presentación notifica a la capa de lógica mediante eventos C# estándar (**event Action**), y espera recibir los datos actualizados para refrescar la vista. Algunos componentes de esta capa acceden además directamente a la capa de datos e infraestructura para consultar configuración estática o gestionar transiciones de escena.

La capa de lógica de videojuego es el núcleo funcional del sistema. Aquí conviven dos tipos de clases con naturalezas distintas. Nodos de Godot, que necesitan estar en el árbol de escenas para gestionar el input del ratón, coordinar animaciones y participar en el ciclo de vida del motor. Y por otro lado, las clases C# puras creadas e instanciadas desde **GameScene** o **Board**, independientes del motor y por tanto más ligeras y fáciles de reutilizar.

La capa de datos e infraestructura comprende los datos estáticos de configuración, estructuras de datos, el gestor de navegación entre escenas, el gestor de audio y los enums que definen los valores del dominio.

El flujo predominante de dependencias es descendente. Sin embargo, los nodos de Godot, en la capa de lógica, mantienen referencias directas a componentes de la capa de interfaz para coordinar la presentación de resultados y el flujo de partida. Estas dependencias inversas puntuales son una consecuencia del modelo de ciclo de vida de Godot, en el que la lógica debe suspender su ejecución hasta que las animaciones de interfaz terminan. No representan una violación del diseño en capas sino una adaptación pragmática al motor.

6.2. Patrones de diseño aplicados

A lo largo del desarrollo de NeuroForge se han aplicado varios patrones de diseño clásicos. En cada caso el patrón responde a un problema concreto del dominio.

6.2.1. Singleton

El patrón Singleton garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella. En el proyecto aparece en dos formas distintas.

GameScene mantiene una propiedad estática **Instance** que se asigna en **_Ready()** cuando Godot instancia la escena. Cualquier clase del sistema puede consultar el estado de la partida o cambiar de fase a través de ese punto de acceso sin necesidad de mantener una referencia explícita.

SceneManager adopta la variante de *clase completamente estática*, todo su estado (historial de navegación, caché de escenas) vive en campos estáticos privados y se auto-inicializa la primera vez que se invoca cualquiera de sus métodos públicos. La unicidad no recae en una instancia sino en el propio espacio de nombres estático.

Como muestra la Figura 6.4, en ambos casos se evita pasar referencias a través de múltiples capas de objetos, centralizando el acceso a los dos gestores de mayor alcance del sistema.

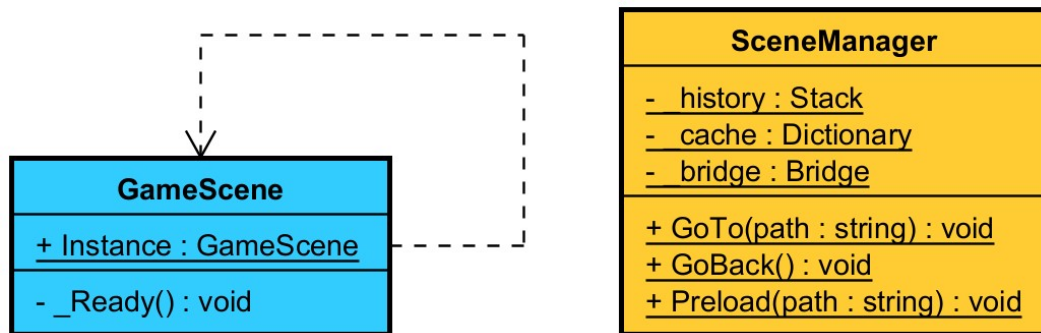


Figura 6.4: Patrón Singleton aplicado en GameScene y SceneManager.

6.2.2. Strategy

El patrón Strategy define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables, permitiendo que el algoritmo varíe de forma independiente respecto a los clientes que lo utilizan.

En el proyecto, MovementSystem y CombatSystem son clases estáticas que encapsulan respectivamente la lógica de movimiento y la lógica de resolución de combate, actuando cada una como una estrategia concreta. Board delega en ellas sin conocer su implementación interna, para mover una pieza consulta MovementSystem.GetAction() y MovementSystem.CanMove(), y para resolver un enfrentamiento invoca CombatSystem.Resolve(). La Figura 6.5 muestra esta relación de delegación.

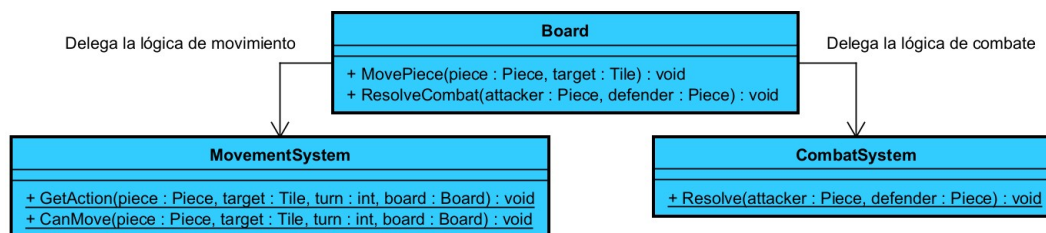


Figura 6.5: Patrón Strategy con MovementSystem y CombatSystem como estrategias delegadas desde Board.

Cabe señalar que en esta implementación las estrategias no comparten una interfaz común explícita ni se intercambian en tiempo de ejecución, ya que el lenguaje C# permite referenciar clases estáticas directamente. No obstante, la intención estructural del patrón se cumple, el algoritmo queda completamente aislado del contexto que lo usa, de modo que modificar o sustituir las reglas de movimiento o de combate no requiere alterar Board ni ninguna otra clase relacionada.

6.2.3. Fachada

El patrón Fachada proporciona una interfaz unificada y simplificada a un conjunto de interfaces de un subsistema, reduciendo el acoplamiento entre los clientes y dicho subsistema.

Board actúa como fachada del subsistema de tablero. Tal como se aprecia en la Figura 6.6, detrás de su interfaz pública agrupa la generación de la cuadrícula, la gestión de **Tile** y **Piece**, la delegación a **MovementSystem** y **CombatSystem**, y el controlador de input **BoardInputController**. Los clientes externos, **GameScene** y **BotController**, solo interactúan con **Board** a través de métodos de alto nivel como **MovePiece()**, **ResolveCombat()**, **SpawnPiece()** o **GetAllPossibleActions()**, sin necesidad de conocer la composición interna del subsistema.

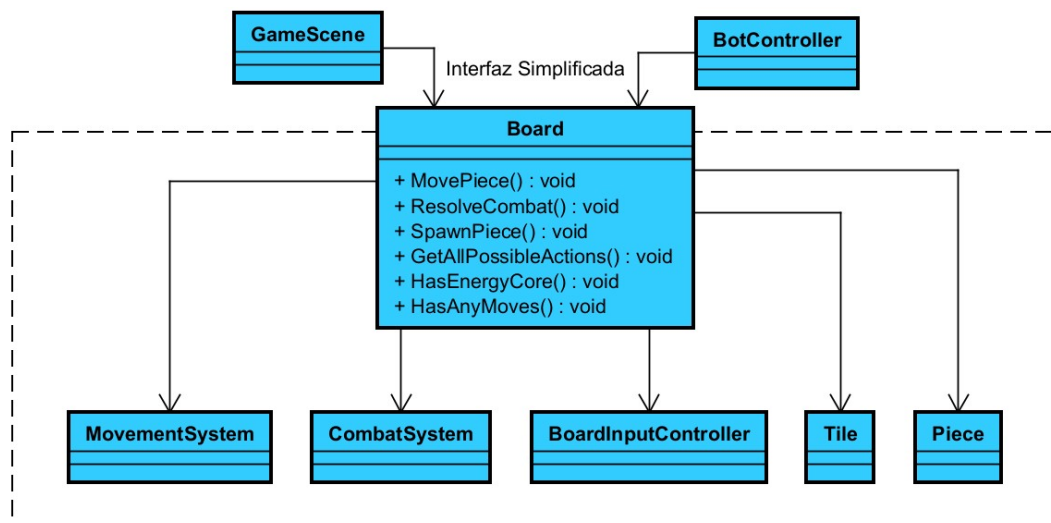


Figura 6.6: Patrón Fachada con **Board** como punto de acceso unificado al subsistema de tablero.

Esta decisión concentra en un único lugar todas las operaciones sobre el estado del tablero y reduce a dos el número de clases que los clientes externos necesitan conocer.

6.2.4. Template Method

El patrón Template Method define el esqueleto de un algoritmo en una operación, dejando que subclases o extensiones concreten algunos de sus pasos sin cambiar la estructura general.

En **SceneManager**, la clase interna **_Bridge** aplica este patrón en los métodos **TransitionIn()** y **TransitionOut()**. Ambos fijan la secuencia invariante de cualquier transición de pantalla, bloquear la entrada del usuario, ejecutar el paso de animación correspondiente y, tras **TransitionOut()**, devolver el control al videojuego.

En la forma canónica del patrón los pasos variables se delegan a subclases mediante herencia; aquí, al tratarse de una clase sellada (**sealed**), la variación se consigue mediante composición con el enumerado **Transition**. El efecto es equivalente, añadir un nuevo tipo de transición solo requiere incorporar un **case** adicional sin modificar el esqueleto del algoritmo ni las clases relacionadas.

6.3. Diseño de interfaz de usuario

Para garantizar una experiencia de usuario fluida, la interfaz de NeuroForge se ha estructurado como un sistema de pantallas y ventanas emergentes interconectadas. El flujo de navegación global y las transiciones permitidas entre los diferentes estados de la interfaz se detallan en el diagrama de la Figura 6.7.

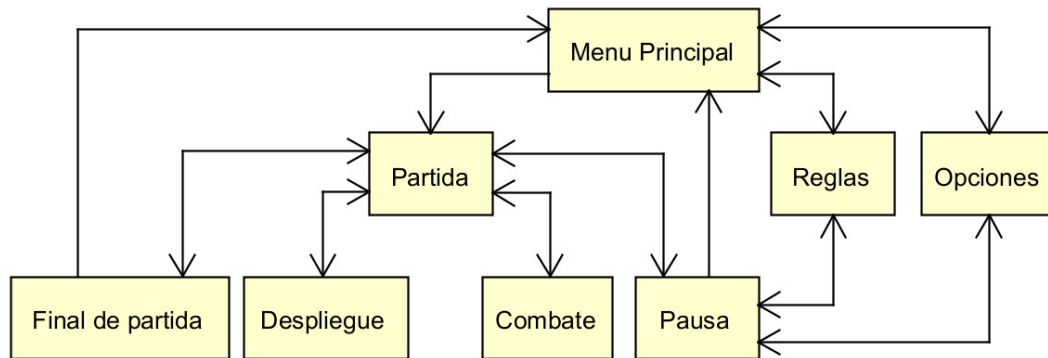


Figura 6.7: Diagrama de navegación de las interfaces de NeuroForge.

A continuación, se describen en detalle cada una de las secciones principales que componen este flujo.

Menú principal

La pantalla de inicio presenta el título del videojuego y, bajo él, tres botones dispuestos en vertical: *Play*, para iniciar una partida; *Rules*, para consultar las reglas; y *Exit*, para cerrar la aplicación. La disposición lineal evita confusiones y permite al jugador orientarse de inmediato, como se muestra en la Figura 6.8.



Figura 6.8: Menú principal de NeuroForge.

Menú de reglas

Accesible tanto desde el menú principal como durante la partida, el menú de reglas emerge como una ventana emergente sobre la pantalla activa. En la esquina superior izquierda se sitúa el botón *Go Back* para cerrarlo.

El contenido se organiza en cinco pestañas seleccionables en la parte superior de la ventana: *Objective*, con el objetivo y las condiciones de victoria; *Board*, con la descripción de los tipos de casillas; *Movement*, con las reglas de desplazamiento; *Combat*, con la mecánica de enfrentamiento; y *Pieces*, que actúa como guía de referencia de todas las piezas del videojuego, ilustrada en la Figura 6.9. Mientras que todas las vistas consisten en textos e imágenes, esta última pestaña es interactuable, incluyendo un botón por cada tipo de pieza; al seleccionar uno, se muestra su rango, si puede moverse y cualquier habilidad especial que posea.



Figura 6.9: Menú de reglas con la pestaña de piezas activa.

Fase de despliegue

Antes de poder iniciar una partida, el jugador ve el tablero vacío junto a un panel lateral izquierdo de despliegue. En la Figura 6.10 se muestra un caso de la interfaz de despliegue con los componentes de la interfaz mas importantes marcados y explicados brevemente.



Figura 6.10: Fase de despliegue con algunas piezas ya colocadas en la zona del jugador.

1. **Panel de despliegue:** En la parte superior se indican las piezas totales que quedan por colocar.
2. **Tablero:** Tablero donde desplegar las piezas que se seleccionan.
3. **Pieza pendiente por colocar:** Botón con un tipo de pieza con contador individual que indica la cantidad de piezas restantes por colocar de ese tipo.
4. **Pieza seleccionada:** Pieza seleccionada que se colocará en la casilla de despliegue del jugador a la que se haga click.
5. **Pieza agotada:** Pieza de la cual se han colocado todas las unidades de ese tipo.
6. **Botón de despliegue aleatorio:** Botón que coloca una distribución aleatoria de entre todas las casillas de despliegue.
7. **Botón de inicio de partida:** Botón que inicia una partida cuando ya están todas las piezas colocadas. Mientras queden piezas por colocar el botón estará desactivado.
8. **Casilla de despliegue del bot:** Casilla en la que el bot colocará sus piezas al empezar la partida.

9. **Pieza desplegada:** Pieza que se ha desplegado en el tablero. Se podrá seleccionar para eliminarla del tablero, seleccionando inmediatamente su tipo en el panel de despliegue para volver a colocarla inmediatamente.
10. **Casilla de despliegue del jugador:** Casilla en el que el jugador puede colocar sus piezas.

6.3.1. Partida en curso

Al comenzar la partida una vez desplegadas las piezas, la vista se actualiza a la que se ve en la Figura 6.11 donde se muestra una partida al cabo de unos pocos turnos donde también se comentan los componentes importantes de manera breve.



Figura 6.11: Pantalla de partida con una pieza seleccionada y sus acciones posibles resaltadas.

1. **Panel de piezas:** En este panel se recogen todas las piezas restantes que quedan en el tablero, ayudando al jugador a llevar la cuenta de que piezas le faltan al oponente y un resumen de las suyas.
2. **Casilla marcada para combate:** Cuando se selecciona una pieza que se puede mover a una casilla ocupada por una pieza rival, esta se marca de color rojo indicando que al moverse a esa posición se iniciará un combate.
3. **Casilla marcada para movimiento:** Cuando se selecciona una pieza que se puede mover a una casilla libre, esta se marca de color verde.
4. **Pieza seleccionada:** Pieza que el jugador selecciona, marcando las casillas a las que esta puede moverse. Al hacer click a una casilla marcada la pieza se moverá a dicha casilla. En concreto la pieza seleccionada de la imagen puede moverse a las casillas que se quiera en línea horizontal y vertical hasta que encuentra un obstáculo.

5. **Casilla transitable:** Casilla del tablero por la que se pueden mover las piezas, incluyendo las de despliegue de ambos bandos.
6. **Casilla intransitable:** Casilla del tablero por la que no se puede mover ninguna pieza.
7. **Marcador de turno:** Panel que indica de quien es el turno actual, para el jugador es un símbolo de una persona, y una calavera para el jugador, Similar a las que hay en las casillas de sus despliegues.
8. **Pieza del jugador revelada para el bot:** Pieza del jugador rodeada por un borde naranja similar al color de las piezas del bot. Significa que la identidad de esta pieza es conocida por el bot, ya sea por que ha sobrevivido a un combate o porque haya hecho un movimiento que compromete su identidad.

6.3.2. Interfaz de combate

Cuando dos piezas se enfrentan, una ventana emergente cubre la pantalla como la de la Figura 6.12. La ventana muestra dos paneles enfrentados, el atacante a la izquierda, indicado por un icono de mira roja, y el defensor a la derecha, identificado con un icono de escudo azul. Cada panel contiene el sprite de la pieza correspondiente y su nombre en la parte inferior.

La secuencia acompaña al desarrollo del combate. Al emerger la ventana, si alguna de las piezas era desconocida para el rival, su sprite parpadea y se revela. A continuación, la pieza o piezas perdedoras parpadean y se oscurecen. Finalmente aparece en la parte inferior un mensaje con el resultado, en el que se indica quién ha ganado y el motivo. El jugador cierra la ventana haciendo clic cuando lo considera oportuno.

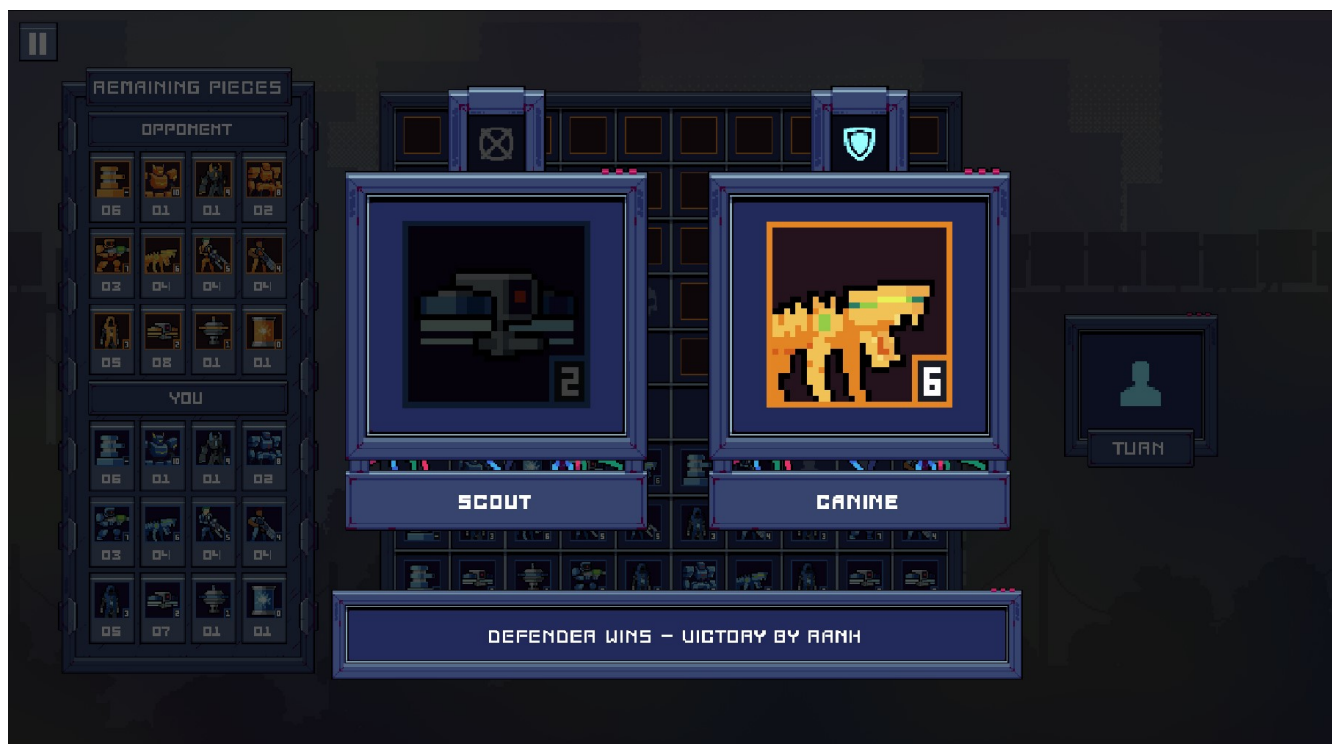


Figura 6.12: Interfaz de combate tras la resolución: pieza perdedora oscurecida y mensaje de resultado visible.

6.3.3. Fin de partida

Al concluir la partida, una ventana emergente informa del resultado, como se muestra en la Figura 6.13. El mensaje principal aparece en un color que codifica el desenlace: verde para la victoria, rojo para la derrota y morado para el empate. Bajo el mensaje se indica el motivo concreto, ya sea la destrucción del núcleo energético o el agotamiento de movimientos. Dos botones en la parte inferior permiten al jugador reiniciar la partida directamente o regresar al menú principal.



Figura 6.13: Pantalla de fin de partida con mensaje de victoria.

6.3.4. Menú de pausa

Durante el turno del jugador, este puede pausar la partida en cualquier momento, lo que provoca la aparición de una ventana emergente sobre el tablero. El menú presenta tres opciones dispuestas en vertical: *Resume*, que cierra el menú y devuelve el control al jugador; *Rules*, que abre el menú de reglas descrito anteriormente sin abandonar la partida; *Options*, que abre el menú de opciones; y *Return to Menu*, que abandona la partida en curso y regresa al menú principal.



Figura 6.14: Menú que emerge al pausar durante una partida.

6.3.5. Menú de opciones

Accesible tanto desde el menú principal como desde el menú de pausa, el menú de opciones emerge como una ventana sobre la pantalla activa. En la esquina superior izquierda se sitúa el botón *Go Back* para cerrarlo y retornar al contexto anterior.

El contenido se organiza en dos bloques. El primero tiene 2 desplegables, uno permite cambiar el idioma del juego, actualizando todos los textos del juego al instante, y otro controla el modo de visualización que permite alternar entre modo ventana y pantalla completa. El segundo bloque agrupa los controles de audio: cuatro sliders independientes que regulan el volumen de los canales *Master*, *Music*, *SFX* y *UI*, cada uno asociado a su propio bus de audio en el motor, de modo que pueden ajustarse de forma completamente independiente.



Figura 6.15: Menú de opciones donde configurar audio y resolución.

6.4. Diagrama de despliegue

El diagrama de despliegue de la Figura 6.16 describe los componentes físicos que conforman el sistema en tiempo de ejecución y cómo se distribuyen sobre el hardware del usuario.

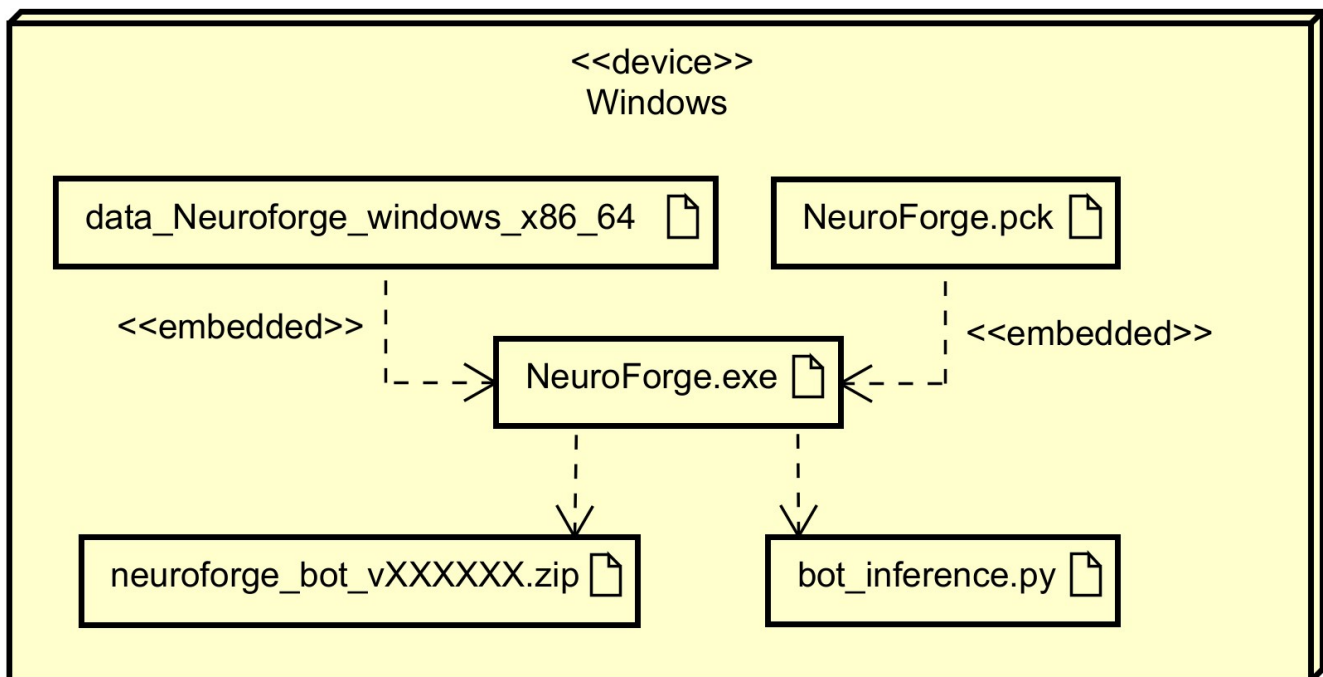


Figura 6.16: Diagrama de despliegue de NeuroForge.

NeuroForge se despliega como una aplicación de escritorio autocontenida que se ejecuta íntegramente en la máquina del jugador, sin dependencia de servidores externos ni conexión a red. El paquete de distribución final se compone de tres elementos principales distribuidos localmente.

El ejecutable principal (*NeuroForge.exe*) integra en un único archivo el motor de Godot compilado, la lógica del videojuego escrita en C# y todos los recursos del proyecto: escenas, sprites, efectos de sonido, música y demás assets. Esto es posible gracias a la opción de exportación de Godot que embebe el archivo *.pck* de recursos directamente dentro del binario, lo que simplifica al máximo la distribución y permite ejecutar el videojuego sin ningún tipo de instalación previa.

A fin de desacoplar el entorno de ejecución de la inteligencia artificial, el sistema incorpora el script intermedio *bot_inference.py*. Este componente actúa como un subsistema de inferencia ligero en Python que es invocado de manera transparente en segundo plano. Sus responsabilidades consisten en canalizar las comunicaciones de E/S estándar con el motor, reconstruir matricialmente el estado del juego transmitido y procesar vectorialmente las máscaras de acciones legales.

El último archivo del paquete es el modelo entrenado de la IA, distribuido en formato *.zip*. Este archivo no se integra en el binario principal, sino que es localizado, abierto y parseado dinámicamente por el script de inferencia en tiempo de ejecución al iniciar una partida contra el bot inteligente. Mantener tanto el script de ejecución como el modelo separados del ejecutable principal permite actualizar la lógica de predicción o sustituir el archivo de la red neuronal de forma independiente sin necesidad de modificar ni redistribuir el resto del binario del videojuego.

Implementación

En este capítulo se detallan los aspectos técnicos de la implementación de NeuroForge. Se describe la organización del código fuente en el motor Godot y la gestión de las licencias de los recursos de audio y gráficos empleados. Asimismo, se exponen los detalles de implementación más relevantes del proyecto, haciendo especial hincapié en la arquitectura técnica del bot inteligente y su integración con el modelo de IA.

7.1. Organización del código

Para garantizar la escalabilidad del proyecto y facilitar el mantenimiento del código, se ha seguido una estructura de directorios basada en la separación de responsabilidades. Dado que Godot utiliza un sistema de escenas y nodos, la organización se ha diseñado para diferenciar claramente entre los elementos visuales, la lógica del videojuego pura y la infraestructura de datos.

A continuación, se detallan las carpetas principales que componen el proyecto y su propósito dentro de la arquitectura global:

- **Assets/**: Esta carpeta centraliza todos los recursos gráficos y visuales del videojuego. Contiene las texturas (sprites), fuentes tipográficas, temas de interfaz (Themes) y recursos específicos de Godot como los **TileMaps**. Al agrupar estos elementos, se consigue que cualquier cambio estético o de diseño visual sea fácil de localizar sin interferir con la lógica programática.
- **BotEnvironment/**: Contiene todo el entorno necesario para el entrenamiento y la ejecución del agente de Inteligencia Artificial. Aquí se incluyen los scripts y configuraciones que permiten al *bot* interactuar con el videojuego, separando el comportamiento del agente del resto de sistemas para permitir pruebas aisladas de la IA.
- **Data & Infrastructure/**: En este directorio se ubican la definición de estructuras de datos y servicios de infraestructura que no pertenecen a la lógica del videojuego, pero que son

fundamentales para el funcionamiento del sistema, como el manejo del audio y la navegación entre escenas.

- **UI/**: Siguiendo el patrón de diseño orientado a escenas de Godot, esta carpeta contiene tanto los scripts de control de la interfaz de usuario como las escenas (.tscn) correspondientes a los menús, paneles y elementos interactivos (HUD). Esto permite que el diseño de la interfaz esté estrechamente ligado a su lógica de control inmediata, pero separado de la lógica de combate o movimiento.
- **GameLogic/**: Es el núcleo del videojuego y presenta una subdivisión estratégica para cumplir con el diseño arquitectónico:
 - **Lógica Pura (C# scripts)**: Contiene clases puras de C# que no heredan de la clase `Node` de Godot. Aquí residen las reglas del videojuego y algoritmos que podrían ser testeados de forma unitaria fuera del motor.
 - **Nodos de Godot**: Subdividida en carpetas como `Pieces`, `Tiles` y `Board`. En cada una de ellas se encuentran las escenas de Godot y los scripts que heredan de la API del motor para gestionar la representación espacial y el comportamiento de los objetos en pantalla.

Esta estructura modular permite que el proyecto sea robusto, ya que cada componente tiene una responsabilidad bien definida, permitiendo que el motor de videojuegos funcione de manera fluida y organizada.

7.2. Licencias requeridas

Para el desarrollo de *NeuroForge*, se han utilizado diversos recursos externos de audio y gráficos, así como el propio motor de desarrollo, bajo licencias que permiten su uso en proyectos académicos y comerciales. A continuación, se detallan las licencias de software y recursos empleadas.

7.2.1. Motor de Videojuegos: Godot Engine

El proyecto ha sido desarrollado utilizando *Godot Engine*. Este motor se distribuye bajo la *Licencia MIT*, una de las licencias de software libre más permisivas y utilizadas en la industria.

- **Permisos**: Permite el uso, copia, modificación, fusión, publicación, distribución, sublicencia y venta del software sin restricciones.
- **Condiciones**: El único requisito es incluir el aviso de copyright y el texto de la licencia original de Godot en cualquier copia sustancial del software o en la documentación del mismo.

7.2.2. Recursos de Audio

Los efectos de sonido y la música han sido obtenidos respetando las licencias especificadas por los autores en plataformas de contenido libre:

- **Música durante la partida:** *Granular 5.wav* por PodcastAC. Disponible en Freesound¹. Licencia: *Attribution 4.0 (CC BY 4.0)*.
- **Música del menú:** *Ambient, Piano, Atmosphere - Reborn* por Andrewkn. Disponible en Freesound². Licencia: *Attribution 4.0 (CC BY 4.0)*.
- **Efectos de sonido de las piezas:** *SCIMisc_Energy UI Low 01* por KVV_Audio. Disponible en Freesound³. Licencia: *Attribution 4.0 (CC BY 4.0)*.
- **Interfaz de usuario:** *UIClick_Button press* por newlocknew. Disponible en Freesound⁴. Licencia: *Creative Commons 0 (CC0)*.
- **Interacción con el tablero:** Recursos obtenidos de *Pixabay* bajo su *Content License*, que permite el uso gratuito y modificación sin atribución obligatoria.

7.2.3. Recursos Gráficos

El apartado visual integra *assets* adaptados y modificados bajo los siguientes términos:

- **CraftPix Assets:** Los recursos provenientes de *Craftpix.net*⁵ se utilizan bajo su licencia comercial, que permite su uso ilimitado en videojuego siempre que no se revendan los archivos fuente originales. Se han utilizado para la mayoría de los recursos gráficos dentro del videojuego.
- **Sprites de piezas (Mechs):** *Mech Assets* por *Marinesosa*. Disponible en *itch.io*⁶. Su licencia permite la modificación y el uso comercial/no comercial, prohibiendo la reventa del paquete original. Se han utilizado para el diseño de algunas piezas

7.2.4. Modificación de recursos

Es importante señalar que gran parte de los recursos gráficos y de audio han sido modificados para ajustarse a las necesidades de *NeuroForge*. Estas modificaciones (ajustes de color, reescalado, edición de audio) se han realizado mediante *Aseprite* y *Audacity*, cumpliendo con los permisos de obras derivadas de las licencias mencionadas.

¹<https://freesound.org/s/663786/>

²<https://freesound.org/s/720772/>

³<https://freesound.org/s/811930/>

⁴<https://freesound.org/s/665222/>

⁵<https://craftpix.net/sets/cyberpunk-platformer-asset-pixel-art/>

⁶<https://marinesosa.itch.io/mech-assets>

7.3. Detalles de implementación relevantes

Entre todos los sistemas implementados, la arquitectura del bot constituye el componente técnicamente más exigente del proyecto, tanto por la complejidad del entorno de entrenamiento como por su integración con el motor de videojuegos.

7.3.1. Arquitectura del bot

El bot presenta dos modos de funcionamiento. El comportamiento por defecto, utilizado como *fallback* cuando el modelo entrenado no está disponible o falla al cargar, consiste en la selección aleatoria de un movimiento válido entre todos los posibles en cada turno. Este modo garantiza que la partida siempre pueda completarse y sirvió además como oponente durante todo el desarrollo previo a la implementación del modelo de inteligencia artificial.

El comportamiento avanzado se apoya en tres decisiones principales: el algoritmo de aprendizaje, el enmascaramiento de acciones y la estrategia de entrenamiento mediante *self-play*. El algoritmo empleado es *Proximal Policy Optimization* (PPO), un método de aprendizaje por refuerzo de política en línea (*on-policy*) que actualiza los parámetros del agente a partir de su propia experiencia. En cada iteración, el agente interactúa con el entorno durante un número fijo de pasos, acumula las transiciones observadas y actualiza su política intentando maximizar la recompensa esperada. Para evitar actualizaciones desestabilizadoras, PPO introduce una restricción de recorte (*clipping*) que limita cuánto puede cambiar la política en cada paso de gradiente, lo que lo convierte en uno de los algoritmos más estables y ampliamente utilizados en tareas de control secuencial.

En dominios con espacios de acción grandes y mayoría de acciones inválidas, aplicar PPO directamente es ineficiente, ya que el agente desperdiciaría gran parte de su exploración en movimientos ilegales. Para resolver esto se utiliza *MaskablePPO*, una extensión de PPO de la biblioteca *SB3-Contrib* que incorpora enmascaramiento de acciones. En cada paso, el entorno proporciona una máscara binaria que indica qué acciones son legales en el estado actual; la distribución de probabilidad de la política se recalcula zerificando las acciones enmascaradas antes de aplicar *softmax*, de modo que el agente únicamente explora y explota movimientos válidos. En *NeuroForge*, donde el espacio de acciones codifica todos los pares origen-destino posibles sobre el tablero, esta restricción resulta indispensable para que el entrenamiento converja.

Respecto a la estrategia de entrenamiento, el agente se entrena mediante *self-play*: el oponente en cada sesión es una versión anterior del propio modelo, seleccionada aleatoriamente entre las más recientes. Esta estrategia, habitual en juegos de información imperfecta, evita que el agente se especialice en explotar los fallos de un rival fijo y fomenta el desarrollo de estrategias más generales y robustas. Al inicio del entrenamiento, cuando no existen versiones previas, el oponente es el bot aleatorio, que proporciona una señal de aprendizaje suficiente para arrancar la convergencia.

Los hiperparámetros del modelo se determinaron mediante una búsqueda bayesiana con *Optuna*, una biblioteca de optimización automática de hiperparámetros. A diferencia de una búsqueda en rejilla exhaustiva, la optimización bayesiana construye un modelo probabilístico del espacio de hiperparámetros y dirige las pruebas hacia las regiones más prometedoras, reduciendo el número de evaluaciones necesarias. Tras 20 ensayos evaluando cada combinación con 200.000 pasos de entrenamiento, los hiperparámetros seleccionados fueron los recogidos en la Tabla 7.1.

Hiperparámetro	Valor
<code>learning_rate</code>	$9,87 \times 10^{-5}$
<code>n_steps</code>	512
<code>batch_size</code>	128
<code>n_epochs</code>	5
<code>ent_coef</code>	0,032
<code>gamma</code>	0,9585
<code>clip_range</code>	0,197

Tabla 7.1: Hiperparámetros del modelo MaskablePPO tras la búsqueda con Optuna

Finalmente, la integración con el motor se realiza mediante el componente `BotController` en C#. Al comenzar la partida, este controlador inicia un subproceso en segundo plano que ejecuta el script `bot_inference.py`, pasándole la ruta del modelo `.zip` como argumento. A partir de este momento, ambos componentes se comunican continuamente intercambiando mensajes en formato JSON a través de los canales de entrada y salida estándar. Al arrancar, el script de Python carga el modelo con la función `MaskablePPO.load()` y escribe en su salida la palabra clave `READY` para avisar al juego de que ya está operativo. Durante la partida, cada vez que es el turno del bot, el motor le envía un JSON con el estado del tablero y la lista de movimientos permitidos (`valid_moves`). El script procesa estos datos con la biblioteca *NumPy*: transforma el tablero en una matriz de tres canales ($3 \times 10 \times 10$) y convierte los movimientos válidos en un vector de máscara booleana de 10,000 posiciones. Con esta información preparada, el modelo realiza la predicción de forma determinista. Por último, el script traduce la acción numérica resultante de vuelta a coordenadas de juego (*from/to*) y las envía por la salida estándar hacia el controlador de C#, el cual recibe la jugada y la ejecuta en el videojuego.

Si el subproceso Python sufre una excepción catastrófica, no responde en el tiempo estipulado o el archivo del modelo está corrupto, el controlador de C# intercepta el error y activa el modo aleatorio de *fallback*, garantizando la continuidad de la ejecución del juego.

Pruebas

En este capítulo se describe la estrategia de verificación adoptada para validar que el sistema desarrollado cumple con los requisitos especificados y con los flujos de interacción definidos en el análisis. El objetivo es demostrar que el videojuego se comporta de forma correcta y estable ante las distintas situaciones que pueden producirse durante su uso. La batería de pruebas se organiza en ocho bloques temáticos que cubren los requisitos funcionales y no funcionales, e incluye adicionalmente una sección dedicada a la usabilidad percibida por usuarios reales durante sesiones de juego supervisadas.

8.1. Enfoque de pruebas

En un entorno de desarrollo profesional, la verificación de un sistema software de esta naturaleza se abordaría mediante una estrategia de pruebas automatizadas estructurada en varios niveles. Las pruebas unitarias verificarían de forma aislada cada componente lógico del sistema, como el motor de reglas de movimiento o el sistema de resolución de combates, comprobando que producen el resultado esperado ante un conjunto exhaustivo de entradas. Las pruebas de integración validarían la interacción correcta entre componentes, asegurando por ejemplo que el controlador de input delega adecuadamente en el tablero y que este actualiza el estado de la partida de forma consistente. Finalmente, las pruebas de sistema o de extremo a extremo ejercitarían flujos completos de usuario, desde el despliegue inicial hasta la condición de victoria, simulando el comportamiento real del jugador. Herramientas como *NUnit* o *xUnit* en el ecosistema .NET, combinadas con un pipeline de integración continua, permitirían ejecutar esta batería de forma automática ante cada cambio en el código, detectando regresiones de manera inmediata.

Dado el carácter académico e individual del proyecto y las restricciones temporales inherentes a un TFG, la automatización completa de las pruebas queda fuera del alcance definido. No obstante, la arquitectura adoptada, con la lógica del videojuego separada en clases C# puras e independientes del motor, facilita que dichas pruebas automatizadas puedan incorporarse en el futuro sin necesidad de modificar el código de producción. En su lugar, se ha llevado a cabo una batería de pruebas de aceptación organizadas por bloques temáticos, cubriendo todos los requisitos funcionales y no funcionales identificados en el análisis. Cada prueba define de forma explícita su precondition, la acción ejecutada y el resultado esperado, lo que permite reproducirlas de manera sistemática y trazarlas directamente con los requisitos del sistema.

8.2. Batería de pruebas

En este apartado se recoge el conjunto de pruebas funcionales realizadas sobre el sistema para verificar que el comportamiento del videojuego se ajusta a los requisitos especificados y a los flujos descritos en los casos de uso. Las pruebas se organizan por bloques temáticos y se presentan en formato tabular, indicando para cada caso la precondition, los pasos ejecutados, el resultado esperado y si la prueba ha sido superada.

8.2.1. Bloque 1 — Despliegue inicial

ID	Descripción	Precondición	Resultado esperado	¿Pasa?
PR-01	Iniciar una nueva partida desde el menú principal	El sistema muestra el menú principal	El tablero se inicializa y se muestra la interfaz de despliegue	Sí
PR-02	Intentar comenzar la partida sin colocar todas las piezas	Pantalla de despliegue activa	El sistema impide el inicio y permanece en la fase de despliegue	Sí
PR-03	Colocar una pieza fuera de la zona de despliegue permitida	Pantalla de despliegue activa	El sistema rechaza la acción y no coloca la pieza	Sí
PR-04	Intentar colocar dos piezas en la misma casilla	Al menos una pieza ya colocada	El sistema quita la pieza original	Sí
PR-05	Colocar todas las piezas y confirmar inicio	Todas las piezas del jugador colocadas	El bot despliega sus piezas y comienza el turno del jugador	Sí

Tabla 8.1: Pruebas del bloque 1: Despliegue inicial

8.2.2. Bloque 2 — Movimiento

ID	Descripción	Precondición	Resultado esperado	¿Pasa?
PR-06	Seleccionar una pieza propia con movimientos disponibles	Turno del jugador activo	Las casillas de destino válidas quedan resaltadas	Sí
PR-07	Seleccionar una pieza propia sin movimientos disponibles	Turno del jugador activo	No se resalta ninguna casilla ni se registra selección	Sí
PR-08	Hacer clic fuera de una casilla válida tras seleccionar una pieza	Pieza seleccionada con casillas resaltadas	Se cancela la selección y se eliminan los resaltados	Sí
PR-09	Mover una pieza a una casilla vacía válida	Pieza seleccionada y casilla resaltada	La pieza se desplaza a la casilla y el turno pasa al bot	Sí
PR-10	Intentar mover una pieza a una casilla intransitable	Turno del jugador activo	El sistema impide el movimiento; la casilla no aparece resaltada	Sí
PR-11	Intentar mover una pieza a una casilla ocupada por otra pieza propia	Turno del jugador activo	El sistema no permite el movimiento; la casilla no está resaltada	Sí
PR-12	Mover el Explorador más de una casilla en línea recta	Turno del jugador, Explorador disponible	La pieza se desplaza y su identidad queda revelada permanentemente	Sí
PR-13	Mover el Explorador exactamente una casilla	Turno del jugador, Explorador disponible	La pieza se desplaza sin revelar su identidad	Sí
PR-14	Intentar realizar una acción durante el turno del bot	Turno del bot en ejecución	El sistema ignora el input del jugador	Sí

Tabla 8.2: Pruebas del bloque 2: Movimiento

8.2.3. Bloque 3 — Combate

ID	Descripción	Precondición	Resultado esperado	¿Pasa?
PR-15	Pieza de mayor rango ataca a pieza de menor rango	Ambas piezas en posiciones adyacentes válidas	La pieza atacante ocupa la casilla; la defensora es eliminada	Sí
PR-16	Pieza de menor rango ataca a pieza de mayor rango	Ambas piezas en posiciones adyacentes válidas	La pieza atacante es eliminada; la defensora permanece	Sí
PR-17	Dos piezas del mismo rango se enfrentan	Ambas piezas en posiciones adyacentes válidas	Empate: ambas piezas son eliminadas del tablero	Sí
PR-18	Saboteador ataca a una Torreta	Saboteador y Torreta en posiciones adyacentes	El Saboteador vence y ocupa la casilla de la Torreta	Sí
PR-19	Pieza distinta al Saboteador ataca a una Torreta	Pieza no-Saboteador y Torreta adyacentes	La pieza atacante es destruida; la Torreta permanece	Sí
PR-20	Phantom ataca a la Máquina de guerra	Phantom y Máquina de guerra en posiciones adyacentes	El Phantom vence independientemente del rango	Sí
PR-21	Pieza distinta al Phantom ataca a la Máquina de guerra	Pieza no-Phantom y Máquina de guerra adyacentes	El combate se resuelve por rango numérico según las reglas normales	Sí
PR-22	Combate entre dos piezas, al menos una oculta	Una o ambas piezas con identidad no revelada	Ambas identidades quedan reveladas y permanecen visibles	Sí
PR-23	Pieza ya revelada entra en un nuevo combate	Pieza previamente revelada en partida	La pieza sigue mostrando su identidad correctamente	Sí
PR-24	La interfaz de combate se muestra y cierra correctamente	Se produce un combate durante un turno	Se muestra la pantalla de resolución; al confirmar, se cierra y el juego continúa	Sí

Tabla 8.3: Pruebas del bloque 3: Combate

8.2.4. Bloque 4 — Condiciones de victoria

ID	Descripción	Precondición	Resultado esperado	¿Pasa?
PR-25	El jugador destruye el Núcleo de energía del bot	Partida en curso, Núcleo rival accesible	La partida termina inmediatamente y el jugador gana	Sí
PR-26	El bot destruye el Núcleo de energía del jugador	Partida en curso, Núcleo del jugador accesible	La partida termina inmediatamente y el jugador pierde	Sí
PR-27	Todas las piezas móviles del jugador son eliminadas	Partida en curso	El sistema detecta que el jugador no puede mover y declara su derrota	Sí
PR-28	Todas las piezas móviles del bot son eliminadas	Partida en curso	El sistema detecta que el bot no puede mover y declara la victoria del jugador	Sí
PR-29	El jugador y el bot se quedan sin piezas móviles en el mismo turno	Partida en curso	La pantalla de fin de partida aparece indicando un empate	Sí
PR-30	El jugador selecciona jugar otra partida desde la pantalla de fin	Pantalla de fin de partida visible	El sistema reinicia completamente el estado y vuelve a la fase de despliegue	Sí
PR-31	El jugador selecciona volver al menú principal desde la pantalla de fin	Pantalla de fin de partida visible	El sistema navega al menú principal	Sí

Tabla 8.4: Pruebas del bloque 4: Condiciones de victoria

8.2.5. Bloque 5 — Interfaz y feedback visual/sonoro

ID	Descripción	Precondición	Resultado esperado	¿Pasa?
PR-32	Sonido de movimiento al desplazar una pieza	Turno del jugador, pieza movida a casilla vacía	Se reproduce el efecto de sonido asociado al movimiento	Sí
PR-33	Sonido de combate al producirse un enfrentamiento	Combate iniciado durante cualquier turno	Se reproduce el efecto de sonido asociado al combate	Sí
PR-34	Sonido de navegación por menús	Usuario navega entre menús	Se reproducen los efectos sonoros asociados a los botones	Sí
PR-35	Toda la interacción es posible exclusivamente con el ratón	El videojuego está en cualquier estado	Ninguna acción requiere el uso del teclado	Sí

Tabla 8.5: Pruebas del bloque 5: Interfaz y feedback

8.2.6. Bloque 6 — Menús y navegación

ID	Descripción	Precondición	Resultado esperado	¿Pasa?
PR-36	Pausar la partida en mitad de un turno del jugador	Turno del jugador activo	El menú de pausa se abre y la partida queda pausado	Sí
PR-37	Reanudar la partida desde el menú de pausa	Menú de pausa abierto	La partida continúa desde el mismo estado en que fue pausada	Sí
PR-38	Abandonar la partida desde el menú de pausa	Menú de pausa abierto	El sistema navega al menú principal sin errores	Sí
PR-39	Consultar las reglas desde el menú principal	Sistema en el menú principal	Se muestra el menú de reglas completo con toda la información	Sí
PR-40	Consultar las reglas desde el menú de pausa	Menú de pausa abierto	Se muestra el menú de reglas; al cerrarlo, el sistema regresa al menú de pausa	Sí
PR-41	Ajustar el volumen de las pistas de sonido desde el menú de opciones	Menú de opciones accesible	El cambio se aplica inmediatamente y se mantiene durante la sesión	Sí
PR-42	Acceder al menú de opciones desde el menú principal	Sistema en el menú principal	El menú de opciones se abre correctamente	Sí
PR-43	Acceder al menú de opciones desde el menú de pausa	Menú de pausa abierto	El menú de opciones se abre; al cerrarlo regresa al menú de pausa	Sí

Tabla 8.6: Pruebas del bloque 6: Menús y navegación

8.2.7. Bloque 7 — Comportamiento del bot

ID	Descripción	Precondición	Resultado esperado	¿Pasa?
PR-44	El bot ejecuta su turno automáticamente tras el turno del jugador	El jugador ha completado su acción	El bot realiza un movimiento válido sin intervención del jugador	Sí
PR-45	El bot nunca se mueve a una casilla intransitable	Turno del bot activo	Ningún movimiento del bot aterriza en una casilla de tipo no-pasable	Sí
PR-46	El bot nunca ataca a sus propias piezas	Turno del bot activo	Todos los ataques del bot se dirigen a piezas del jugador	Sí
PR-47	El bot no actúa durante el turno del jugador	Turno del jugador activo	El bot permanece inactivo hasta que el jugador completa su acción	Sí
PR-48	El bot respeta las reglas especiales de combate	Turno del bot, situación con Torreta o Phantom	Los resultados de combate del bot siguen las mismas reglas especiales que las del jugador	Sí

Tabla 8.7: Pruebas del bloque 7: Comportamiento del bot

8.2.8. Bloque 8 — Requisitos no funcionales

ID	Descripción	Precondición	Resultado esperado	¿Pasa?
PR-49	El videojuego arranca correctamente en Windows 10	Ejecutable instalado en Windows 10	El videojuego se inicia sin errores y llega al menú principal	Sí
PR-50	El videojuego arranca correctamente en Windows 11	Ejecutable instalado en Windows 11	El videojuego se inicia sin errores y llega al menú principal	Sí
PR-51	El videojuego funciona en un equipo con las especificaciones mínimas	Hardware mínimo: CPU doble núcleo 2,0 GHz, 8 GB RAM, GPU OpenGL 3.3	El videojuego se ejecuta sin cuelgues, artefactos visuales ni caídas de rendimiento significativas	Sí
PR-52	Las acciones del jugador reciben respuesta en menos de 2 segundos	Videojuego en ejecución en condiciones normales	Ninguna acción provoca una espera superior a 2 segundos antes de que el sistema reaccione	Sí
PR-53	Un jugador sin experiencia identifica qué piezas puede mover sin consultar las reglas	Primera partida del jugador, sin leer el menú de reglas	El jugador es capaz de seleccionar y mover una pieza correctamente gracias al resaltado de casillas	Sí

Tabla 8.8: Pruebas del bloque 8: Requisitos no funcionales

8.3. Pruebas de usabilidad

Las pruebas de usabilidad tienen como objetivo evaluar en qué medida el videojuego resulta comprensible e intuitivo para usuarios que se enfrentan a él por primera vez, sin haber recibido instrucciones previas más allá de las que ofrece el propio sistema. A diferencia de las pruebas funcionales, que verifican la corrección del comportamiento del sistema, las pruebas de usabilidad miden la experiencia subjetiva del usuario y su capacidad para alcanzar los objetivos de juego de forma autónoma.

Se realizaron sesiones de prueba con varios usuarios de perfil distinto: 1 usuario sin experiencia en videojuegos de estrategia y 2 usuario con experiencia media o habitual en juegos de estrategia por turnos. En cada sesión se pidió al participante que realizara las siguientes tareas sin ayuda externa: iniciar una partida, desplegar todas sus piezas, completar al menos tres turnos de movimiento y combate, y navegar por el menú de reglas y el menú de pausa. Durante las sesiones se registraron los errores cometidos, los momentos de bloqueo y los comentarios verbales espontáneos de los participantes.

ID	Descripción	Método	Resultado esperado	¿Pasa?
PU-01	El usuario comprende qué piezas puede mover sin leer las reglas	Observación directa en primera partida	El usuario selecciona una pieza propia y observa el resaltado de casillas disponibles sin bloqueos prolongados	Sí
PU-02	El usuario identifica cuándo es su turno y cuándo es el turno del bot	Observación directa	Ningún participante intenta realizar acciones durante el turno del bot	Sí
PU-03	El usuario comprende el resultado de un combate sin instrucciones	Observación y entrevista posterior	Todos los participantes describen correctamente el resultado tras presenciar la pantalla de resolución de combate	Sí
PU-04	El usuario es capaz de pausar la partida y retomar el juego	Observación directa	El participante localiza el botón de pausa y retoma la partida sin dificultad	Sí
PU-05	El usuario valora positivamente la claridad de la interfaz	Cuestionario post-sesión (escala 1–5)	Puntuación media igual o superior a 4 sobre 5 en apartado visual	Sí

Tabla 8.9: Pruebas de usabilidad

Los resultados de las sesiones fueron positivos en términos generales. El mecanismo de resaltado de casillas disponibles resultó ser el elemento más valorado, ya que permite al usuario descubrir las reglas de movimiento de forma inductiva sin necesidad de consultar la guía. El único punto de fricción recurrente fue la fase de despliegue inicial, donde el usuario sin experiencia tardó

más tiempo en comprender cómo debía colocar todas las piezas antes de poder iniciar la partida. El botón de despliegue aleatorio indirectamente acabó ayudando en esa tarea, ya que los jugadores podrían utilizarlo por curiosidad y es entonces cuando verían que pueden quitar piezas, seleccionadas a placer en la interfaz de despliegue y demás.

8.3.1. Cuestionario de evaluación

Con el objetivo de obtener una valoración más amplia y estructurada por parte de usuarios externos, se diseñó un cuestionario de evaluación distribuido a través de la plataforma Google Forms. Este instrumento permite recopilar datos cuantitativos y cualitativos sobre la percepción del software de manera estandarizada mediante la aplicación de preguntas binarias y escalas de valoración de cinco niveles ordenadas de mayor a menor grado de conformidad.

El cuestionario se organiza en seis bloques temáticos clave para evaluar la calidad del videojuego desde la perspectiva de la experiencia de usuario:

- **Sonido:** Evalúa el uso del audio integrado, el nivel de agrado de las pistas musicales y los efectos, así como su impacto directo en la experiencia de juego.
- **Apartado visual:** Analiza el atractivo de la interfaz, la idoneidad de la paleta de colores seleccionada y la coherencia estética de los gráficos con la temática de ciencia ficción.
- **Operabilidad:** Mide la facilidad de uso de los controles, la claridad en la exposición de las reglas y la legibilidad de la tipografía y los textos informativos en pantalla.
- **Entretenimiento:** Cuantifica el grado de diversión percibido y la predisposición del usuario a recomendar el videojuego a terceros.
- **Inmersión:** Explora la capacidad del juego para capturar la atención inicial, la retención del usuario al finalizar la sesión y la alteración de la percepción del tiempo durante la partida.
- **Calificación global:** Recoge una valoración numérica general del proyecto en una escala del 1 al 10 y un apartado de observaciones abiertas para propuestas de mejora.

Los resultados cuantitativos obtenidos se muestran distribuidos en la Tabla 8.10, mapeando de manera explícita la frecuencia de respuestas a lo largo de los diferentes niveles de la escala de evaluación para ofrecer una panorámica visual del rendimiento de cada indicador.

Evaluación	Distribución de respuestas				
	Nivel 5	Nivel 4	Nivel 3	Nivel 2	Nivel 1
<i>Escalas generales</i>	<i>Muy bueno / Total. de acuerdo</i>	<i>Bueno / De acuerdo</i>	<i>Regular / Indiferente</i>	<i>Malo / En desacuerdo</i>	<i>Muy malo / Tot. en desacuerdo</i>
Disfrute del sonido	-	5	-	-	-
Sonido mejora la experiencia	4	1	-	-	-
Atractivo visual	2	3	-	-	-
Paleta de colores	1	4	-	-	-
Gráficos apropiados	2	3	-	-	-
Fácil manejo de controles	4	1	-	-	-
Reglas del juego claras	1	4	-	-	-
Textos y fuentes legibles	2	-	2	1	-
Colores prácticos	4	1	-	-	-
Satisfacción	1	2	1	1	-
Retención final	1	3	1	-	-
Retención inicial	3	2	-	-	-
Entretenimiento	1	-	2	-	-
<i>Preguntas binarias</i>	Sí		No		
Audio activo	5		-		
Recomendación del videojuego	4		1		

Tabla 8.10: Matriz de frecuencias y distribución de respuestas del cuestionario de evaluación.

A partir de la distribución de frecuencias reflejada en la matriz y el análisis de las valoraciones cualitativas recogidas en el canal abierto, se extraen las siguientes conclusiones generales sobre el estado actual del desarrollo:

En primer lugar, los elementos visuales y auditivos que componen el apartado artístico han obtenido una recepción excelente por parte de los usuarios, logrando concentrar la totalidad de sus valoraciones en los niveles superiores de la escala. Los participantes coinciden de manera unánime en que la música ambiente no solo resulta agradable al oído, sino que cumple una función práctica esencial al fomentar la concentración y la inmersión necesarias para afrontar una partida de estrategia.

En segundo lugar, la usabilidad del sistema y la claridad en el diseño de las mecánicas demuestran poseer unas bases muy sólidas. Tanto los esquemas de control del software como el reglamento del videojuego han sido catalogados como altamente intuitivos y comprensibles desde la primera toma de contacto. En este mismo sentido, la codificación cromática del tablero se consolida como un gran acierto de diseño que agiliza la asimilación de la información de las partidas en tiempo real.

Por otro lado, el indicador relativo a la legibilidad de los textos y fuentes en pantalla es el único parámetro de la evaluación que muestra una dispersión de datos orientada hacia los espectros neutros y negativos. Este comportamiento estadístico evidencia que, aunque la tipografía actual permite jugar sin impedimentos críticos, existe un margen de mejora técnico evidente en cuanto a los tamaños, contrastes y familias tipográficas elegidas con el fin de prevenir síntomas de fatiga visual durante sesiones de juego continuadas.

Finalmente, los resultados certifican una notable capacidad de entretenimiento y fidelización, lo que se ve respaldado por un índice de recomendación pleno y una inmediata captura de la atención del jugador. No obstante, las observaciones cualitativas coinciden en señalar que el nivel de desafío de los escenarios es algo reducido. Las sugerencias de mejora recogidas determinan que el desarrollo futuro de una inteligencia artificial con patrones tácticos más complejos o la integración de una modalidad de juego multijugador en red serían las vías óptimas para incrementar la rejugabilidad y el compromiso a largo plazo.

8.4. Pruebas automatizadas con NUnit

Como se indicó en el enfoque de pruebas, la arquitectura del proyecto separa la lógica del videojuego en clases C# puras independientes del motor Godot, lo que las hace directamente instanciables desde un proyecto de pruebas externo. Para ilustrar cómo se incorporarían pruebas automatizadas en el futuro, se ha desarrollado un proyecto de pruebas con *NUnit* que verifica el comportamiento del sistema de resolución de combates, uno de los componentes con mayor número de casos especiales.

Para su puesta en marcha, basta con crear un proyecto de tipo **NUnit Test Project** en el directorio del repositorio mediante el comando:

```
dotnet new nunit -n NeuroForge.Tests
```

A continuación se referencia el ensamblado de lógica pura del proyecto principal y se añade el paquete **NUnit** como dependencia. Para desacoplar las pruebas del motor Godot, se define en el propio archivo de tests un tipo auxiliar **TestPiece** que implementa la interfaz **ICombatant**, la misma que acepta **CombatSystem.Resolve**.

De este modo los tests no dependen del ciclo de vida de los nodos de Godot y pueden ejecutarse de forma completamente aislada. Los casos de prueba siguen el patrón *Arrange-Act-Assert*: se construyen las piezas con el estado necesario, se invoca el método bajo prueba y se verifica el resultado mediante aserciones explícitas. El siguiente fragmento recoge los casos más representativos del sistema de combate:

```
using NUnit.Framework;

public record TestPiece(PieceType Type = default, int Rank = 0) : ICombatant;

[TestFixture]
public class CombatSystemTests
{
    [Test]
    public void HigherRank_Wins_AgainstLowerRank()
    {
        var result = CombatSystem.Resolve(
            new TestPiece(Rank: 5),
            new TestPiece(Rank: 3));
        Assert.That(result, Is.EqualTo(CombatResult.DEFENDER_DIES));
    }

    [Test]
    public void LowerRank_Loses_AgainstHigherRank()
    {
        var result = CombatSystem.Resolve(
            new TestPiece(Rank: 2),
            new TestPiece(Rank: 7));
        Assert.That(result, Is.EqualTo(CombatResult.ATTACKER_DIES));
    }
}
```



```

[Test]
public void SameRank_Results_InDraw()
{
    var result = CombatSystem.Resolve(
        new TestPiece(Rank: 4),
        new TestPiece(Rank: 4));
    Assert.That(result, Is.EqualTo(CombatResult.BOTH_DIE));
}

[Test]
public void Saboteur_Always_Beats_Turret()
{
    var result = CombatSystem.Resolve(
        new TestPiece(Type: PieceType.SABOTEUR),
        new TestPiece(Type: PieceType.TURRET));
    Assert.That(result, Is.EqualTo(CombatResult.DEFENDER_DIES));
}

[Test]
public void NonSaboteur_Loses_AgainstTurret()
{
    var result = CombatSystem.Resolve(
        new TestPiece(Rank: 9),
        new TestPiece(Type: PieceType.TURRET));
    Assert.That(result, Is.EqualTo(CombatResult.ATTACKER_DIES));
}

[Test]
public void Phantom_Always_Beats_WarMachine()
{
    var result = CombatSystem.Resolve(
        new TestPiece(Type: PieceType.PHANTOM),
        new TestPiece(Type: PieceType.WAR_MACHINE));
    Assert.That(result, Is.EqualTo(CombatResult.DEFENDER_DIES));
}
}

```

Listing 8.1: Pruebas unitarias del sistema de resolución de combates con NUnit

La ejecución de la batería completa se lanza desde la terminal con:

```
dotnet test
```

La Figura 8.1 muestra el resultado de ejecutar estos casos de prueba, donde todos superan la verificación sin errores.

```
 NUnit Adapter 4.6.0.0: Test execution complete
  Correctas HigherRank_Wins_AgainstLowerRank [6 ms]
  Correctas LowerRank_Loses_AgainstHigherRank [< 1 ms]
  Correctas NonSaboteur_Loses_AgainstTurret [< 1 ms]
  Correctas Phantom_Always_Beats_WarMachine [< 1 ms]
  Correctas Saboteur_Always_Beats_Turret [< 1 ms]
  Correctas SameRank_Results_InDraw [< 1 ms]

La serie de pruebas se ejecutó correctamente.
Pruebas totales: 6
  Correcto: 6
Tiempo total: 0,9629 Segundos
  NeuroForge.Tests prueba realizado correctamente (1,3s)

Resumen de pruebas: total: 6; con errores: 0; correcto: 6; omitido: 0; duración: 1,2 s
Compilación realizado correctamente en 2,7s
```

Figura 8.1: Resultado de la ejecución de las pruebas unitarias con NUnit.

Seguimiento del proyecto

Este capítulo describe el transcurso real de cada una de las cinco iteraciones que componen el desarrollo del proyecto, detallando los avances técnicos alcanzados y la evolución visual del sistema a lo largo del tiempo. El desarrollo se organizó siguiendo una metodología iterativa e incremental, lo que permitió incorporar funcionalidad de forma progresiva, validar cada bloque antes de avanzar al siguiente y responder a los cambios de diseño que surgieron durante el proceso. Cada iteración partía de los entregables de la anterior y tenía definidos unos objetivos concretos cuyo cumplimiento se verificó al finalizar cada fase.

9.1. Iteración 1: Núcleo jugable y lógica básica

Esta primera fase se centró en la implementación de la base jugable, tenía como objetivo principal crear un *Producto Mínimo Viable* funcional. En esta etapa, el videojuego carecía por completo de elementos estéticos, menús o *feedback* visual y sonoro, centrándose exclusivamente en la validez de los movimientos y la resolución de combates.

Durante esta iteración se utilizaron *placeholders* (elementos provisionales) para representar todas las entidades del videojuegos. La interfaz de despliegue consistía únicamente en botones con texto plano como se puede ver en la Figura 9.1, y las piezas se diferenciaban solo por colores e iconos esquemáticos sobre formas cuadradas. Como detalle de la imagen, también se pueden apreciar que los nombres de las piezas no reflejaban las definitivas del producto final, ya que se fueron haciendo cambios según se volvía a dar una vuelta a sus conceptos durante el desarrollo, o se implementaban su apariencia.

- **Piezas con rango:** Identificadas por un número según su valor de combate.
- **Núcleo de energía:** Representado por un signo de exclamación (!).
- **Torretas:** Identificadas mediante una letra "T".

La elección del color de las piezas responde a criterios de usabilidad y psicología del color básicos que se aplicaron casi de manera inconsciente, utilizando el azul para el jugador por su asociación con la calma y la seguridad, mientras que el naranja para el rival facilita un contraste visual inmediato y resalta la naturaleza competitiva y de alerta frente a las unidades enemigas.

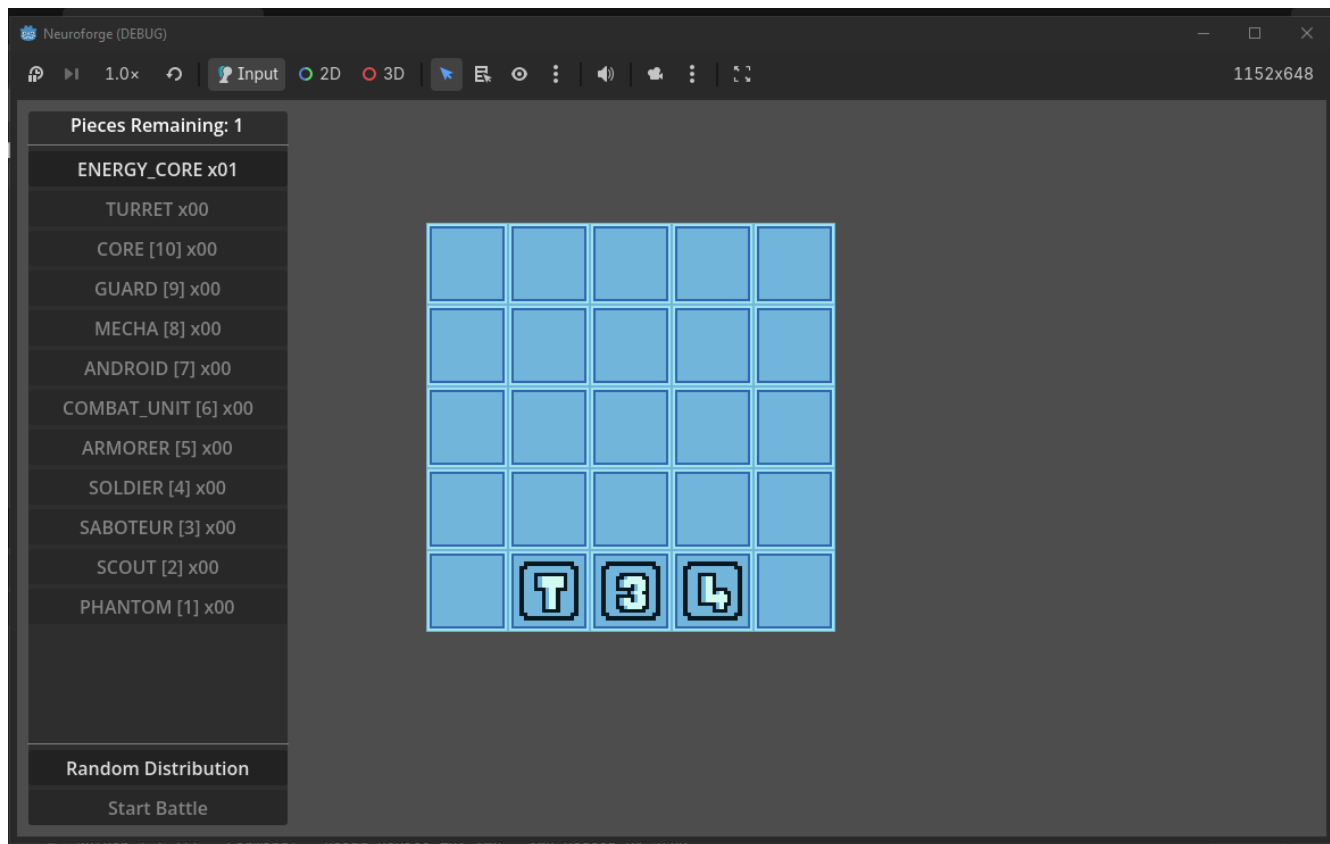


Figura 9.1: Detalle de la interfaz de despliegue inicial basada en botones de texto.

El tablero utilizaba una codificación de colores básica para la retroalimentación visual del usuario:

- **Gris:** Casillas intransitables que actúan como obstáculos permanentes.
- **Azul:** Casillas de agua o vacío. En esta fase no existía distinción visual entre las zonas de despliegue del jugador y de la inteligencia artificial, lo que dificultaba el posicionamiento inicial para alguien que no conociera el diseño del videojuego.
- **Verde:** Resaltado de casillas disponibles para movimiento tras seleccionar una pieza propia.
- **Rojo:** Indicador de casilla de combate inminente al seleccionar un objetivo enemigo.

La interacción era puramente lógica y carecía de retroalimentación visual. Los movimientos y combates eran instantáneos, sin animaciones de transición. Esta inmediatez provocaba problemas de seguimiento de la partida, ya que en ocasiones resultaba difícil saber que jugada realizado por el bot después del movimiento del jugador.

Asimismo, no existía un flujo de partida completo, al finalizar el encuentro por la captura del núcleo, el sistema no mostraba pantallas de victoria o derrota, limitándose a imprimir un mensaje en la terminal de depuración de Godot y a detener la ejecución del videojuego como se puede observar en la Figura 9.2. Tampoco se implementaron menús de pausa, opciones ni efectos de sonido. En la Figura 9.3 se pueden observar distintas partidas y situaciones del productor de la primera iteración.

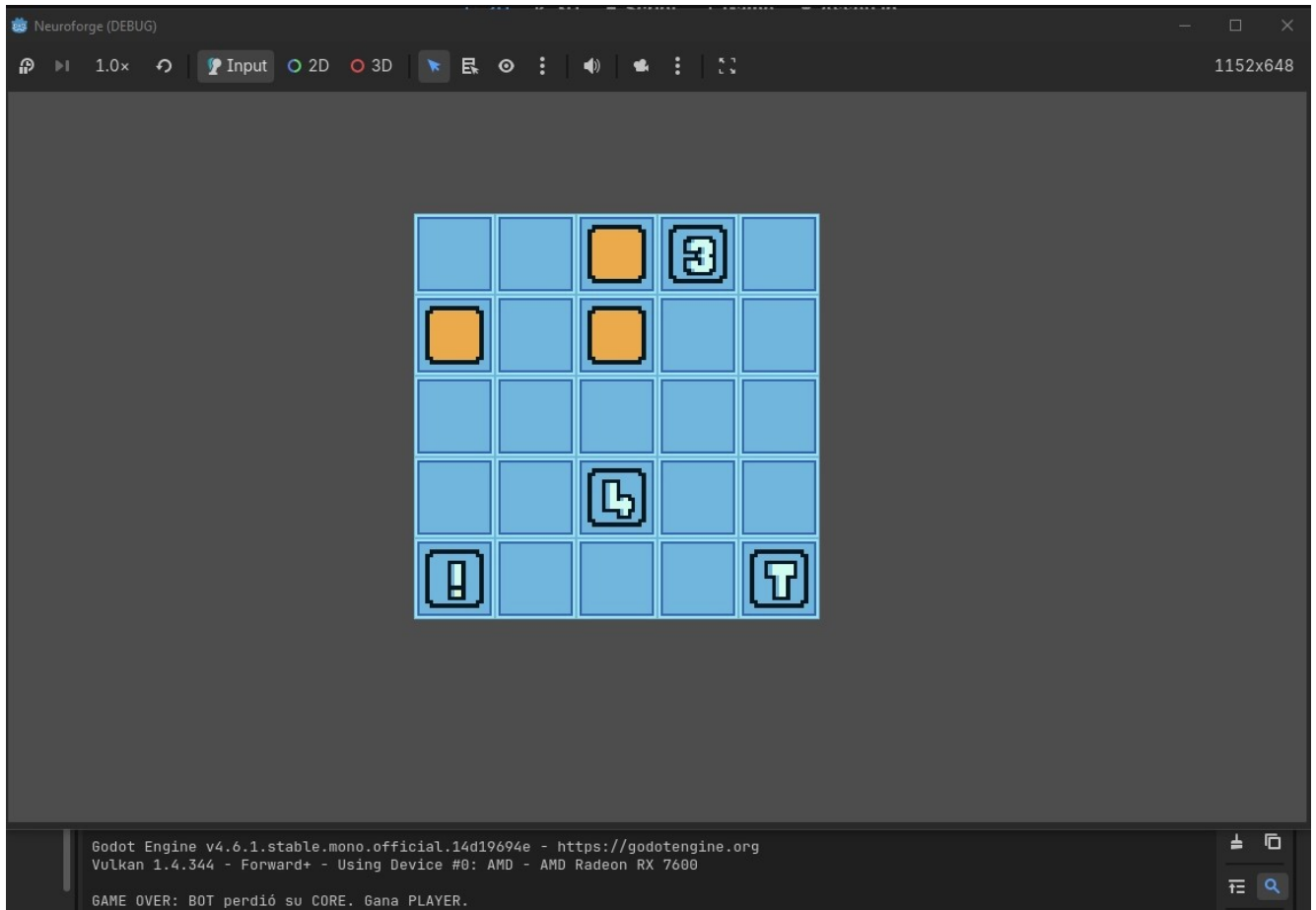


Figura 9.2: Mensaje de final de partida en la Iteración 1

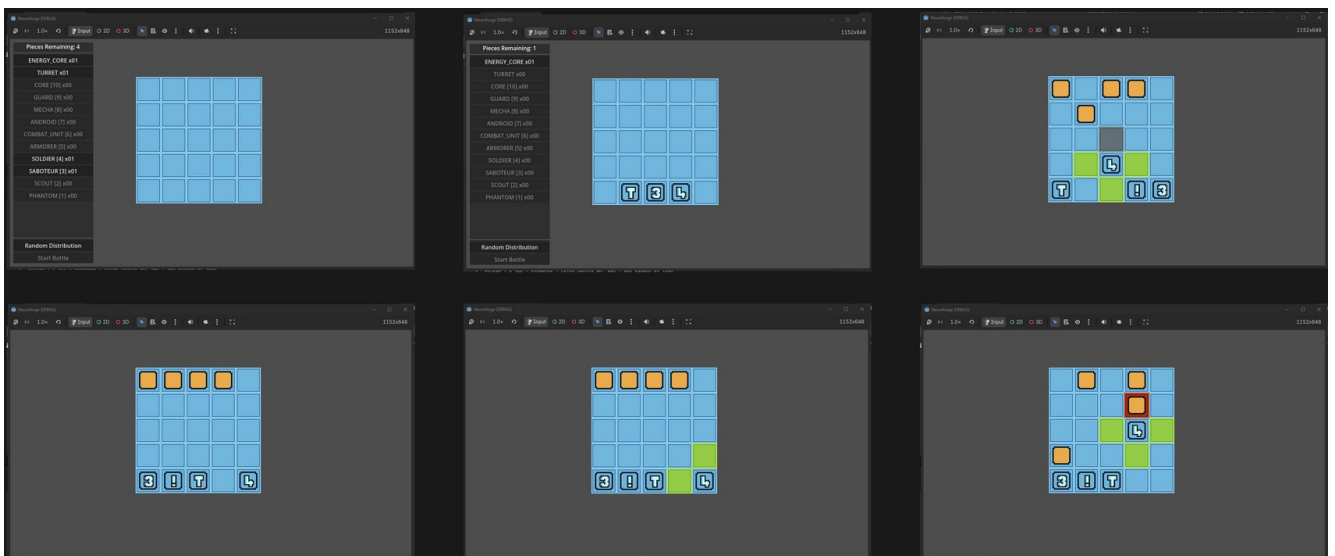


Figura 9.3: Capturas del juego durante la Iteración 1.

9.2. Iteración 2: Interfaz de usuario y pulido visual

En esta segunda fase, el desarrollo se centró en transformar los elementos provisionales de la iteración anterior en una interfaz de usuario (UI) coherente, intuitiva y visualmente atractiva. Para la planificación y el diseño de estas interfaces se utilizó la herramienta *draw.io*, permitiendo bocetar la disposición de los elementos antes de su implementación definitiva en el motor.

Interfaz de partida

La interfaz principal de videojuego se diseñó con el objetivo de centralizar toda la información necesaria para el jugador sin sobrecargar la vista. En su primera concepción, el boceto mostraba un tablero central rodeado por los indicadores de piezas restantes a ambos lados y un historial de movimientos situado en la esquina inferior izquierda. El estado del turno se indicaba mediante un texto descriptivo en la parte superior como se puede observar en la Figura 9.4.

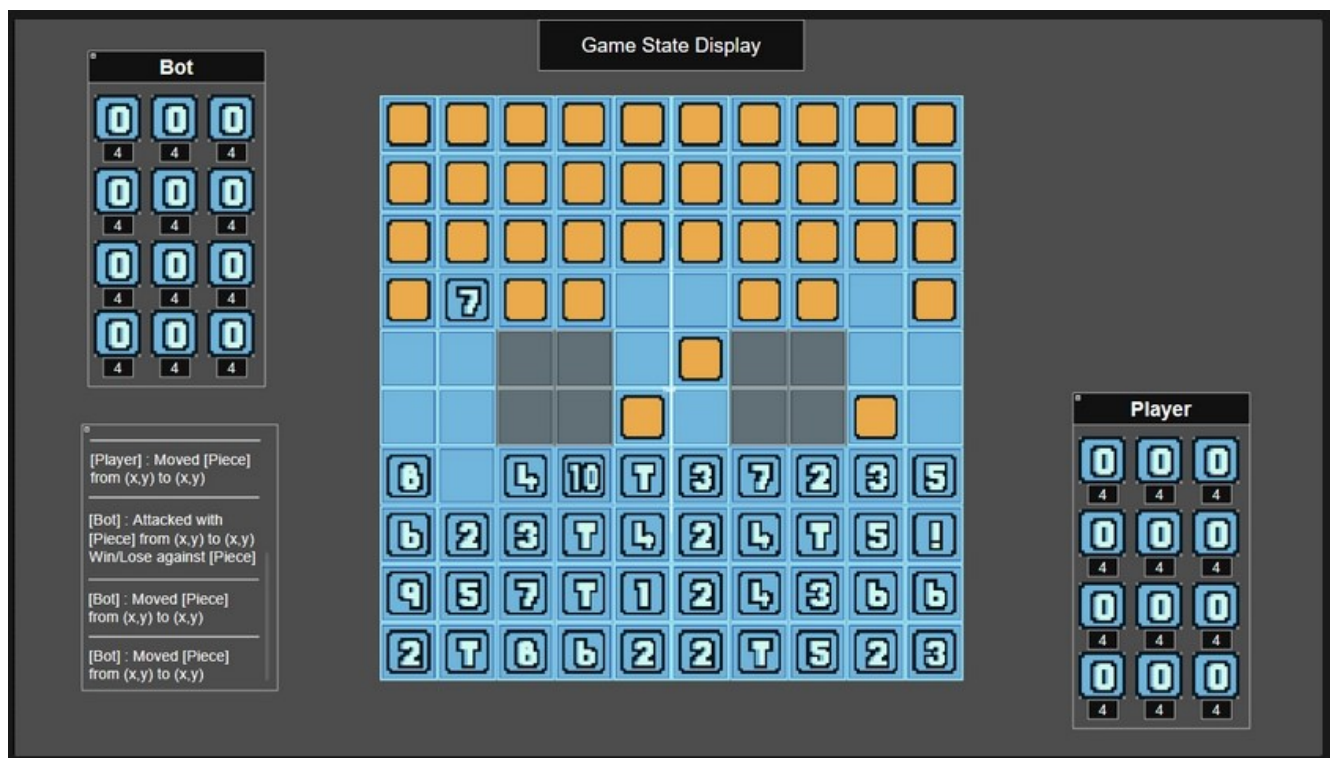


Figura 9.4: Primer boceto de la interfaz de partida con indicadores laterales y texto de turno.

En una segunda revisión del diseño, se buscó una mayor limpieza visual. Se sustituyó el display de texto superior por un marcador de turno más iconográfico y llamativo, situando el icono del jugador actual (o el bot) en un lateral. Asimismo, los contadores de piezas se agruparon a la izquierda para liberar espacio, desplazando el historial a la derecha como aparece en la Figura 9.5.

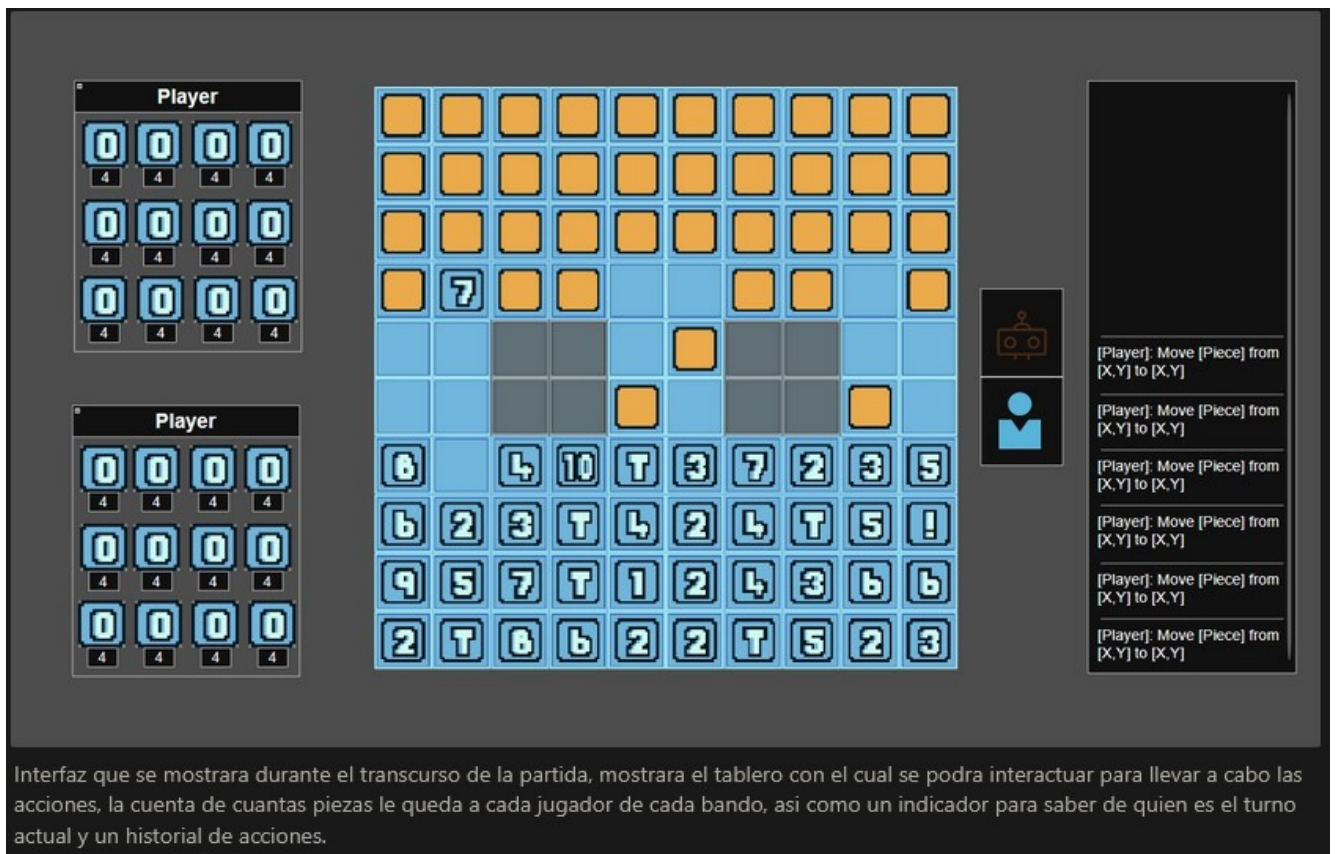


Figura 9.5: Evolución del diseño de partida con indicadores visuales de turno.

Finalmente, la estructura definitiva eliminó el historial de movimientos. Se consideró que esta información resultaba redundante y generaba un exceso de ruido visual que no aportaba un valor crítico a la experiencia de juego, dejando como resultado final el boceto de la Figura 9.6. En la implementación final, el marcador de turnos se simplificó de dos iconos que se iluminaban alternativamente a un único icono dinámico que cambia según el turno activo.

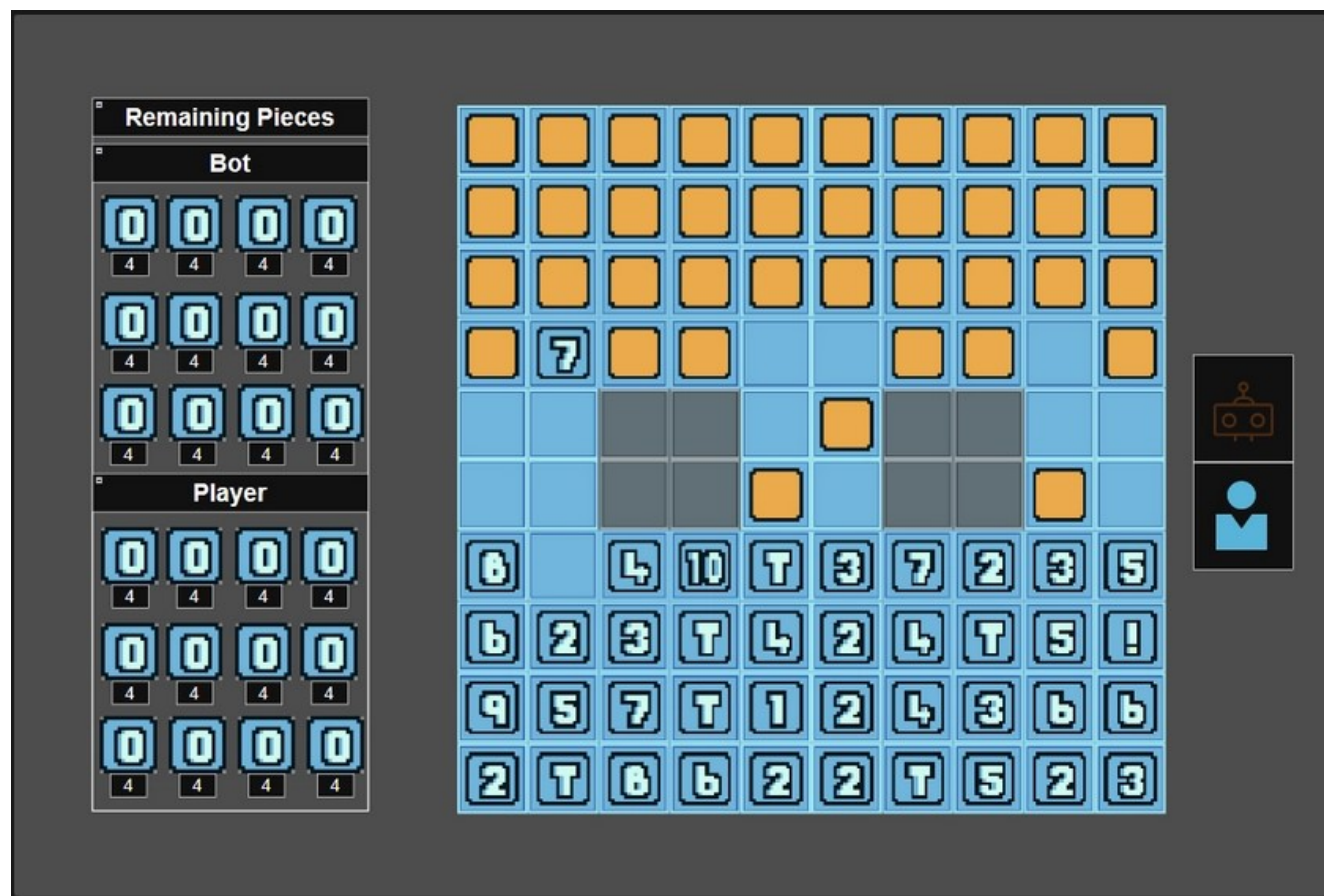


Figura 9.6: Layout definitivo del boceto de la interfaz de partida.

Interfaz de combate y reglas

El diseño de la interfaz de combate siguió una lógica similar de simplificación. Inicialmente, se planteó como una estructura de contenedores anidados que separaban visualmente al atacante del defensor. Sin embargo, para la versión final se optó por eliminar los marcos contenedores externos, dejando los elementos de las piezas más libres y situando el mensaje del resultado del combate en la parte inferior de las mismas para no entorpecer la visión de la acción. Dicha diferencia puede apreciarse en los bocetos que aparecen en la Figura 9.10.

A continuación, se presentan los bocetos que detallan el funcionamiento de los distintos estados de resolución de un combate: en la Figura 9.7 el inicio del combate, en la Figura 9.8 la comparación entre piezas, y en la Figura 9.9 el resultado final antes de cerrar la interfaz.



Figura 9.7: Boceto de inicio de combate: presentación de piezas.

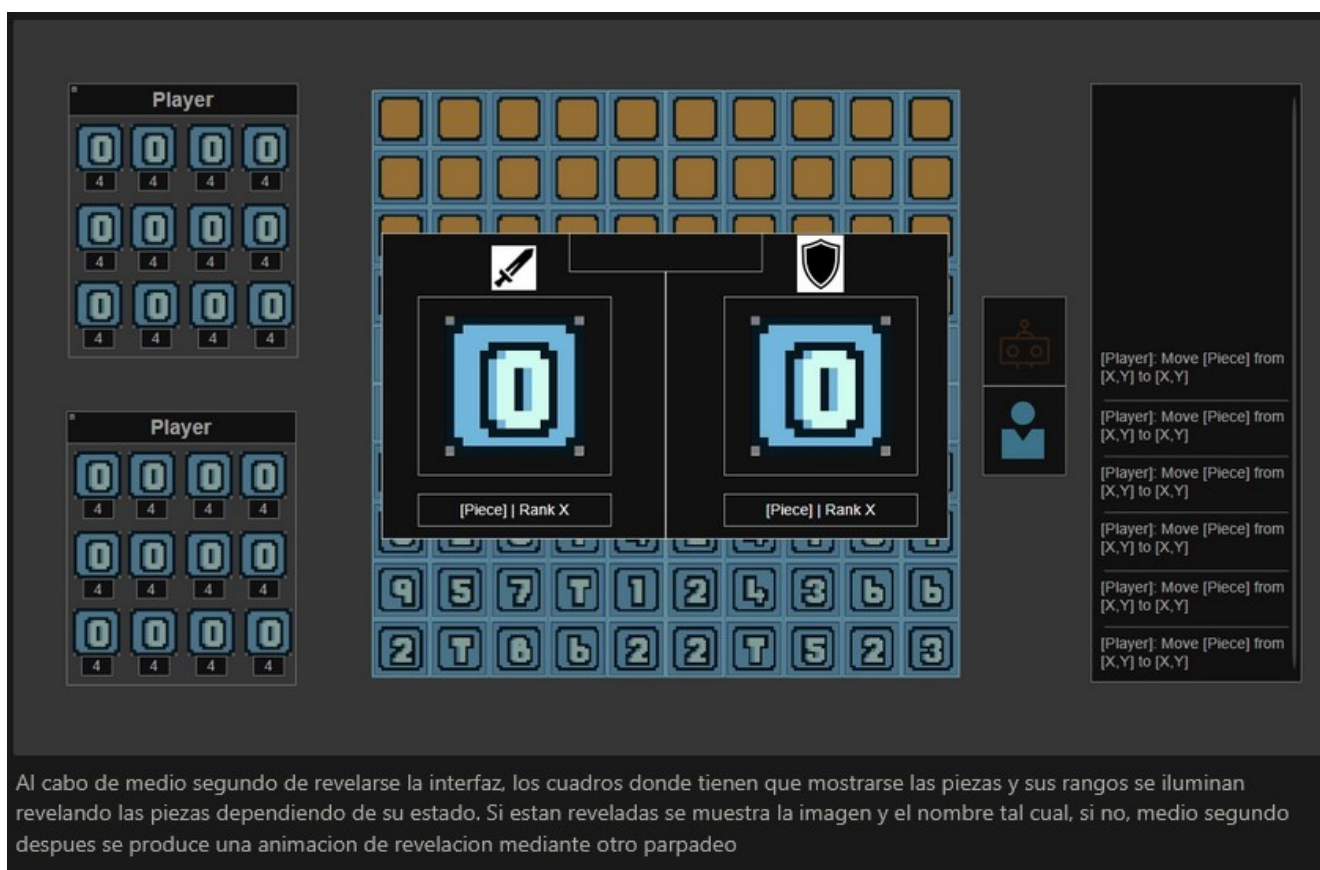


Figura 9.8: Boceto de resolución: comparación de rangos.

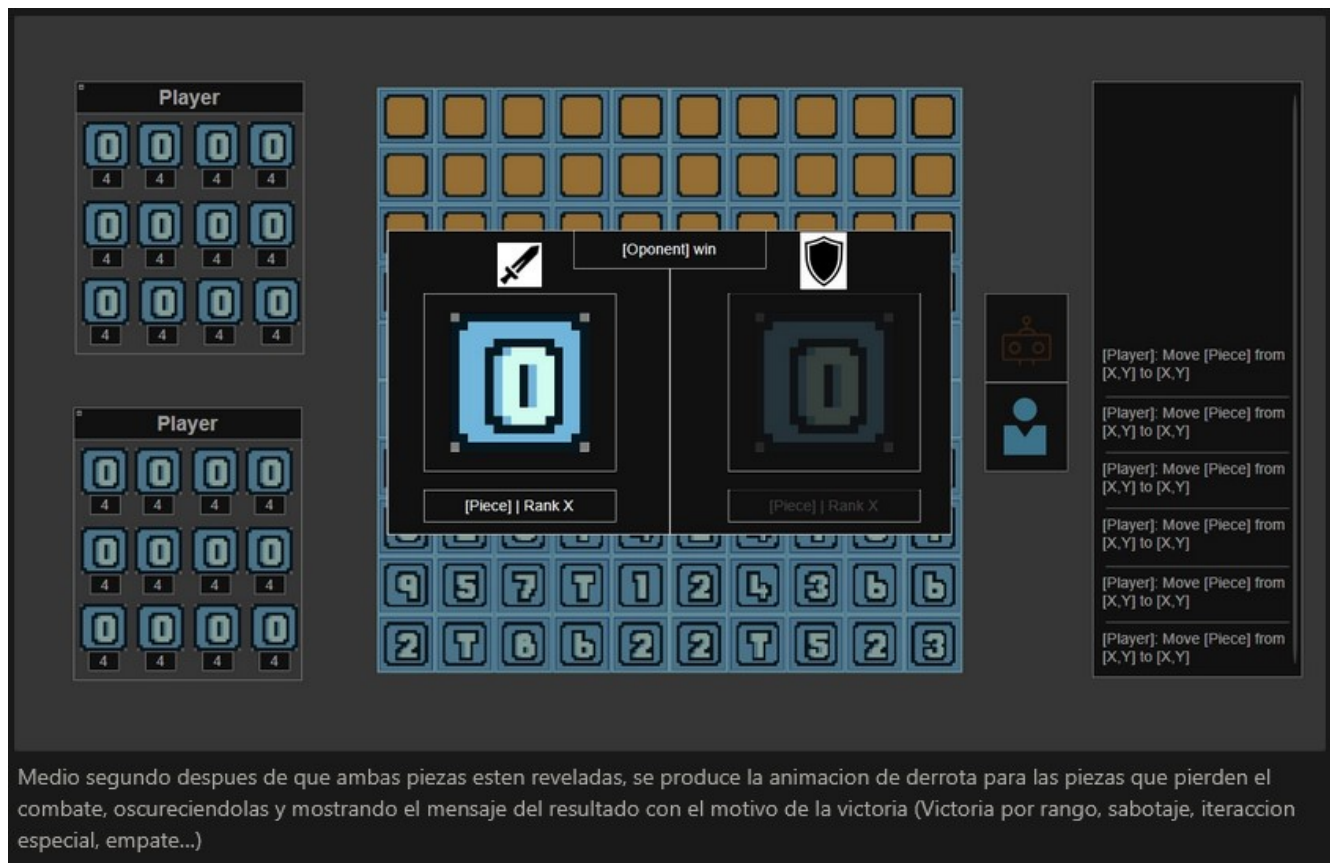


Figura 9.9: Boceto de fin de combate: pieza eliminada.

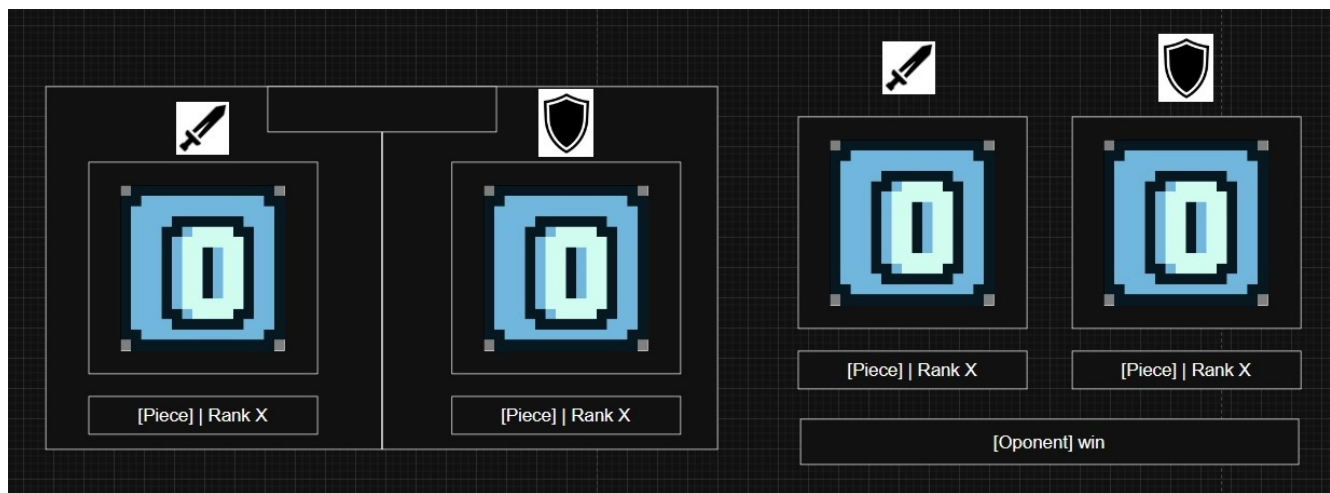


Figura 9.10: Comparativa entre el diseño inicial de combate y el diseño final simplificado.

Por último, se diseñó el menú de pausa de la Figura 9.11 y la guía de reglas. Para este último, se descartó la idea inicial de un *popup* único con texto denso, como el de la Figura 9.12, por el de un sistema de cinco viñetas navegables. Cada viñeta aborda un aspecto específico: el objetivo principal, tipos de casillas, reglas de movimiento, reglas de combate y datos de las piezas. Este enfoque permite al jugador asimilar las mecánicas de forma progresiva.

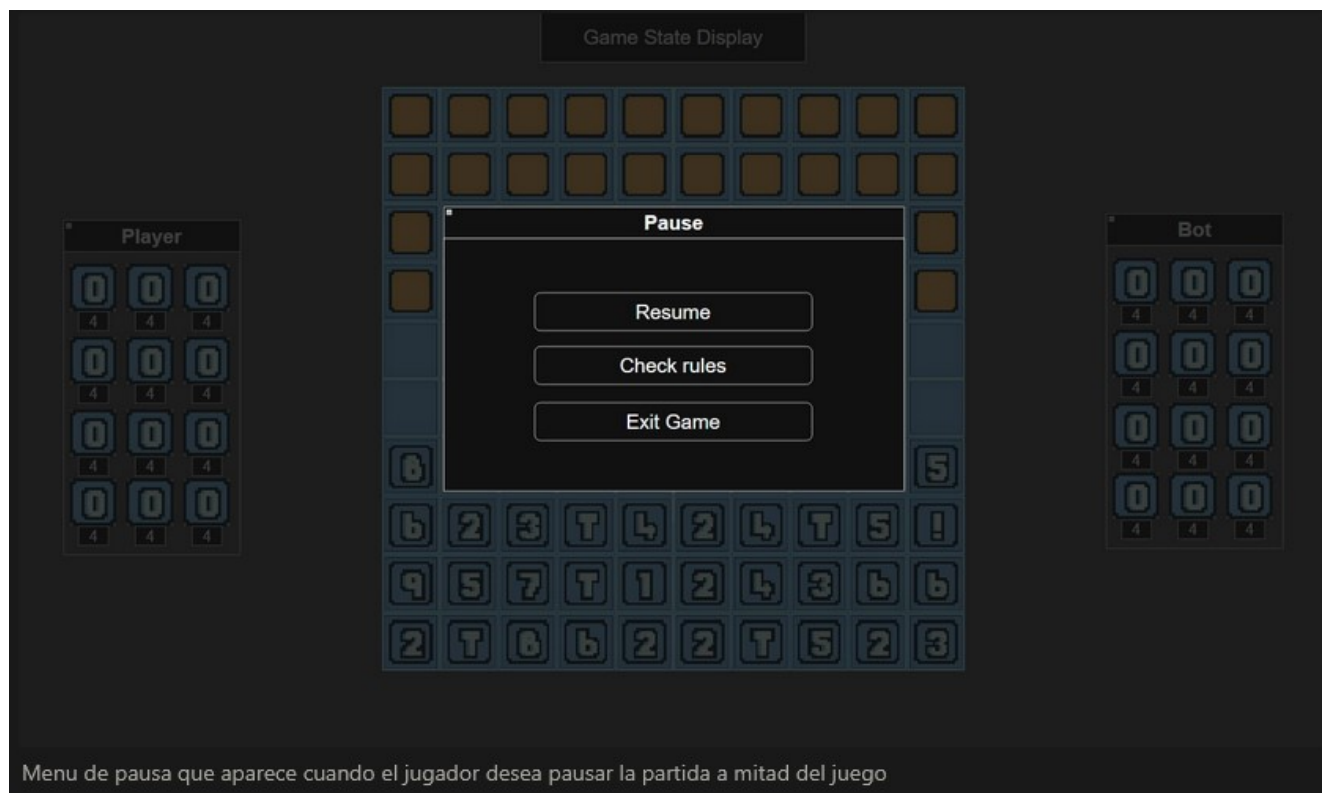


Figura 9.11: Boceto del menú de pausa de la partida.

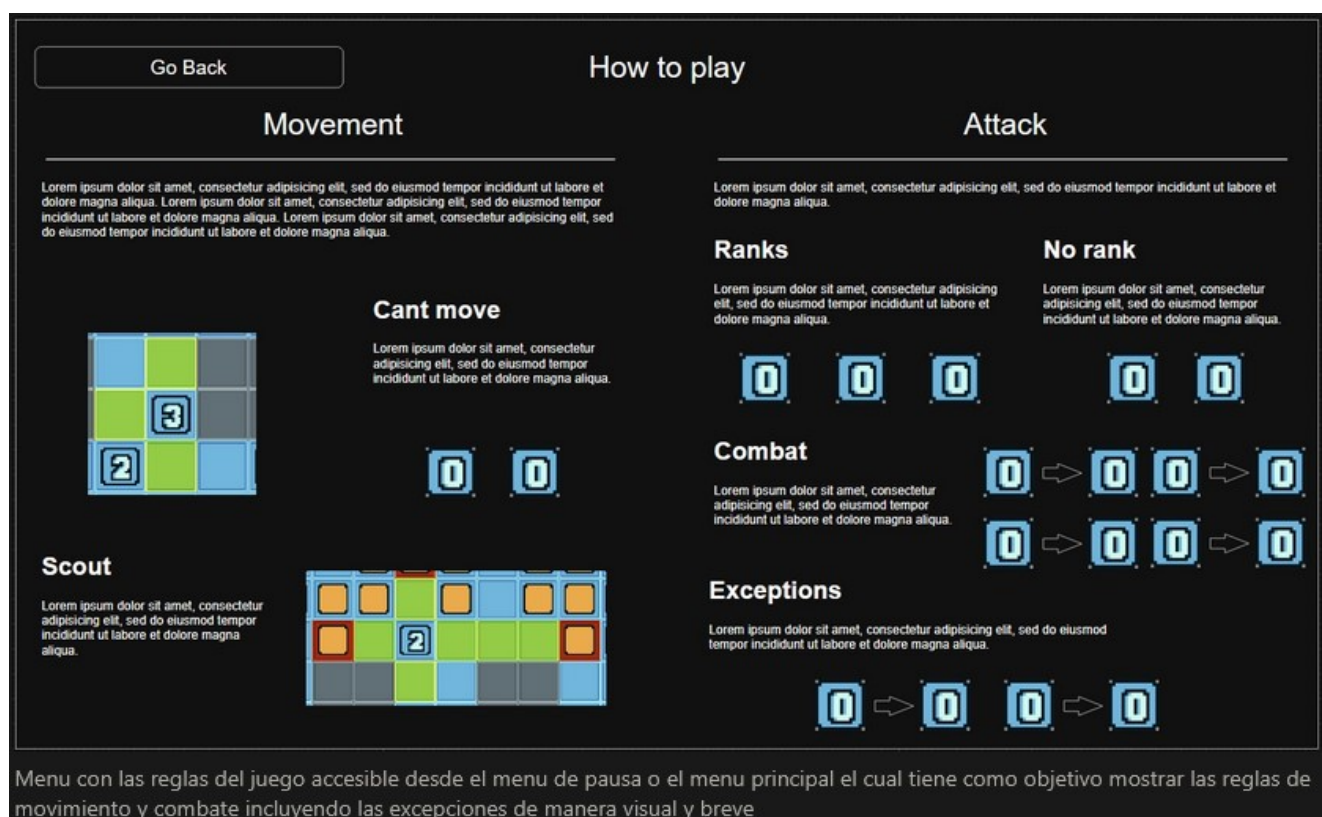


Figura 9.12: Boceto de la interfaz de reglas con sistema de navegación por viñetas.

No se realizaron bocetos adicionales para los menús principal, de opciones o de fin de partida, dada su sencillez estructural. Cabe destacar que el diseño estético definitivo y la implementación final de todas estas interfaces que se desarrollaron a lo largo de esta iteración se detallan en el capítulo de Diseño.

9.3. Iteración 3: Mejoras audiovisuales

Una vez consolidada la base visual y lógica, la tercera iteración se dedicó íntegramente al apartado sonoro y al refuerzo del *game feeling*. El objetivo principal fue eliminar la sensación de monotonía y vacío que presentaba el prototipo hasta el momento.

Para ello, se incorporaron pistas de música ambiental diferenciadas tanto para los menús como para el transcurso de la partida, dotando al videojuego de una atmósfera más inmersiva. En cuanto a los efectos de sonido (SFX), se implementaron tres niveles de interacción:

- **Interfaz:** Sonidos sutiles de *hover* y *click* para que el jugador perciba qué elementos son interactivables.
- **Tablero:** Feedback auditivo al seleccionar y desplazar piezas, reforzando la sensación de manipulación física.
- **Eventos de partida:** Sonidos específicos para la revelación de piezas, derrotas en combate o cambios de estado, proporcionando una respuesta inmediata a las acciones del jugador.

Finalmente, se incluyeron fondos coherentes entre el menú principal y la escena de la partida para unificar la estética y evitar que el transcurso de la partida se sintiera visualmente vacío. A estos fondos se les aplicó un ligero efecto de *parallax*, lo que aporta profundidad y una mayor sensación de pulido a la experiencia.

9.4. Iteración 4: Implementación del bot inteligente

La cuarta iteración se dedicó al desarrollo del sistema de inteligencia artificial. El trabajo se realizó en Python utilizando la biblioteca *Gymnasium* para crear el entorno de juego y el algoritmo *MaskablePPO* para el entrenamiento.

El primer bloque consistió en programar el entorno de entrenamiento de forma independiente al motor Godot. Este entorno simula el tablero de juego mediante matrices de tres canales que representan las piezas, los obstáculos y los movimientos posibles. Además, incluye una máscara de acciones que impide al agente elegir jugadas ilegales, lo que ayuda a estabilizar el aprendizaje.

El segundo bloque se centró en el entrenamiento mediante la estrategia de *self-play*. En este modo, el agente juega contra versiones anteriores de sí mismo, lo que le permite aprender tácticas avanzadas de forma autónoma sin necesidad de programar manualmente las reglas de puntuación. Las versiones del modelo se guardaron periódicamente para poder retomar el proceso si fuera necesario.

El tercer bloque consistió en integrar el modelo con Godot de forma desacoplada. Para ello, se creó el script de Python `bot_inference.py` y se programó la clase `BotController` en C# dentro del juego. Al comenzar una partida, el código en C# abre un subproceso para ejecutar el script de Python y comunicarse con él mediante mensajes JSON por la entrada y salida estándar. Durante las pruebas, se comprobó que el flujo de datos funcionaba de manera fluida y sin congelar la pantalla del juego. Esta separación permite actualizar el modelo de la IA sin tener que modificar ni volver a compilar el videojuego.

9.5. Iteración 5: Pulido final

La quinta y última iteración se dedicó a la consolidación del sistema completo, abordando los aspectos de calidad y estabilidad que habían quedado pendientes en fases anteriores.

En el apartado de rendimiento, se revisaron los ciclos de actualización de los nodos más activos durante la partida y se optimizó la gestión de señales y eventos para reducir el número de llamadas innecesarias en cada fotograma. El resultado fue una ejecución fluida y estable en el hardware mínimo especificado en los requisitos no funcionales.

La revisión del feedback al jugador permitió detectar y corregir varios casos en los que la respuesta visual o sonora a una acción no era suficientemente clara, especialmente en situaciones de combate entre piezas reveladas previamente y en la transición entre el turno del jugador y el del bot. Se ajustaron los tiempos de animación y se reforzaron los efectos sonoros asociados a estos momentos para mejorar la legibilidad de la partida.

En cuanto a la corrección de errores, se resolvieron varios casos límite relacionados con la regla antioscilación en situaciones de tablero muy congestionado, y se corrigió un comportamiento incorrecto del explorador al intentar moverse a través de casillas ocupadas en determinadas configuraciones. Las pruebas de integración finales verificaron el comportamiento completo del sistema de principio a fin, desde el despliegue inicial hasta todas las condiciones de victoria posibles, sin detectar errores adicionales.

Conclusiones

Este capítulo recoge la valoración global del trabajo realizado, analizando el grado de cumplimiento de los objetivos planteados al inicio del proyecto y reflexionando sobre las decisiones técnicas adoptadas a lo largo del desarrollo. Se incluye además una declaración sobre el uso de herramientas de inteligencia artificial generativa durante el proceso, y se proponen las líneas de trabajo futuro más relevantes para continuar evolucionando el proyecto más allá del alcance definido en este TFG.

10.1. Valoración de objetivos

El desarrollo de *NeuroForge* ha permitido alcanzar de forma satisfactoria los objetivos principales planteados al inicio del proyecto. En primer lugar, se ha diseñado e implementado con éxito la lógica fundamental del videojuego, consolidando una estructura de tablero funcional, la definición precisa de las piezas y las reglas que rigen su movimiento y combate. Sobre esta base, se ha desarrollado un robusto sistema de control de turnos y gestión del estado de la partida, lo que garantiza la ejecución de enfrentamientos completos bajo el estricto cumplimiento de las mecánicas diseñadas. Todo este proceso ha propiciado una valiosa adquisición de experiencia práctica en el uso del motor de videojuegos Godot y en el desarrollo de sistemas interactivos empleando C# como lenguaje principal. Como resultado, la experiencia de juego es coherente, estable y accesible.

La interfaz de usuario alcanza el nivel de pulido propuesto, todas las pantallas están implementadas, el flujo de navegación es intuitivo y el apartado audiovisual refuerza adecuadamente la ambientación de ciencia ficción del proyecto. La arquitectura en tres capas adoptada ha demostrado ser una decisión acertada, manteniendo la lógica del videojuego completamente desacoplada del motor y facilitando tanto las pruebas como el mantenimiento del código a lo largo de las iteraciones.

El desarrollo del bot inteligente mediante aprendizaje por refuerzo constituyó el objetivo técnicamente más ambicioso del proyecto y ha sido completado satisfactoriamente. El entorno de entrenamiento basado en Gymnasium replica con fidelidad las reglas del videojuego, incluyendo la

gestión de información oculta y las reglas especiales de combate, y el agente entrenado mediante MaskablePPO y estrategia de *self-play* ha demostrado un comportamiento estratégico coherente, siendo capaz de gestionar la incertidumbre inherente al juego sin necesidad de una función de evaluación diseñada manualmente. Su integración con Godot a través de un subproceso Python desacoplado del motor ha resultado en una solución técnicamente sólida y fácilmente actualizable.

10.2. Uso de inteligencia artificial

Durante el desarrollo de este proyecto se ha hecho uso de herramientas de inteligencia artificial generativa como apoyo en determinadas tareas, sin que en ningún caso hayan sustituido la toma de decisiones técnicas ni el trabajo de diseño e implementación propios.

En el ámbito de la documentación, la IA se empleó principalmente como asistente para la redacción en L^AT_EX para resolver dudas sobre sintaxis específica, formatear correctamente tablas o figuras, y transformar texto redactado a mano en código L^AT_EX bien estructurado. Esto permitió agilizar la fase de escritura de la memoria sin delegar en ningún momento el contenido ni el criterio técnico de lo escrito.

En el ámbito del código, su uso se limitó a tareas de soporte, refactorización de fragmentos para mejorar su legibilidad, consultas puntuales sobre la API de C# o sobre patrones de uso específicos de Godot, y en algún caso la sugerencia de estructuras alternativas para organizar mejor un bloque de lógica ya diseñado. La arquitectura del sistema, las decisiones de diseño y la implementación de las mecánicas del videojuego fueron elaboradas de forma íntegramente propia.

10.3. Líneas futuras

El estado actual del proyecto constituye una base sólida sobre la que es posible crecer en múltiples direcciones. A continuación se detallan las líneas de trabajo futuro más relevantes, ordenadas aproximadamente de menor a mayor coste de implementación.

Una de las fortalezas de la arquitectura implementada es que el tablero no está codificado de forma rígida en la lógica del videojuego, sino que se define como una matriz que Godot interpreta directamente desde el editor. Esto significa que crear una variante con dimensiones distintas, una distribución diferente de zonas intransitables o un conjunto de piezas modificado requiere únicamente editar esa configuración sin tocar el código, lo que abre la puerta a modos de juego alternativos con un coste de desarrollo muy reducido.

Con un esfuerzo algo mayor, la estructura de datos del estado de la partida, basada en clases C# puras con información bien delimitada por pieza y por casilla, se presta de forma natural a la serialización. Implementar un sistema de guardado y carga consistiría esencialmente en serializar ese estado a un archivo JSON y deserializarlo al reanudar, una tarea que la arquitectura actual no requeriría modificar en absoluto. En esa misma línea, con el agente inteligente ya entrenado e integrado, resultaría relativamente sencillo exponer distintos niveles de dificultad al jugador

mediante modelos entrenados durante diferente número de iteraciones, ajustando el grado de aleatoriedad de la política o combinando el bot inteligente con el bot aleatorio en proporciones variables, lo que produciría una curva de dificultad progresiva sin necesidad de desarrollar un sistema nuevo.

Con las bases del sistema implementadas, añadir nuevo contenido que no diverja de los fundamentos del juego tiene también un coste relativamente bajo. Entre las posibilidades más interesantes se encuentran modificadores opcionales activables antes de cada partida, como alterar la regla de empate para que gane el atacante o introducir un temporizador por turno al estilo del ajedrez. Nuevos tipos de piezas también enriquecerían la experiencia: una torre de radar inmóvil que tras cierto número de turnos permita revelar una pieza enemiga a elección ampliaría la dimensión deductiva del videojuego de forma coherente con sus mecánicas actuales. Del mismo modo, nuevos tipos de casillas como teletransportadores vinculados por pares o zonas que destruyen periódicamente las piezas que las ocupan añadirían profundidad táctica sin necesidad de rediseñar el sistema de base.

Finalmente, respecto a la publicación y distribución, antes de plantear una distribución comercial resultaría conveniente publicar el videojuego en *itch.io*, plataforma que permite obtener retroalimentación real de jugadores sin ningún coste de entrada, identificar posibles problemas de usabilidad y generar una pequeña base de usuarios. Una vez incorporadas las mejoras anteriores, especialmente el bot con niveles de dificultad, el sistema de guardado y al menos una variedad de tableros adicionales, el producto estaría en condiciones de justificar su presencia en *Steam* al precio de 1,99 € analizado en el capítulo de planificación. Si se quisiera apoyar el lanzamiento con mayor solidez, la sustitución de los recursos visuales actuales por arte original marcaría la diferencia entre un prototipo académico y un producto independiente con identidad propia. En cualquier caso, se recomienda esperar al menos un mes tras las mejoras antes de publicar, tiempo suficiente para estabilizar el producto y preparar una página de tienda cuidada, factores que influyen directamente en la conversión de visitas a ventas en esa plataforma.

Manual de instalación

NeuroForge se distribuye como una aplicación de escritorio autocontenida para Windows y Linux. No requiere proceso de instalación ni dependencias externas; basta con descomprimir el paquete de distribución y ejecutar el archivo principal.

A.1. Requisitos del sistema

Antes de ejecutar el videojuego, se recomienda un equipo con las siguientes especificaciones:

- **Sistema operativo:** Windows 10 o Windows 11 (64 bits), o distribución de Linux lanzada a partir de 2018.
- **Procesador:** CPU de doble núcleo a 2,0 GHz o superior.
- **Memoria RAM:** 4 GB.
- **Tarjeta gráfica:** GPU con soporte completo de Vulkan 1.0 (por ejemplo, Intel HD Graphics 510 o superior).
- **Almacenamiento:** 200 MB de espacio libre en disco.

A.2. Procedimiento de ejecución

Para poner en marcha el videojuego por primera vez, se deben seguir los siguientes pasos orientados al despliegue del software:

1. Descomprimir el archivo comprimido de la distribución (`NeuroForgeWindows.zip` o `NeuroForgeLinux.tar.gz`) en el directorio local deseado dentro del sistema de archivos.
2. Comprobar que el directorio resultante de la extracción contiene la estructura base necesaria para la correcta ejecución del sistema:
 - `NeuroForge.exe` (en entornos Windows) o `NeuroForge.x86_64` (en entornos Linux).
 - **Carpeta Bot/:**
 - `bot_inference.py`
 - `neuroforge_bot_vXXXXXX.zip`
3. Proceder con el lanzamiento de la aplicación según el sistema operativo anfitrión:
 - **En Microsoft Windows:** Realizar doble clic sobre el archivo ejecutable `NeuroForge.exe`.
 - **En Linux:** Asegurar los permisos de ejecución del binario abriendo una terminal en el directorio y ejecutando el comando `chmod +x NeuroForge.x86_64`. Posteriormente, iniciar el juego mediante doble clic o a través de la consola con la instrucción `./NeuroForge.x86_64`.

A.3. Solución de problemas comunes

Si el videojuego no arranca o la ventana se cierra inesperadamente al iniciarse, se recomienda verificar que los controladores de la tarjeta gráfica del equipo están actualizados a su última versión y ofrecen soporte completo para la API Vulkan 1.0. En equipos portátiles que dispongan de arquitectura de gráficos híbrida (tarjeta integrada y tarjeta dedicada), puede ser necesario forzar el uso de la GPU de alto rendimiento desde el panel de control del fabricante (NVIDIA Control Panel o AMD Software).

La subcarpeta `Bot` se encuentra en la misma ruta relativa que el binario ejecutable, manteniendo intactos en su interior tanto el script `bot_inference.py` como el archivo comprimido del modelo (`neuroforge_bot_vXXXXXX.zip`) sin modificaciones en sus cadenas de texto. De lo contrario el videojuego seguirá funcionando igualmente pero sin oponente inteligente.

Manual de usuario

Este manual describe el funcionamiento de *NeuroForge* y guía al jugador a través de todas las pantallas y mecánicas del videojuego. Toda la interacción se puede realizar solamente con el ratón.

B.1. Menú principal

Al iniciar el videojuego se muestra el menú principal (Figura B.1), desde el que se puede acceder a las tres opciones disponibles: *Play*, para comenzar una nueva partida; *Rules*, para consultar las reglas del videojuego; y *Exit*, para cerrar la aplicación.



Figura B.1: Menú principal de NeuroForge.

B.2. Menú de reglas

El menú de reglas es accesible tanto desde el menú principal como desde el menú de pausa durante una partida. Se organiza en cinco pestañas navegables en la parte superior de la ventana: *Objective*, con el objetivo y las condiciones de victoria; *Board*, con los tipos de casillas del tablero; *Movement*, con las reglas de desplazamiento; *Combat*, con la mecánica de enfrentamiento; y *Pieces*, que permite consultar el rango, la movilidad y las reglas especiales de cada tipo de pieza pulsando sobre su botón correspondiente. Para cerrar el menú se pulsa el botón *Go Back* en la esquina superior izquierda.

B.3. Fase de despliegue

Antes de comenzar la partida, el jugador debe colocar todas sus piezas en su zona de despliegue, las cuatro filas más próximas al borde inferior del tablero. La Figura B.2 muestra esta pantalla con algunas piezas ya colocadas, con los elementos principales numerados.

Para colocar una pieza manualmente, se pulsa sobre su botón en el panel lateral izquierdo (3) para seleccionarla (4) y a continuación se hace clic sobre cualquier casilla de despliegue del jugador (10). Si se hace clic sobre una pieza ya colocada en el tablero (9), esta se retira y su tipo vuelve a quedar seleccionado en el panel para reposicionarla. Cuando se han colocado todas las unidades de un tipo, el botón correspondiente aparece desactivado (5). El contador en la parte superior del panel (1) indica cuántas piezas quedan todavía por colocar.

El botón de *Despliegue aleatorio* (6) distribuye automáticamente todas las piezas restantes en posiciones aleatorias dentro de la zona de despliegue. Una vez colocadas todas las piezas, el botón de *Comenzar partida* (7) se activa y al pulsarlo el bot despliega sus piezas en su zona (8) y comienza la partida.



Figura B.2: Fase de despliegue con algunas piezas ya colocadas en la zona del jugador.

1. **Panel de despliegue:** Indica las piezas totales que quedan por colocar.
2. **Tablero:** Zona donde desplegar las piezas seleccionadas.
3. **Pieza pendiente por colocar:** Botón con contador individual que muestra cuántas unidades de ese tipo quedan por colocar.
4. **Pieza seleccionada:** Tipo de pieza activo que se colocará en la siguiente casilla válida en la que se haga clic.
5. **Pieza agotada:** Tipo del que ya se han colocado todas las unidades disponibles.
6. **Botón de despliegue aleatorio:** Distribuye aleatoriamente todas las piezas restantes en la zona de despliegue del jugador.
7. **Botón de inicio de partida:** Inicia la partida. Permanece desactivado mientras queden piezas por colocar.
8. **Casilla de despliegue del bot:** Zona reservada al bot; sus piezas se colocan aquí automáticamente al pulsar *Start Battle*.
9. **Pieza desplegada:** Pieza ya colocada en el tablero. Al hacer clic sobre ella se retira y su tipo vuelve a quedar seleccionado en el panel.
10. **Casilla de despliegue del jugador:** Casillas donde el jugador puede colocar sus piezas.

B.4. Partida en curso

La partida se desarrolla en turnos alternos, comenzando siempre el jugador. Al inicio de cada turno del jugador, las piezas que tienen algún movimiento disponible aparecen ligeramente resaltadas para identificarlas de un vistazo. Al hacer clic sobre una de ellas, el tablero muestra en verde las casillas a las que puede desplazarse (3) y en rojo las casillas ocupadas por piezas enemigas a las que puede atacar (2), tal como se aprecia en la Figura B.3. Haciendo clic sobre una casilla resaltada se confirma la acción. Para cancelar la selección basta con hacer clic fuera de las casillas resaltadas.

Si la pieza se mueve a una casilla vacía, el turno pasa automáticamente al bot. Si se mueve a una casilla ocupada por una pieza enemiga, se inicia un combate y aparece la interfaz de resolución. El bot ejecuta su turno de forma automática y sin intervención del jugador.

El panel lateral izquierdo (1) muestra en todo momento el recuento de piezas restantes de cada bando. El marcador de turno (7) situado a la derecha muestra un icono de persona durante el turno del jugador y una calavera durante el del bot. Las piezas propias cuya identidad ya conoce el bot aparecen rodeadas por un borde naranja (8), lo que ocurre cuando han sobrevivido a un combate o han realizado un movimiento que revela su tipo.



Figura B.3: Pantalla de partida con una pieza seleccionada y sus acciones posibles resaltadas.

1. **Panel de piezas:** Recuento de piezas restantes de ambos bandos, útil para llevar la cuenta de qué tipos quedan en juego.
2. **Casilla marcada para combate:** Casilla ocupada por una pieza enemiga a la que la pieza seleccionada puede atacar; se resalta en rojo.

3. **Casilla marcada para movimiento:** Casilla libre a la que la pieza seleccionada puede desplazarse; se resalta en verde.
4. **Pieza seleccionada:** Pieza activa cuyas casillas de acción quedan resaltadas al hacer clic sobre ella.
5. **Casilla transitable:** Casilla por la que se pueden mover las piezas.
6. **Casilla intransitable:** Casilla que ninguna pieza puede atravesar ni ocupar.
7. **Marcador de turno:** Indica de quién es el turno: icono de persona para el jugador, calavera para el bot.
8. **Pieza revelada para el bot:** Pieza propia cuya identidad ya conoce el bot, indicada por un borde naranja.

B.5. Interfaz de combate

Cuando dos piezas se enfrentan, aparece la ventana de combate que se muestra en la Figura B.4. El atacante aparece a la izquierda, marcado con un icono de mira roja, y el defensor a la derecha, identificado con un escudo azul. Si alguna de las piezas era desconocida para el rival, su sprite parpadea y se revela en este momento. A continuación, la pieza o piezas perdedoras se oscurecen y en la parte inferior aparece un mensaje indicando quién ha ganado y el motivo. Para continuar la partida basta con hacer clic en cualquier punto de la ventana.

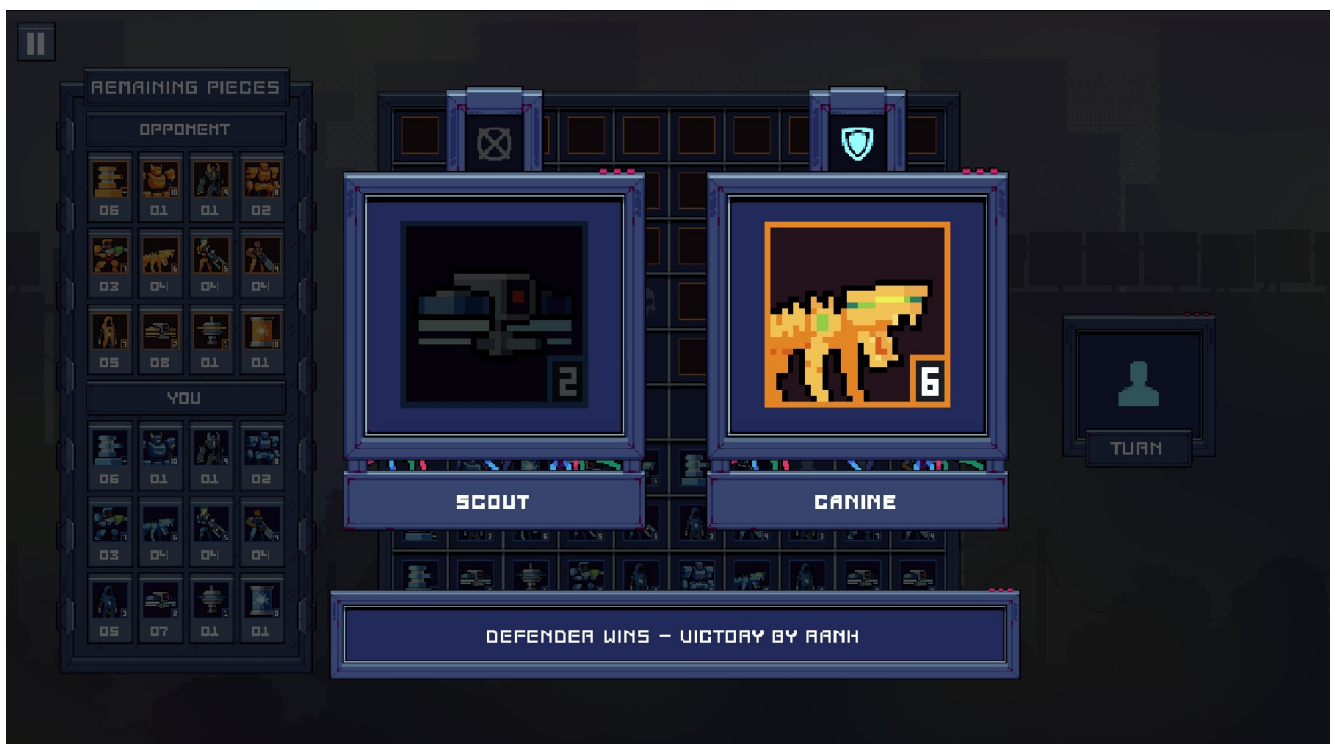


Figura B.4: Interfaz de combate tras la resolución: pieza perdedora oscurecida y mensaje de resultado visible.

B.6. Menú de pausa

Durante el turno del jugador es posible pausar la partida pulsando la tecla *Escape*. El menú de pausa ofrece cuatro opciones: *Resume* para reanudar la partida desde el punto en que se pausó; *Rules* para consultar las reglas sin abandonar la partida; *Options* para acceder a la configuración de audio y visualización; y *Return to Menu* para abandonar la partida en curso y volver al menú principal.

B.7. Menú de opciones

Accesible desde el menú principal y desde el menú de pausa, el menú de opciones permite cambiar el idioma del juego, alternar entre modo ventana y pantalla completa, y ajustar de forma independiente el volumen de los cuatro canales de audio disponibles: *Master*, *Music*, *SFX* y *UI*. Todos los cambios se aplican de forma inmediata. Para cerrar el menú se pulsa *Go Back*.

B.8. Fin de partida

La partida termina cuando se destruye el núcleo de energía de uno de los bandos o cuando uno de ellos se queda sin piezas con capacidad de movimiento. La ventana de fin de partida (Figura B.5) informa del resultado con un color que codifica el desenlace: verde para la victoria, rojo para la derrota y morado para el empate, indicando además el motivo concreto. Desde esta pantalla se puede iniciar una nueva partida pulsando *Play Again* o regresar al menú principal pulsando *Main Menu*.



Figura B.5: Pantalla de fin de partida con mensaje de victoria.

Bibliografía

- [1] Newzoo. Global games market report 2025. https://investgame.net/wp-content/uploads/2025/09/2025_Newzoo_Free_Global_Games_Market_Report.pdf, 2025. Consultado en abril de 2026.
- [2] Marc Palaus, Raquel Viejo-Sobera, Diego Redolar-Ripoll, and Elena M. Marrón. Cognitive enhancement via neuromodulation and video games: Synergistic effects? *Frontiers in Human Neuroscience*, <https://doi.org/10.3389/fnhum.2020.00235>, 2020. Consultado en abril de 2026.
- [3] Alejo García-Naveira, Martín Jiménez Toribio, Borja Teruel Molero, and Alejandro Suárez. Beneficios cognitivos, psicológicos y personales del uso de los videojuegos y esports: una revisión. *Revista de Psicología Aplicada al Deporte y al Ejercicio Físico*, <https://www.revistaspsicologiaaplicadadeporteyejercicio.org/art/rpadef2018a15>, 2018. Consultado en abril de 2026.
- [4] British Museum. The royal game of ur. https://www.britishmuseum.org/collection/object/W_1928-1009-378.
- [5] J. M. Sadurní. El juego real de ur, un entretenimiento en la antigua mesopotamia. https://historia.nationalgeographic.com.es/a/el-juego-real-de-ur-un-entretenimiento-en-la-antigua-mesopotamia_19167.
- [6] Brooklyn Museum. Gaming board inscribed for amenhotep iii with separate sliding drawer. https://web.archive.org/web/20160307212542/https://www.brooklynmuseum.org/opencollection/objects/3536/Gaming_Board_Inscribed_for_Amenhotep_III_with_Separate_Sliding_Drawer.
- [7] Danielle Zwang. Senet and twenty squares: Two board games played by ancient egyptians. <https://www.metmuseum.org/es/perspectives/ancient-egypt-board-games>.
- [8] Colin Stapczynski. History of chess | from early stages to magnus. <https://www.chess.com/article/view/history-of-chess>.
- [9] Chess Variants. Dou shou qi (animal chess). <https://www.chessvariants.org/other.dir/animal.html>.
- [10] British Antique Dealers' Association. L'attaque game. <https://www.bada.org/object/la-attaque-game>.

- [11] History of stratego. <https://www.ultraboardgames.com/stratego/history.php>.
- [12] Ancient Chess. How to play the chinese land battle game luzhanqi. <https://ancientchess.com/page/play-luzhanqi.htm>.
- [13] David Kushner. Masters of doom: How two guys created an empire and transformed pop culture. Random House, 2003. Reseña y ficha accesible en https://www.goodreads.com/book/show/222146.Masters_of_Doom.
- [14] Godot Engine Contributors. Godot engine documentation. <https://docs.godotengine.org>, 2024. Consultado en 2024.
- [15] Stanford Encyclopedia of Philosophy. Game theory. <https://plato.stanford.edu/entries/game-theory/>, 2023. Consultado en mayo de 2026. Recoge los fundamentos de la teoría de juegos y el principio Minimax formulados por von Neumann y Morgenstern (1944).
- [16] Eric Roberts. Alpha-beta pruning — intelligent search. Stanford University, Computer Science, 2004. <https://cs.stanford.edu/people/eroberts/courses/soco/projects/2003-04/intelligent-search/alphabeta.html>. Consultado en mayo de 2026. Describe el algoritmo Alpha-Beta Pruning desarrollado por Knuth y Moore (1975).
- [17] Murray Campbell, A. Joseph Hoane, and Feng-hsiung Hsu. Deep blue. ScienceDirect, Artificial Intelligence, vol. 134, 2002. <https://www.sciencedirect.com/science/article/pii/S0004370201001291>.
- [18] Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. Semantic Scholar, 2006. <https://www.semanticscholar.org/paper/Efficient-Selectivity-and-Backup-Operators-in-Tree-Coulom/02cc6a5944d57d2353a55639c7b77336b94f29b6>. Consultado en mayo de 2026.
- [19] David Silver, Aja Huang, Chris J. Maddison, et al. Mastering the game of go with deep neural networks and tree search. Nature, vol. 529, 2016. <https://www.nature.com/articles/nature16961>.
- [20] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. Nature, vol. 518, 2015. <https://www.nature.com/articles/nature14236>.
- [21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017. <https://arxiv.org/abs/1707.06347>.
- [22] Greg Brockman, Vicki Cheung, Ludwig Pettersson, et al. Openai gym. arXiv:1606.01540, 2016. <https://arxiv.org/abs/1606.01540>.
- [23] Mark Towers, Jordan K. Terry, Ariel Kwiatkowski, et al. Gymnasium. Zenodo, 2023. <https://zenodo.org/record/8127025>.

- [24] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, vol. 22, n.º 268, 2021. <https://jmlr.org/papers/v22/20-1364.html>.
- [25] Pedro Cabalar. Tema 6: El proceso unificado (UP / RUP). <https://www.dc.fi.udc.es/~cabalar/md/t6.pdf>, 2021. Consultado en mayo de 2026.
- [26] Erick Solano-Fernández and Diego Porras-Alfaro. El modelo iterativo e incremental para el desarrollo de la aplicación de realidad aumentada Amón_RA. *Tecnología en Marcha*, vol. 33, n.º 5, https://revistas.tec.ac.cr/index.php/tec_marcha/article/download/5518/5233/15939, 2020. Consultado en abril de 2026.
- [27] Epic Games. Unreal engine 5 – features. <https://www.unrealengine.com/unreal-engine-5>, 2025. Consultado en abril de 2026.
- [28] Game Developer staff. Unity apologizes to devs, reveals updated Runtime Fee policy. *Game Developer*, <https://www.gamedeveloper.com/business/unity-apologizes-to-devs-reveals-updated-runtime-fee-policy>, 2023. Consultado en abril de 2026.
- [29] Unity Technologies. A message to our community: Unity is canceling the Runtime Fee. *Unity Discussions*, <https://discussions.unity.com/t/a-message-to-our-community-unity-is-canceling-the-runtime-fee/1517714>, 2024. Consultado en abril de 2026.
- [30] Unity Technologies. Unity – real-time development platform. <https://unity.com/solutions/game>, 2025. Consultado en abril de 2026.
- [31] Godot Engine. Godot engine – features. <https://godotengine.org/features/>, 2025. Consultado en abril de 2026.
- [32] Tyler Wilde. Slay the spire 2 ditched Unity for open-source engine Godot after over 2 years of development. *PC Gamer*, <https://www.pcgamer.com/games/card-games/slay-the-spire-2-ditched-unity-for-open-source-engine-godot-after-2-years-of-development/>, 2024. Consultado en abril de 2026.
- [33] David Cailleteau. Slay the spire 2 had a massive launch with an estimated 4.6M copies sold, over \$92M in revenue so far. *Wccftech*, <https://wccftech.com/slay-the-spire-2-estimated-4-6-million-copies-sold-92-million-revenue-generated/>, 2026. Consultado en abril de 2026.
- [34] Glassdoor. Sueldo: Junior game developer en españa. https://www.glassdoor.es/Sueldos/junior-game-developer-sueldo-SRCH_K00,21.htm, 2025. Consultado en abril de 2026.
- [35] Ministerio de Inclusión, Seguridad Social y Migraciones. Orden PJC/297/2026, de 30 de marzo, por la que se desarrollan las normas legales de cotización a la Seguridad Social, desempleo, protección por cese de actividad, Fondo de Garantía Salarial y formación profesional para el ejercicio 2026. *Boletín Oficial del Estado*, núm. 77, https://www.boe.es/diario_boe/txt.php?id=BOE-A-2026-7296, 2026. Tipos a cargo de la empresa en Régimen General (contrato indefinido): contingencias comunes 23,60 %, desempleo 5,50 %, FOGASA

0,20 %, formación profesional 0,60 %, MEI 0,75 %; total aproximado ≈ 32 % incluyendo accidentes de trabajo y enfermedades profesionales.

- [36] Change Vision, Inc. Pricing for astah software. <https://astah.net/pricing/>, 2025. Consultado en 2025.
- [37] Change Vision, Inc. Astah professional — Team Licensing Options. <https://astah.net/pricing/team/>, 2026. Licencia flotante anual para 1 usuario: 180 €/año.
- [38] Notion Labs, Inc. Notion pricing plans. <https://www.notion.so/pricing>, 2025. Consultado en 2025.
- [39] OCU. Precio de la luz: seguimiento mensual de la factura eléctrica. <https://www.ocu.org/vivienda-y-energia/gas-luz/informe/precio-luz>, 2026. Factura media mensual en tarifa regulada PVPC durante 2025: 69,34 €/mes.
- [40] Digi Spain Telecom, S.L.U. Tarifas de fibra óptica — Digi España. <https://www.digimobil.es/fibra>, 2026. Tarifa de fibra óptica básica (1 Gb simétrico) sin servicios adicionales: 20,00 €/mes.
- [41] Coworking Inspira. Tarifas coworking y el salto a tu propio espacio de trabajo. <https://www.coworkinginspira.es/tarifas-coworking-y-el-salto-a-tu-propio-espacio-de-trabajo/>, 2024.
- [42] SKEPP. El alquiler de oficinas para las empresas pequeñas: ¿qué deben considerar? <https://skepp.com/es/blog/consejos/alquiler-oficina-empresa-pequena>.
- [43] Sage. Contabilizando la amortización de los equipos informáticos: pasos a seguir. <https://www.sage.com/es-es/blog/contabilizando-la-amortizacion-de-los-equipos-informaticos-pasos-a-seguir/>, 2026.
- [44] Valve Corporation. Steam direct — distributing on steam. https://partner.steamgames.com/doc/finance/payments_salesreporting, 2024. Consultado en 2024.
- [45] Valve Corporation. Incorporación a steamworks. <https://partner.steamgames.com/doc/gettingstarted/onboarding?l=spanish>.
- [46] Quipu. ¿cuánto cuesta crear una sociedad limitada en España? <https://getquipu.com/blog/cuanto-cuesta-crear-sl/>, 2024.
- [47] Abogados Mercantiles. ¿cuánto cuesta montar una sociedad en España en 2025? <https://abogadosmercantiles.es/cuanto-cuesta-montar-una-sociedad-en-espana-2025/>, 2025.
- [48] Project Management Institute. Contingency reserve — pmi lexicon of project management terms. <https://www.pmi.org/pmbok-guide-standards/lexicon>, 2024. Consultado en 2024.